

1. INTRODUÇÃO

- **Algoritmos** são o cerne da computação. Um programa codifica um algoritmo de modo a ser executado em um computador, resolvendo assim um problema.
- É fundamental que um **programa** produza a solução com dispêndio de tempo e de memória razoáveis. Daí se deve a importância de projeto e análise de algoritmos.
- Assim, este capítulo introdutório procura ressaltar a importância de **complexidade** e de métodos de projeto e análise de algoritmos.

Visão geral do capítulo

- 1.1 A complexidade no desempenho de algoritmos.

Ilustra a importância da complexidade, situando seu impacto nos problemas que podem ser resolvidos em tempo razoável.

- 1.2 A complexidade de problemas e algoritmos.

Apresenta e motiva os principais tópicos a serem tratados.

- 1.3 Revisão matemática.

Revê algumas fórmulas matemáticas úteis no cálculo de complexidade de algoritmos.

1.1 A COMPLEXIDADE NO DESEMPENHO DE ALGORITMOS

- Um **algoritmo** é um procedimento, consistindo de um conjunto de regras não ambíguas, as quais especificam, para cada entrada, uma **seqüência finita de operações**, terminando com uma saída correspondente.
- Um algoritmo resolve um problema quando, para qualquer entrada, produz uma **resposta correta**, se forem concedidos **tempo e memória suficientes** para sua execução.
- Os recursos de **espaço** e **tempo** requeridos têm grande importância em **casos práticos**.
- Às vezes, o algoritmo mais imediato está longe de ser razoável em termos de **eficiência**.

- A tabela 1.1.1 mostra o **desempenho** desses dois algoritmos que calculam o determinante de uma matriz $n \times n$, considerando tempos de operações de um computador real.

n	Método de Cramer	Método de Gauss
2	22 μ s	50 μ s
3	102 μ s	159 μ s
4	456 μ s	353 μ s
5	2,35 ms	666 μ s
10	1,19 min	4,95 ms
20	15 225 séculos	38,63 ms
40	$5 \cdot 10^{33}$ séculos	0,315 s

Tabela 1.1.1 – Tamanho do Problema x Tempo de Execução

- O crescente avanço tecnológico, permitindo a criação de máquinas cada vez mais rápidas, pode parecer ofuscar a importância da **complexidade de tempo** de um algoritmo.

- Para ilustrar, considera-se o impacto de um **aumento de velocidade** de operação sobre alguns algoritmos, considerando uma segunda máquina dez vezes mais rápida que a primeira.
- Dados algoritmos com complexidades $\log_2 n$, n , $n \cdot \log_2 n$, n^2 , n^3 , 2^n , 3^n e um tempo t , a tabela 1.1.2 dá o **tamanho máximo** de problema resolvido por cada algoritmo no tempo t , na 1ª máquina e na 2ª dez vezes mais rápida.

Complexidade de Tempo	Tamanho máximo de problema resolvível na máquina lenta	Tamanho máximo de problema resolvível na máquina rápida
$\log_2 n$	x_0	$(x_0)^{10}$
n	x_1	$10x_1$
$n \cdot \log_2 n$	x_2	$10x_2$ (p/ x_2 grande)
n^2	x_3	$3,16x_3$
n^3	x_4	$2,15x_4$
2^n	x_5	$x_5 + 3,3$
3^n	x_6	$x_6 + 2,096$

Tabela 1.1.2 - Complexidade do algoritmo x Tamanho máximo de problema resolvível

- Supondo-se que um computador execute uma operação em $1\mu s$, chega-se à tabela 1.1.3, que descreve os **tamanhos limites** de problemas resolvíveis por algoritmos de diferentes complexidades em 1 segundo, 1 minuto e 1 hora.

Complexidade de tempo	Tamanho de problema executável em:		
	1 segundo	1 minuto	1 hora
$\log_2 n$	$2 \cdot 10^6$	$2^{6 \cdot 10^7}$	$2^{3,6 \cdot 10^9}$
n	10^6	$6 \cdot 10^7$	$3,6 \cdot 10^9$
$n \cdot \log_2 n$	62 746	$2,8 \cdot 10^6$	$1,3 \cdot 10^8$
n^2	10^3	$7,746 \cdot 10^3$	60 000
n^3	10^2	$3,9 \cdot 10^2$	$1,5 \cdot 10^3$
2^n	20	25	32
3^n	13	16	20

Tabela 1.1.3 - Complexidade do algoritmo x Tempo de execução

- As tabelas 1.1.1, 1.1.2 e 1.1.3 mostram que a **capacidade de um algoritmo**, é função de sua **complexidade**.

1.2 COMPLEXIDADE DE PROBLEMAS E ALGORITMOS

- A complexidade vem ganhando destaque a ponto de que afirmações, como a de Aho [AHU 74], de que a **complexidade** é o **coração da Computação**, já não surpreendem.

1.2.1 Metodologia de cálculo da complexidade

- A análise da complexidade de um algoritmo é realizada, usualmente, de maneira muito particular. Alguns conceitos são gerais, no sentido de que só dependem da **estrutura do algoritmo**. Esses conceitos são apresentados nos capítulos 3 e 4.

1.2.2 Complexidade na fase de projeto do algoritmo

- O estudo das **técnicas** de desenvolvimento de algoritmo permite prever alguns aspectos da complexidade do algoritmo resultante.
- O **projeto** de algoritmos eficientes é sempre um propósito de qualquer projetista e é uma preocupação já nessa fase.
- No capítulo 5, alguns **métodos** de desenvolvimento de algoritmos são apresentados, e a complexidade dos algoritmos gerados é analisada.

1.2.3 A complexidade do problema e a intratabilidade

- A complexidade também pode ser vista como uma **propriedade do problema**, o que significa dar uma medida independente do tratamento dado ao problema. Alguns problemas são “bem comportados” e permitem chegar a limites de complexidade.
- Outros problemas parecem ficar tão **difíceis** de serem resolvidos para instâncias grandes que parecem tornar-se **intratáveis**. Esses problemas são apresentados no capítulo 6.

1.3 REVISÃO MATEMÁTICA

- No **cálculo da complexidade** de um algoritmo, é necessário o uso de conceitos e resultados matemáticos.

- Soma dos termos de uma progressão aritmética:

$$\sum_{i=0}^n a_i = \frac{(a_0 + a_n) \cdot (n+1)}{2} \quad \text{com } a_i = a_0 + i \cdot n.$$

- Soma dos termos de uma progressão geométrica:

$$\sum_{i=0}^{n-1} a \cdot q^i = \frac{a \cdot (q^n - 1)}{q - 1} \quad \text{para } q \neq 1.$$

- Binômio de Newton:

$$(a + b)^n = \sum_{i=0}^n \binom{n}{i} a^{n-i} \cdot b^i$$

- Outras somas:

$$\sum_{i=0}^n i^2 = \frac{1}{6} \cdot n \cdot (n+1) \cdot (2n+1)$$

$$\sum_{i=0}^n i \cdot 2^i = 2 + (n-1) \cdot 2^{n+1}$$

- Combinação:

$$\binom{n}{k} = \frac{n!}{(n-k)! \cdot k!}$$

- Somas combinatoriais também são comuns em cálculos de complexidade:

$$\sum_{i=0}^n \binom{n}{k} \cdot (x-1)^k = x^n$$

- Logaritmos também são funções comuns no estudo da complexidade. Por definição,

$$a^{\log_a n} = n$$

- Propriedades:

$$\log_a (n \cdot m) = \log_a n + \log_a m$$

$$\log_a \left(\frac{n}{m}\right) = \log_a n - \log_a m$$

$$\log_a (n^m) = m \cdot \log_a n$$

$$a^{\log_c b} = b^{\log_c a}$$

$$\log_a n = \log_b n + \log_a b$$

1.4 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é ressaltar a importância da complexidade e de métodos de projeto e análise de algoritmos.
- Nem sempre o algoritmo mais imediato é viável na prática.
- Algoritmos mais eficientes costumam tirar maior proveito de máquinas melhores.
- A análise de complexidade de um algoritmo pode ser feita com base em sua estrutura. Há métodos gerais de desenvolvimento de algoritmos com amplo espectro de aplicação.
- A complexidade de um problema é a do melhor algoritmo imaginável (conhecido ou não) que possa resolvê-lo.
- Fórmulas matemáticas envolvendo somatórios, como na seção 3, são úteis para a análise de complexidade de algoritmos.

2. CONCEITOS BÁSICOS

- Existem vários **critérios de avaliação** de um algoritmo como: quantidade de trabalho requerido, quantidade de espaço requerido, simplicidade, exatidão de resposta e otimalidade.
- Este capítulo introduz os conceitos básicos de complexidade e **análise assintótica**, ilustrando

Visão geral do capítulo

- 2.1 Medidas de Complexidade.
Introduz as idéias de complexidade pessimista (pior caso) e complexidade média, bem como as medidas de complexidade de tempo e de espaço.
- 2.2 Critérios de Complexidade.
Introduz as idéias de operações fundamentais e de tamanho da entrada. Introduz também a idéia de comparação entre complexidades.
- 2.3 Ordens Assintóticas.
Introduz as idéias de comportamento assintótico (notações O , Ω e Θ), e examinam-se algumas de suas propriedades.
- 2.4 Exemplos.
Ilustram as idéias anteriores com alguns exemplos de algoritmos.

2.1 MEDIDAS DE COMPLEXIDADE

- A quantidade de trabalho requerido por um algoritmo não pode ser descrita simplesmente por um número, porque o número de operações básicas efetuadas em geral não é o mesmo para qualquer entrada. A expressão “quantidade de trabalho requerido” também é chamada *complexidade do algoritmo*.
- Se a complexidade é tomada como a máxima para qualquer entrada de um dado “tamanho”, a complexidade é chamada *complexidade no pior caso*. Se, entretanto, é levada em conta a probabilidade de ocorrência de cada entrada, a complexidade é chamada *complexidade média*.

2.1.1 Complexidade de Tempo

- Uma das medidas de complexidade mais importante é a *complexidade de tempo*.
- Existem vários *aspectos* a considerar na determinação da complexidade de tempo de um algoritmo. Um critério pode ser medir o tempo de execução, mas:
 - Uma pequena mudança no programa pode não representar uma mudança significativa no algoritmo, mas, apesar disso, a *velocidade de execução* pode ser afetada.
 - Além disso, se dois programas são comparados, primeiro numa máquina, depois em outra, as comparações podem dar *resultados diferentes*.
 - Este critério é fortemente dependente tanto da implementação quanto da máquina.
- Uma alternativa útil é uma *análise matemática* das dificuldades intrínsecas da resolução do problema computacionalmente.

2.1.2 Complexidade de Espaço

- A **memória** usada por um programa é dependente da implementação.
- Porém, algumas conclusões sobre o **espaço** podem ser tiradas examinando-se o algoritmo.
- Espaços como: a **área** para guardar as instruções, as constantes, as variáveis e os dados.
- Se os dados têm uma **representação óbvia** como uma matriz, são considerados para a análise de espaço somente os **espaços extras**.

2.2 CRITÉRIOS DE COMPLEXIDADE

- Para medir a quantidade de trabalho realizado por um algoritmo, é escolhida uma operação, chamada *operação fundamental*. Às vezes, é necessária mais de uma operação fundamental.
- Por exemplo, para um *algoritmo de ordenação*, uma operação fundamental natural é a *comparação entre elementos* quanto à ordem.

2.1.2 Complexidade de Algoritmo

- O tempo requerido por um algoritmo em dada entrada, pode ser medido através de suas *seqüências de execução*.
- Seja **A** um conjunto de algoritmos, **D** o conjunto de dados e **E** o conjunto de todas as seqüências de execuções de operações fundamentais.

• ***exec*** : $A \times D \rightarrow E$;

exec(*a*, *d*) := seqüência de execuções de operações fundamentais efetuadas na execução do algoritmo ***a***, com entrada ***d***.

• ***custo*** : $E \rightarrow \mathbb{IN}$;

custo(*s*) := comprimento da seqüência ***s***, definido conforme o peso estabelecido para as operações fundamentais.

• ***desemp***(***d***) := ***custo(exec(a, d))***;

O desempenho do algoritmo ***a***, na entrada ***d***, é dado por *custo(exec(a, d))*.

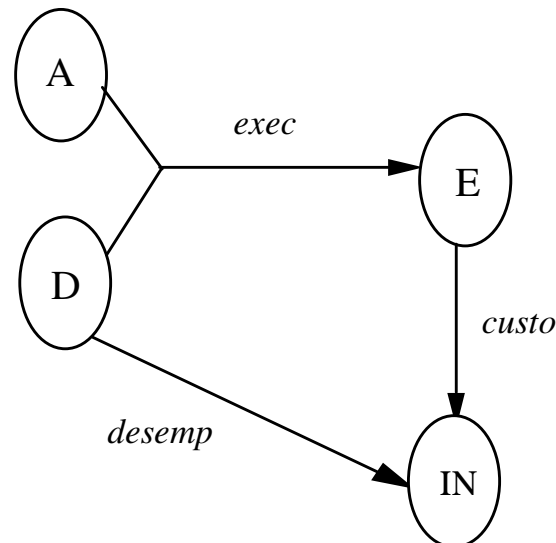


Figura 2.2.1: Desempenho do algoritmo

- Note que $desemp : D \rightarrow \mathbb{N}$ dá informação sobre cada entrada específica.
- Deseja-se condensar a informação dada pela **função desempenho** em termos das entradas de um tamanho dado.
- Para tanto, seleciona-se uma função:
 θ **$tam : D \rightarrow \mathbb{N}$** ;
 $tam(d) :=$ tamanho da entrada d , que mede o tamanho da entrada.
- De posse de tal função tam , pode-se condensar a informação dada pela função $desemp : D \rightarrow \mathbb{N}$, em uma função **$aval : \mathbb{N} \rightarrow \mathbb{N}$** tal que:
 $aval(n) = desemp(d)$ para $tam(d) = n$

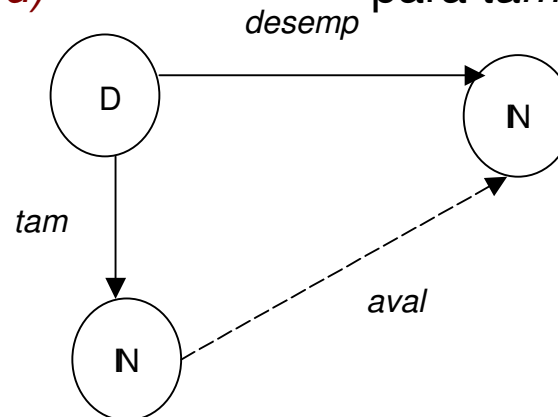


Figura 2.2.2: Avaliação de algoritmo

- Note que a função $tam : D \rightarrow \mathbb{N}$ em geral não é inversível. Costuma haver **várias entradas** com mesmo tamanho (e desempenhos distintos). **Qual delas escolher?**
- Dado $n \in \mathbb{N}$, seja: $D := \{ d / d \in D \text{ e } tam(d) = n \}$.
- Note que, em geral, a função $desemp$ não é constante sobre D . Assim, tomam-se **dois critérios**:
 - Complexidade pessimista (pior caso)
$$avalCP(n) := \text{Máx} \{ desemp(d) / d \in D \}$$
 - Complexidade média (caso médio)
$$avalCM(n) := \sum_{d \in D_n} \text{prob}(d) \cdot desemp(d) ,$$
onde $\text{prob}(d)$ dá a probabilidade de ocorrer a entrada d no conjunto D_n .

- Os critérios de complexidade que acabamos de introduzir consideram o **desempenho do algoritmo** sobre o conjunto de todas as entradas com tamanho n . Há um **critério alternativo**:

- No pior caso

$$c_{\bar{p}}(n) = \text{Máx}\{desemp(d) / tam(d) = n\}$$

- No critério relaxado

$$c_{\leq p}(n) = \text{Máx}\{desemp(d) / tam(d) \leq n\}$$

- O critério relaxado dá uma cota superior

$$c_{\bar{p}}(n) \leq c_{\leq p}(n) \quad \forall n \in \mathbb{N}$$

- A vantagem do critério relaxado é ser uma função monotônica

$$c_{\leq p}(m) \leq c_{\leq p}(n) \quad \text{para } m \leq n$$

2.2.2 Comparação entre complexidades

- Um problema pode ter **mais de um algoritmo** para resolvê-lo. **Qual deles escolher?**
- De maneira geral, consideremos um conjunto **A** de algoritmos para um dado problema Π .
- Escolhidas as operações fundamentais e a função tamanho, fixemos um **critério de complexidade**. Então, temos associado a cada algoritmo $a \in \mathbf{A}$, sua avaliação $aval : \mathbb{N} \rightarrow \mathbb{N}$. Assim temos um funcional $\mathbf{A} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$.
- Frequentemente, interessa-nos identificar um **algoritmo ótimo**. Trata-se de um algoritmo $o \in \mathbf{A}$, tal que $aval(o)$ é melhor do que $aval(a)$ para todo $a \in \mathbf{A}$.
- Essas idéias envolvem comparações entre funções **f** e **g** de $\mathbb{N}$ em $\mathbb{N}$. Quando **f** é “**melhor**” do que **g**?
- Para isso, utiliza-se o conceito de comportamento **assintótico** (entradas de tamanho grande).

2.3 ORDENS ASSINTÓTICAS

- A **complexidade assintótica** é definida pelo crescimento da complexidade para entradas suficientemente grandes.
- O algoritmo assintoticamente **mais eficiente** é melhor para todas as entradas, exceto para entradas relativamente pequenas.
- Além disso, a complexidade exata costuma conter muita informação, resultando em uma função de **difícil análise** e **manipulação**.
- Consideremos funções **f** e **g**, mapeando naturais em reais não negativos: de $\mathbb{N}$ em $\mathbb{R}_+$.
- Uma **cota assintótica superior** (CAS) é uma função que cresce mais rapidamente do que outra: está acima, a partir de certo ponto.

- Em geral, define-se **g** uma cota assintótica superior para **f** sse:

$$(\exists n_0 \in \mathbb{N})(\forall n \geq n_0)(f(n) \leq g(n)),$$

significando que, para um n suficientemente grande, **g(n)** domina **f(n)**.

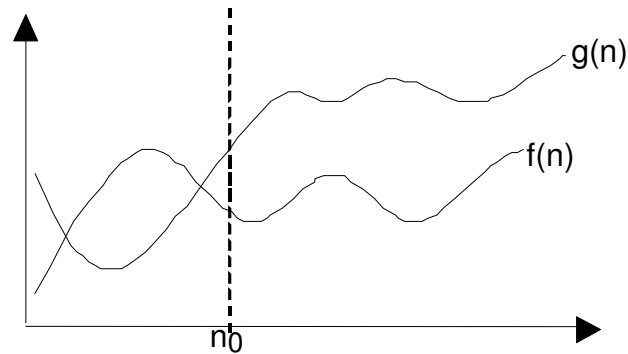


Figura 2.3.1: $g(n)$ é CAS para $f(n)$

Notação O

- A notação **O** define uma *cota assintótica superior*.
- Define-se que $f(n)$ é $O(g(n))$ sse:
para alguma constante $c \in \mathbb{R}_+$: **$c \cdot g(n)$ é CAS para $f(n)$** ,
i. e. $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (f(n) \leq c \cdot g(n))$.

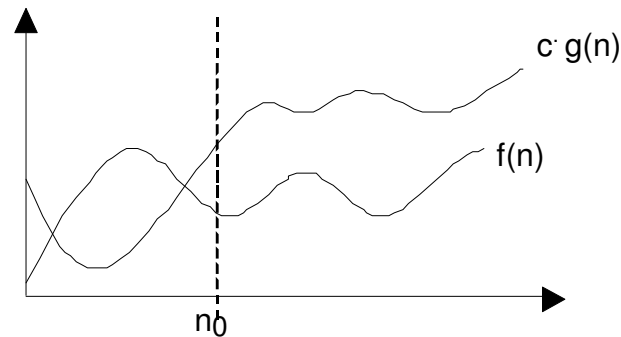


Figura 2.3.2: $f(n)$ é $O(g(n))$

Fato: Se $f(n) = a_m n^m + \dots + a_1 n + a_0$ é um polinômio de grau **m** então $|f(n)|$ é $O(n^m)$.

Demonstração:

$$\begin{aligned} |f(n)| &\leq |a_m| n^m + \dots + |a_1| n + |a_0| \\ &\leq (|a_m| + |a_{m-1}|/n + \dots + |a_0|/n^m) n^m \\ &\leq (|a_m| + |a_{m-1}| + \dots + |a_0|) n^m, \text{ para } n \geq 1. \end{aligned}$$

Então escolha $c = |a_m| + |a_{m-1}| + \dots + |a_0|$ e $n_0 = 1$ e tem-se:

$$|f(n)| \leq c n^m \text{ para todo } n \geq n_0.$$

Notação Ω

- A notação Ω define uma *cota assintótica inferior*.
- Define-se que $f(n)$ é $\Omega(g(n))$ sse:
para alguma constante $c \in \mathbb{R}_+$: $c \cdot f(n)$ é CAS para $g(n)$,
i. e. $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (g(n) \leq c \cdot f(n))$.

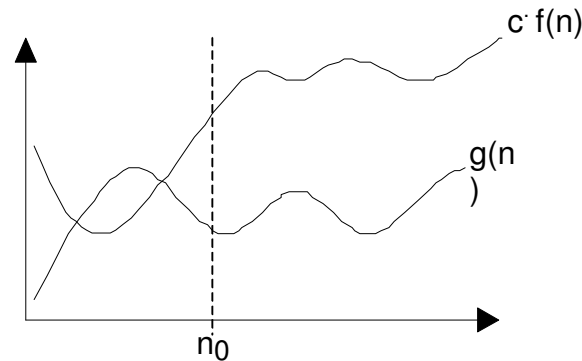


Figura 2.3.3: $f(n)$ é $\Omega(g(n))$

- Os algoritmos de $O(1)$, $O(n)$, $O(n^2)$, $\dots$ são *polinomiais*. Já 2^n é uma função não limitada por polinômio. Assim, um algoritmo de $\Omega(2^n)$, $\Omega(3^n)$, $\dots$ é dito *exponencial*.

Notação Θ

- A notação Θ define um *limite assintótico exato*.
- Diz-se que $f(n)$ é $\Theta(g(n))$.
- Define-se que $f(n)$ é $\Theta(g(n))$ sse:
 $(\exists c, c' \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (f(n) \geq c \cdot g(n) \wedge f(n) \leq c' \cdot g(n)).$

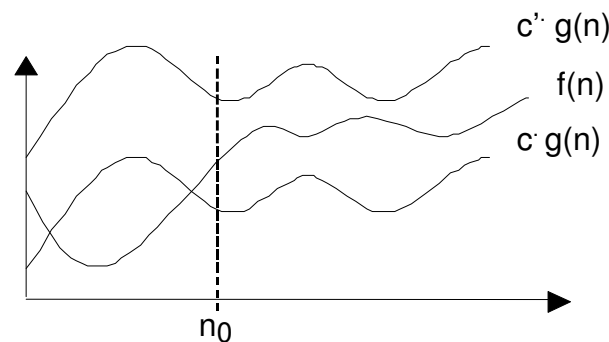


Figura 2.3.4: $f(n)$ é $\Theta(g(n))$

- Como $\log_a n = \Theta(\log_b n) \forall a, b$, para simplificar a notação muitas vezes, nos cálculos de complexidade será usado simplesmente $\log n$.

- Funções que são Θ uma da outra têm o mesmo comportamento assintótico.
- Note que a ordem Θ é um **caso particular** das ordens O e Ω , isto é, se $f(n)$ é $\Theta(g(n))$, então $f(n)$ também é $O(g(n))$ e $\Omega(g(n))$.
- Algumas **classes de comportamento assintótico** são listadas abaixo:

$$\underbrace{1 \quad \log n \quad n \quad n \log n \quad n^2 \quad n^3 \quad \dots}_{\text{polinomiais}} \quad \dots \quad \underbrace{2^n \quad 3^n \quad \dots}_{\text{exponenciais}}$$

- Nessa lista, se $f(n)$ está à esquerda de $g(n)$, então $f(n) = O(g(n))$, e $g(n)$ não é $O(f(n))$.

2.4 EXEMPLOS

- **Exemplo 2.4.1:** O problema é determinar o valor máximo de um conjunto armazenado em uma tabela **tab** de **n** valores (reais, por exemplo).
 - *Algoritmo 2.1: Máximo de tabela.*

<u>Procedimento</u> Max(tab: tabela)	<i>{máximo de tabela}</i>
<i>{Entrada: tabela tab: vetor [1..n] de Elmord.</i>	
<i>{Saída: Mx = valor máximo em tab.}</i>	
1. $Mx \leftarrow tab[1];$	<i>{candidato a máximo}</i>
2. <u>para</u> i <u>de</u> 2 <u>até</u> n <u>faça</u>	<i>{varre tabela: de 2 a n}</i>
3. <u>se</u> $Mx \leq tab[i]$ <u>então</u> $Mx \leftarrow tab[i];$	<i>{atualiza Mx}</i>
4. <u>fim-para</u> ;	<i>{2: i de 2 até n}</i>
5. <u>retorne-saída</u> (Mx);	<i>{fim do algoritmo: máximo de tabela}</i>
6. <u>fim-Procedimento</u> .	

- O algoritmo sempre tem **n-1** comparações, isto é $\Theta(n)$, como complexidade média no pior ou no melhor caso.

- **Exemplo 2.4.2:** O problema é procurar um valor (chave) em uma tabela **tab** para determinar se **ch** encontra-se ou não em **tab**.
 - *Algoritmo 2.2:* **Busca seqüencial em tabela.**

Procedimento BuscaSeq(ch: Valor, tab: tabela) {busca seqüencial em tabela}

{*Entrada*: ch: Valor, tabela tab: vetor [1..n] de Valor.

Saída: achou, pos (achou = F se ch não está em tab, senão achou = V e tab[pos] = ch).}

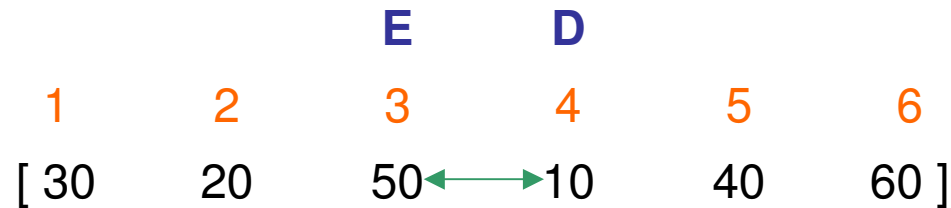
1. $i \leftarrow 0$; achou $\leftarrow$ F;	{inicialização}
2. <u>repita</u>	{varre tabela}
3. $i \leftarrow i+1$;	{incrementa índice}
4. <u>se</u> tab[i] = ch <u>então</u> achou $\leftarrow$ V;	{compara com chave}
5. <u>até</u> achou v $i = n$;	{fim do repita de 2}
6. <u>se</u> achou <u>então</u> pos $\leftarrow$ i;	{verifica se achou chave}
7. <u>retorne-saída</u> (achou, pos);	{fim do algoritmo: busca seqüencial em tabela}
8. <u>fim-Procedimento</u> .	

- **Operação fundamental**: a comparação = (no se da linha 4).
- A **relação** da complexidade com a posição de *ch* em *tab* é:
 - Se *tab* está:
 - na 1ª posição, BuscaSeq faz 1 comparação;
 - na 2ª posição, BuscaSeq faz 2 comparações;
 -;
 - na nª posição ou não está em *tab*, BuscaSeq faz n comparações.
- Complexidade no **pior caso**: $\Theta(n)$.
- Complexidade **média**: $\frac{1}{n} \cdot (1 + 2 + \dots + n) = \frac{1+n}{2}$ que é $\Theta(n)$.

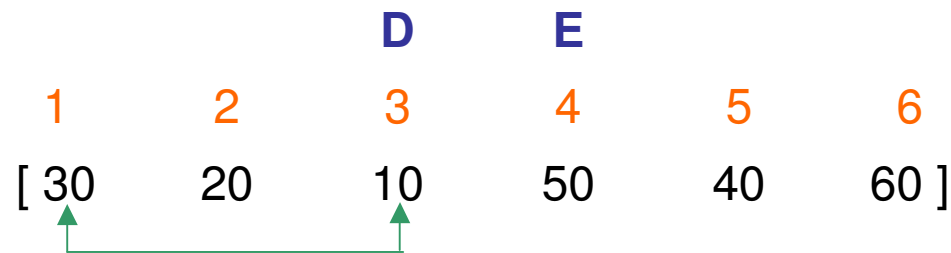
- **Exemplo 2.4.3:** O algoritmo é: dado um **pivô**, particionar os **elementos** das posições m até k de um vetor, movimentando-os de tal forma que no final, todos os elementos à direita do pivô serão maiores ou iguais a ele, enquanto os à sua esquerda serão menores que ele.
- O algoritmo usa dois apontadores que caminham em direções opostas:



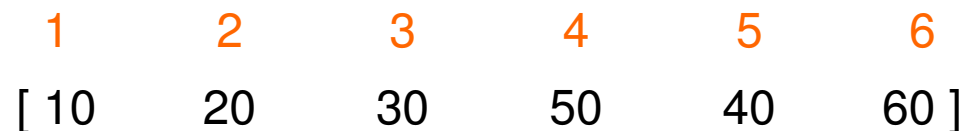
- Quando o apontador da esquerda encontra um elemento menor que o pivô e o apontador da direita, um elemento maior que o pivô, então eles são trocados:



- Quando o apontador da esquerda alcança ou ultrapassa o pivô da direita, o apontador é colocado na posição:



- Saída:



○ Algoritmo 2.3: **Particionamento**.

Procedimento Particionamento(A: vetor, m, k: Elmord) *{particionar vetor}*

{Entrada: vetor A: vetor [1..n] de Elmord, m, k.

Saída: (j, A), o vetor A com os elementos rearranjados de tal forma que todos os elementos inferiores a A[m] estarão à esquerda da posição j.}

```

1.  $v \leftarrow A[m]; i \leftarrow m+1; j \leftarrow k;$            {v é pivô; i apontador da esq.; j apontador da dir.}
2. repita                                           {2 a 6: percorre a nas duas direções}
3.   enqto  $A[i] < v$  faça  $i \leftarrow i+1;$            {elem. da esq. é < pivô, move apontador}
4.   enqto  $A[j] \geq v$  faça  $j \leftarrow j-1;$          {elem. da dir. é  $\geq$  pivô, move apontador}
5.   se  $i < j$  então troca( $A[i]$  e  $A[j]$ )             {A[i] maior e A[j] menor que pivô}
6.      $i \leftarrow i + 1;$ 
7.      $j \leftarrow j + 1;$ 
8.   fim-se;
9. até  $i > j;$                                            {2: todo a foi percorrido}
10.  $A[m] \leftarrow A[j]; A[j] \leftarrow v;$            {na posição j está o elem. < pivô mais à esquerda}
11. retorne-saída(j, A);                               {fim do algoritmo: particionar vetor}
12. fim-Procedimento.
```

2.5 RESUMO DO CAPÍTULO

- O objetivo deste capítulo é introduzir e ilustrar os conceitos básicos de complexidade (medidas e critérios) e de análise assintótica.
- As principais medidas de complexidade são tempo e espaço, relacionadas à velocidade e quantidade de memória. A complexidade depende da entrada particular: os principais critérios são pior caso e caso médio.
- A complexidade é determinada com base em operações fundamentais e no tamanho da entrada.
- Análise matemática fornece a complexidade intrínseca do algoritmo. A complexidade assintótica dá o desempenho para entradas de tamanho grande.
- As principais notações de comportamento assintótico dão limites superior (O), inferior (Ω) e exato (Θ), a menos de constantes multiplicativas.
- Algoritmos polinomiais têm complexidade $O(n^k)$; os com complexidade $\Omega(2^n)$ são exponenciais.

3. ANÁLISE DE COMPLEXIDADE PESSIMISTA

- O processo de construção de um algoritmo seria mais eficaz se a complexidade fosse um fator integrante do **projeto** do mesmo.
- Apesar de a **complexidade** ser uma **medida muito particular** à classe do algoritmo que está sendo analisado, alguns aspectos do cálculo da complexidade não dependem do que faz o algoritmo.
- Este capítulo introduz uma metodologia para analisar a **complexidade pessimista** (i. e. no pior caso) de um algoritmo com base em sua estrutura.
- Entre as medidas de complexidade, a **complexidade no pior caso** é o critério de avaliação mais utilizado.

- Serão estudadas, uma a uma, as principais *estruturas algorítmicas* a seguir:
 - *Atribuição*: $\mathbf{v} \leftarrow \mathbf{e}$,
 - *Seqüência* (ou composição): $\mathbf{S} ; \mathbf{T}$,
 - *Condicional*: se $\mathbf{b}$ então $\mathbf{S}$ senão $\mathbf{T}$ (ou se $\mathbf{b}$ então $\mathbf{S}$),
 - *Iteração definida* (ou incondicional): para $\mathbf{i}$ de $\mathbf{j}$ até $\mathbf{m}$ faça $\mathbf{S}$,
 - *Iteração indefinida* (ou condicional): enqto $\mathbf{b}$ faça $\mathbf{S}$.
- Na seção 2 do capítulo 2 (cf. 2.2.1), introduzimos as seguintes variantes para o critério pessimista de complexidade de um algoritmo $\mathbf{a}$:

$$c_{\overline{\mathbf{p}}}[\mathbf{a}](n) := \text{Máx} \{ \text{desemp}[\mathbf{a}](d) \in \text{IN} / \text{tam}(d) = n \},$$

$$c_{\mathbf{p}}^{\leq}[\mathbf{a}](n) := \text{Máx} \{ \text{desemp}[\mathbf{a}](d) \in \text{IN} / \text{tam}(d) \leq n \};$$
 onde $\text{desemp}[\mathbf{a}](d) := \text{custo}(\text{exec}[\mathbf{a}](d))$.
- Neste capítulo daremos ênfase à variante $c_{\mathbf{p}}^{\leq}[\mathbf{a}](n)$, entretanto vários resultados podem ser adaptados para a variante $c_{\overline{\mathbf{p}}}[\mathbf{a}](n)$.

Visão geral do capítulo

- 3.1 Preliminares e conceitos auxiliares.

Introduz algumas idéias básicas importantes e conceitos auxiliares, como os de absorção e máximo assintótico em ordem.

- 3.2 Equações de complexidade pessimista.

Estuda a complexidade de cada uma das principais estruturas algorítmicas, estabelecendo equações, ilustradas por exemplos simples.

- 3.3 Exemplos de aplicação.

Ilustra a metodologia da seção 3.2, aplicando-a a alguns exemplos de algoritmos.

3.1 PRELIMINARES E CONCEITOS AUXILIARES

- A motivação é reduzir o desempenho - e a complexidade - de um algoritmo a informações sobre suas partes.
- Consideremos funções **f** e **g** de um conjunto **D** no conjunto $\mathbb{R}_+$. Definimos:
soma pontual por: $(f + g)(d) := f(d) + g(d)$, para cada $d \in D$.
- O **máximo e mínimo pontuais**, respectivamente, por:
 $\text{Máx}(f, g)(d) := \text{Máx}\{f(d), g(d)\}$ e $\text{mín}(f, g)(d) := \text{mín}\{f(d), g(d)\}$.

3.1.1 Idéias básicas: componentes de algoritmos

- A complexidade de um algoritmo poderá ser obtida a partir das complexidades das suas partes.
- Em geral, cada **componente executada** dá sua contribuição para o desempenho e a complexidade do algoritmo.
- Torna-se conveniente classificar as componentes de um algoritmo em **conjuntivas** ou **disjuntivas**, conforme sejam sempre executadas ou não.

- Por exemplo, no (trecho de) algoritmo
 $j \leftarrow i + 1 ; k \leftarrow i - 1 ;$
 essas duas atribuições são suas componentes conjuntivas.
- Por outro lado, no (trecho de) algoritmo
 $\text{se } i \neq j \text{ então } j \leftarrow i + 1 \text{ senão } k \leftarrow i - 1 ;$
 essas duas atribuições são suas componentes disjuntivas.
- Consideremos um algoritmo $\mathbf{a}_0$ com duas componentes $\mathbf{a}_1$ e $\mathbf{a}_2$.
- Para uma entrada $\mathbf{d}$ com $\text{tam}(\mathbf{d}) \leq n$, sabemos que:
 $\text{desemp}[\mathbf{a}_j](\mathbf{d}) \leq c_P^\leq [\mathbf{a}_j](n)$, para $j = 0, 1, 2$.
- Para o caso de **componentes conjuntivas**:
 $\text{desemp}[\mathbf{a}_0](\mathbf{d}) = \text{desemp}[\mathbf{a}_1](\mathbf{d}) + \text{desemp}[\mathbf{a}_2](\mathbf{d}) \quad (3.1.1).$
- Como as complexidades pessimistas dão cotas superiores para os desempenhos, temos:
 $\text{desemp}[\mathbf{a}_0](\mathbf{d}) \leq c_P^\leq [\mathbf{a}_1](n) + c_P^\leq [\mathbf{a}_2](n).$

- Assim, pela definição de **complexidade pessimista**, obtemos a seguinte **cota superior**:

$$c_P^{\leq} [a_0](n) \leq c_P^{\leq} [a_1](n) + c_P^{\leq} [a_2](n).$$

- Por outro lado, pela equação (3.1.1) os desempenhos das partes dão cotas inferiores:

$$desemp[a_j](d) \leq desemp[a_0](d), \text{ para cada } j = 1, 2.$$

- De novo, como a complexidade pessimista é cota superior para o desempenho, temos:

$$desemp[a_i](d) \leq c_P^{\leq} [a_0](n), \text{ para cada } i = 1, 2.$$

- Assim, pela definição de **complexidade pessimista**, obtemos as seguintes **cotas inferiores**:

$$c_P^{\leq} [a_j](n) \leq c_P^{\leq} [a_0](n), \text{ para cada } j = 1, 2.$$

- Portanto, da definição de **máximo pontual**, vem a seguinte **cota inferior** para $c_P^{\leq} [a_0]$:

$$\text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2])(n) \leq c_P^{\leq} [a_0](n).$$

- Considere um algoritmo $\mathbf{a}_0$, com duas componentes conjuntivas $\mathbf{a}_1$ e $\mathbf{a}_2$ ($desemp[\mathbf{a}_0](d) = desemp[\mathbf{a}_1](d) + desemp[\mathbf{a}_2](d)$, para qualquer entrada $\mathbf{d}$).

Princípio das partes conjuntivas

- Então a **complexidade pessimista** do algoritmo $\mathbf{a}_0$ tem cotas:

$$\text{Máx}(c_P^{\leq}[\mathbf{a}_1], c_P^{\leq}[\mathbf{a}_2]) \leq c_P^{\leq}[\mathbf{a}_0] \leq c_P^{\leq}[\mathbf{a}_1] + c_P^{\leq}[\mathbf{a}_2].$$

- Logo, a complexidade pessimista do algoritmo $\mathbf{a}_0$ tem ordem $\Theta(c_P^{\leq}[\mathbf{a}_1] + c_P^{\leq}[\mathbf{a}_2])$.

- Consideremos agora um algoritmo $\mathbf{a}_0$ com duas componentes disjuntivas $\mathbf{a}_1$ e $\mathbf{a}_2$.
- Neste caso, temos esforços computacionais associados às execuções ora da componente $\mathbf{a}_1$, ora da componente $\mathbf{a}_2$.
- Assim, para cada entrada $\mathbf{d}$, temos:

$$desemp[\mathbf{a}_0](\mathbf{d}) = desemp[\mathbf{a}_1](\mathbf{d}) \text{ ou } desemp[\mathbf{a}_0](\mathbf{d}) = desemp[\mathbf{a}_2](\mathbf{d}) \quad (3.1.2).$$
- Como as complexidades pessimistas dão cotas superiores para os **desempenhos**, temos:

$$desemp[\mathbf{a}_0](\mathbf{d}) \leq c_P^{\leq} [\mathbf{a}_1](\mathbf{n}) \text{ ou } desemp[\mathbf{a}_0](\mathbf{d}) \leq c_P^{\leq} [\mathbf{a}_2](\mathbf{n}).$$
- Assim, pela definição de **máximo pontual**, obtemos a seguinte **cota superior**:

$$desemp[\mathbf{a}_0](\mathbf{d}) \leq \text{Máx} (c_P^{\leq} [\mathbf{a}_1], c_P^{\leq} [\mathbf{a}_2])(\mathbf{n}).$$
- Portanto, pela definição de **complexidade pessimista**, obtemos a seguinte **cota superior**: $c_P^{\leq} [\mathbf{a}_0](\mathbf{n}) \leq \text{Máx} (c_P^{\leq} [\mathbf{a}_1], c_P^{\leq} [\mathbf{a}_2])(\mathbf{n}).$

- Por outro lado, como a complexidade pessimista é cota superior para o desempenho, da expressão (3.1.2) temos:
 se $desemp[a_0](d) = desemp[a_1](d)$, então: $desemp[a_1](d) \leq c_P^{\leq}[a_0](n)$;
 se $desemp[a_0](d) = desemp[a_2](d)$, então: $desemp[a_2](d) \leq c_P^{\leq}[a_0](n)$.
- Assim, para qualquer entrada d com $tam(d) \leq n$, temos a seguinte cota inferior:

$$\text{Máx} \{ desemp[a_1](d), desemp[a_2](d) \} \leq c_P^{\leq}[a_0](n).$$
- Logo, para o **pior caso** de entradas com tamanho até n , temos a **cota inferior**:

$$\text{Máx} \{ \text{Máx} \{ desemp[a_1](d), desemp[a_2](d) \} / tam(d) \leq n \} \leq c_P^{\leq}[a_0](n).$$
- Mas, pode-se mostrar que $\text{Máx} \{ c_P^{\leq}[a_1](n), c_P^{\leq}[a_2](n) \}$ é uma cota inferior para:

$$\text{Máx} \{ \text{Máx} \{ desemp[a_1](d), desemp[a_1](d) \} / tam(d) \leq n \}.$$

- Portanto, a definição de **máximo pontual**, vem a ser **cota inferior** para $c_P^{\leq} [a_0]$:

$$\text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2])(n) \leq c_P^{\leq} [a_0](n).$$
- Considere um algoritmo a_0 , cujas duas componentes a_1 e a_2 são disjuntivas, de modo que $(\text{desemp}[a_0](d) = \text{desemp}[a_1](d) + \text{desemp}[a_2](d))$, para qualquer entrada d).

Princípio das partes disjuntivas

- A **complexidade pessimista** do algoritmo a_0 tem cotas:

$$c_P^{\leq} [a_0] = \text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2]).$$

- Logo, a complexidade pessimista das componentes a_1 e a_2 têm ordens $\Omega(c_P^{\leq} [a_0])$.

3.2.2 Conceitos auxiliares

- Os conceitos de absorção e máximo assintótico (em ordem) são motivados por idéias de reduzir a complexidade de um algoritmo às ordens das suas partes.
- A idéia de uma componente de um algoritmo **absorver outra** se aplica quando ambas as componentes são **conjuntivas**. Para o caso de componentes **disjuntivas**, aparece a idéia de **máximo assintótico**, que será considerada em seguida.
- Uma complexidade é **absorvida** por outra, por exemplo, se ambas são polinômios na mesma variável. Nesse caso, a complexidade de menor grau é absorvida. Assim: n é absorvida por n^2 ; também $n \cdot \log n$ é absorvida por n^2 .

- Dadas funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, dizemos que **f** é absorvida por **g**, sse, **f** é $O(g)$. Isto significa que:
 $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0): f(n) \leq c \cdot g(n)$.

Princípio da absorção

- Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, se **f** é absorvida por **g**, sua **soma pontual**:
f + g é $\Theta(g)$.
- Assim, como $n \cdot \log n$ é absorvida por n^2 , a soma **($n \cdot \log n + n^2$)** é $\Theta(n^2)$.

Máximo assintótico

- Considerando funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, vamos a seguir examinar algumas expressões mais simples para a ordem do **máximo pontual** $\text{Máx} (f, g)(n) := \text{Máx} \{ f(n), g(n) \}$.
- Frequentemente, a partir de certo ponto, $f(n)$ **fica sempre maior** do que $g(n)$, ou vice-versa.
Mais geralmente: se **f** é $O(g)$, então $\text{Máx} (f, g)$ também é $O(g)$.
Exemplo: como $2n + 3$ é $O(n^2)$, então $\text{Máx} (2n + 3, n^2)$ é $O(n^2)$.
- Pode acontecer, no entanto, que **não exista uma dominância**. Quando isso acontece, uma solução simplista, mas muitas vezes usada, é utilizar para a ordem do máximo pontual $\text{Máx} (f, g)$ a **soma pontual**, dada por $(f + g)(n) := f(n) + g(n)$.
- Consideremos, por sua vez, funções **f** e **g** de $\mathbb{N}$ em $\mathbb{N}$. O que devemos esperar de seu **máximo assintótico** $\text{MxAO} (f, g)$?

- Inicialmente, é claro que o máximo assintótico deve ser **comutativo**.

(i): $\text{MxAO} (f, g) = \text{MxAO} (g, f)$.

- Devemos também requerer que as funções sejam **dominadas**.

(ii): $f = O(\text{MxAO} (f, g))$ e $g = O(\text{MxAO} (f, g))$.

- Para ter uma cota menos folgada podemos exigir que o máximo assintótico seja a melhor **cota superior**.

(iii): $\text{MxAO} (f, g) = O(h)$, sempre que $f = O(h)$ e $g = O(h)$.

- Para tratar melhor o caso de dominância de uma das funções podemos requerer também a seguinte propriedade.

(iv): $\text{MxAO} (f, g) = g$, sempre que $(\exists n_0 \in \mathbb{N}) (\forall n \geq n_0): f(n) \leq g(n)$.

3.2 EQUAÇÕES DE COMPLEXIDADE PESSIMISTA

- Precisamos das complexidades das componentes básicas e saber como combinar complexidades de componentes.
- Vamos analisar a complexidade de cada uma das principais **estruturas algorítmicas**.
- Daremos ênfase à variante $c_p^{\leq} [a]$, cuja notação simplificaremos para $c_p [a]$. Entretanto, vários resultados podem ser adaptados para a variante $c_p^{\equiv} [a]$.

3.2.1 Atribuição

Exemplo 3.2.1. Considere as atribuições a seguir:

- a) Para variáveis inteiras i e j :
- | | |
|------------------|------------------|
| $i \leftarrow 0$ | {inicialização} |
| $j \leftarrow i$ | {transferência}. |

b) Para lista v de inteiros e variável inteira m :
 $m \leftarrow \text{Max}(v)$ {valor máximo}.

Esta atribuição envolve (sendo n o comprimento da lista em v):

- determinar o máximo da lista v , com complexidade $\Theta(n)$;
- transferir este valor, com complexidade $\Theta(1)$.

c) Para listas u , v e w :

$u \leftarrow v$ {transfere lista}
 $w \leftarrow \text{Reversa}(v)$ {inverte lista}.

A atribuição de transferência de cada elemento da lista v , para uma lista u com comprimento n , tem complexidade $O(n)$.

A atribuição $w \leftarrow \text{Reversa}(v)$ envolve:

- inverter a lista v , com complexidade $\Theta(n)$;
- transferir os elemento da lista invertida, com complexidade $\Theta(n)$.

- Assim, o desempenho da atribuição $v \leftarrow e$, com entrada d é dado por:

$$desemp[v \leftarrow e](d) = \text{cálcl}[e](d) + \text{transf}[e(d)].$$

- Dada uma atribuição $v \leftarrow e$, sejam $c_P [e]$ a complexidade de avaliação da expressão e e $c_P [\leftarrow_e]$ a complexidade de transferência do valor resultante.

Complexidade pessimista da atribuição

- A complexidade pessimista da atribuição $v \leftarrow e$ tem cotas:

$$\text{Máx} (c_P [e], c_P [\leftarrow_e]) \leq c_P [v \leftarrow e] \leq c_P [e] + c_P [e].$$

- Logo, a ordem **da complexidade pessimista** da atribuição $v \leftarrow e$ é dada por:

$$c_P [v \leftarrow e] = O(c_P [e] + c_P [e]).$$

Exemplo 3.2.2. Para listas u e v de inteiros, temos:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(c_P [\text{Ordene}] + c_P [\leftarrow_{\text{Ordene}(v)}]).$$

- Supondo que Ordene é um algoritmo com **tempo** $O(n^2)$, teremos:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(n^2 + n) = O(n^2).$$

- Note que, se a operação fundamental é a **comparação**, então:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(c_P [\text{Ordene}]).$$

3.2.2 Seqüência (ou composição)

Exemplo 3.2.3. Considere as seqüências a seguir:

- a) Para variáveis inteiras i e j :

$$i \leftarrow 0 ; j \leftarrow i.$$

b) Para lista v de inteiros e variável inteira m :

$m \leftarrow \text{Max}(v)$; $m \leftarrow m + 1$.

c) Para listas u , v e w :

$u \leftarrow v$; $w \leftarrow \text{Reversa}(v)$.

- Assim, o desempenho da seqüência **S** ; **T** com entrada **d** é dado por:

$$\text{desemp}[S ; T](d) = \text{desemp}[S](d) + \text{desemp}[T](S(d)).$$

- Note que a execução de **S** pode **alterar o tamanho dos dados**. Isto pode ocorrer, por exemplo, no caso em que **S** e **T** atuam sobre uma lista, a execução de **S** reduz o tamanho da lista e **T** faz uma procura seqüencial na lista.

Complexidade da seqüência com preservação (assintótica) de tamanho

- Se **S** preserva tamanho assintoticamente, então a complexidade pessimista da seqüência **S ; T** tem cotas:

$$\text{Máx} (c_P [S], c_P [T]) \leq c_P [S ; T] \leq c_P [S] + c_P [T].$$

- A ordem da **complexidade pessimista** da seqüência **S ; T** é dada por:

$$c_P [S ; T] = O(c_P [S] + c_P [T]).$$

Exemplo 3.2.4. Considere o trecho de algoritmo:

$v \leftarrow \text{Reversa}(u) ;$

$w \leftarrow \text{Ordene}(v).$

Reversa e Ordene têm complexidades $\Theta(n)$ e $O(n^2)$, respectivamente.

- Complexidade pessimista da seqüência (em geral). Um exemplo em que não há preservação de tamanho:

Exemplo 3.2.5. Dados algoritmos $\text{Prim}(u)$ e $\text{Buscab}(a,v)$, considere a seqüência:

$v \leftarrow \text{Prim}(u) ; \text{Buscab}(a, v).$

Vamos supor que:

- $\text{Prim}(u)$ dá como saída a primeira metade da lista em u , com comprimento $\lfloor n / 2 \rfloor$, e tem complexidade $\Theta(n)$;
- $\text{Buscab}(a, v)$ procura a na lista v , com complexidade $O(\log m)$, para lista v com comprimento m .

- Dada uma seqüência **S ; T**, considere $s(n)$ dando o tamanho máximo da saída **S** para entradas com tamanho até n , i. e.:

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) / \text{tam}(d) \leq n \}.$$

Complexidade pessimista da seqüência (em geral)

- A complexidade pessimista da seqüência **S ; T** tem cotas:

$$\text{Máx} (c_P [S](n), c_P [T](s(n))) \leq c_P [S ; T](n) \leq c_P [S](n) + c_P [T](s(n)).$$

- Logo, a **complexidade pessimista** da seqüência **S ; T** é dada por:

$$c_P [S ; T](n) = O(c_P [S](n) + c_P [T](s(n))).$$

Exemplo 3.2.6. Considere a seqüência:

$v \leftarrow \text{Prim}(u) ; w \leftarrow \text{Fin}(u) ;$
 $v \leftarrow \text{Ordene}(v) ; w \leftarrow \text{Ordene}(w) ;$
 $u \leftarrow \text{Concat}(v , w).$

Vamos supor que:

- Prim é como no exemplo 3.2.5 e, analogamente, Fin dá a segunda metade de sua entrada;
- Ordene tem complexidade quadrática;
- Concat(v , w) dá como saída a concatenação das listas v e w , com complexidade $O(p + q)$.

3.2.3 Condicionais

se b então S ou se b então S senão T

Exemplo 3.2.7. Considere as estruturas condicionais a seguir:

a) Para variável inteira i:

se $i = 0$ então $i \leftarrow i + 1$.

Esta estrutura condicional envolve:

- determinar se o valor de i é 0, com complexidade $\Theta(1)$;
- se sim, executar a atribuição $i \leftarrow i + 1$, com complexidade $\Theta(1)$.

b) Para lista v de inteiros e variável inteira m:

se $m = 0$ então $m \leftarrow \text{Max}(v)$.

Esta atribuição envolve:

- determinar se o valor de m é 0, com complexidade $\Theta(1)$;
- se sim, executar a atribuição $m \leftarrow \text{Max}(v)$, com complexidade $\Theta(n)$.

- O desempenho $desemp[\text{se } b \text{ então } S](d)$ da estrutura condicional se **b** então **S**,

com entrada **d** é dado por:

$$\begin{array}{ll} aval [b](d) + desemp[S](d) & \text{caso o valor de } b \text{ em } d \text{ seja verdadeiro;} \\ aval [b](d) & \text{caso o valor de } b \text{ em } d \text{ seja falso.} \end{array}$$

Complexidade da estrutura condicional simples

- A complexidade pessimista de se **b** então **S** tem cotas:

$$c_P [b] \leq c_P [\text{se } b \text{ então } S] \leq c_P [b] + c_P [S].$$

- Logo, a ordem da **complexidade pessimista** de se **b** então **S** é dada por:

$$c_P [\text{se } b \text{ então } S] = O(c_P [b] + c_P [S]).$$

Exemplo 3.2.8. Dados algoritmos Max e Ordene, (ex. 3.2.1 e 3.2.2), considere o seguinte trecho de algoritmo, para lista v de inteiros:

se $\text{Max}(v) = 0$ então $v \leftarrow \text{Ordene}(v)$.

Para esta estrutura condicional temos:

$$\begin{aligned} c_P [\text{se } \text{Max}(v) = 0 \text{ então } v \leftarrow \text{Ordene}(v)] &= \\ &= O(c_P [\text{Max}(v) = 0] + c_P [v \leftarrow \text{Ordene}(v)]). \end{aligned}$$

Como seus componentes têm complexidades pessimistas:

$$c_P [\text{Max}(v) = 0] = O(n) \text{ e } c_P [v \leftarrow \text{Ordene}(v)] = O(n^2);$$

Exemplo 3.2.9. Considere as estruturas condicionais a seguir:

a) Para variáveis inteiras i e j :

se $i \neq j$ então $i \leftarrow i + j$ senão $j \leftarrow i + 1$.

Esta estrutura condicional envolve:

- saber se os valores de i e j são diferentes, tem complexidade $\Theta(1)$;
- se sim, executar a atribuição $i \leftarrow i + j$, com complexidade $\Theta(1)$;
- se não, executar a atribuição $j \leftarrow i + 1$, com complexidade $\Theta(1)$.

b) Para listas u e v (de inteiros):

se $u = v$ então $v \leftarrow \text{Prim}(u)$ senão $u \leftarrow \text{Ordene}(v)$.

Esta estrutura condicional envolve:

- determinar se as listas u e v são iguais, com complexidade $O(n)$;
- se sim, executar a atribuição $v \leftarrow \text{Prim}(u)$, com complexidade $O(n)$;
- se não, executar a atribuição $u \leftarrow \text{Ordene}(v)$, com complexidade $O(n^2)$.

- Assim, a estrutura condicional se **b** então **S** senão **T**, com entrada **d** tem desempenho $desemp[\text{se } b \text{ então } S \text{ senão } T](d)$ dado por:

$aval [b](d) + desemp[S](d)$ caso o valor de **b** em **d** seja verdadeiro;
 $aval [b](d) + desemp[T](d)$ caso o valor de **b** em **d** seja falso.

Complexidade pessimista da estrutura condicional composta

- A complexidade pessimista de se **b** então **S** senão **T** tem as cotas:

inferior : $Máx (c_P [b], c_P [S], c_P [T]) \leq c_P [\text{se } b \text{ então } S \text{ senão } T]$;

superior : $c_P [\text{se } b \text{ então } S \text{ senão } T] \leq c_P [b] + MxAO (c_P [S], c_P [T])$.

- A ordem da **complexidade pessimista** de se **b** então **S** senão **T** é dada por:

$$c_P [\text{se } b \text{ então } S \text{ senão } T] = O(c_P [b] + MxAO (c_P [S], c_P [T])).$$

Exemplo 3.2.10. Dados algoritmos Max, Reversa e Ordene (ex. 3.2.1 e 3.2.2), considere o seguinte trecho de algoritmo, para lista v de inteiros:

se $\text{Max}(v) \geq 0$ então $v \leftarrow \text{Reversa}(v)$ senão $v \leftarrow \text{Ordene}(v)$.

Para esta estrutura condicional temos:

$$\begin{aligned} c_p [\text{se } \text{Max}(v) \geq 0 \text{ então } v \leftarrow \text{Reversa}(v) \text{ senão } v \leftarrow \text{Ordene}(v)] &= \\ = O(c_p [\text{Max}(v) \geq 0] + \text{MxAO}(c_p [v \leftarrow \text{Reversa}(v)], c_p [v \leftarrow \text{Ordene}(v)])). \end{aligned}$$

Suas componentes têm as seguintes complexidades pessimistas:

$$c_p [\text{Max}(v) \geq 0] = O(n);$$

$$c_p [v \leftarrow \text{Reversa}(v)] = O(n) \text{ e } c_p [v \leftarrow \text{Ordene}(v)] = O(n^2).$$

- Assim, temos o seguinte máximo assintótico em ordem de complexidades:

$$\begin{aligned} \text{MxAO}(c_p [v \leftarrow \text{Reversa}(v)], c_p [v \leftarrow \text{Ordene}(v)]) &= \\ = \text{MxAO}(n, n^2) &= O(n^2). \end{aligned}$$

3.2.4 Iteração definida (ou incondicional)

para i de j até m faça S

Exemplo 3.2.11. Considere as iterações definidas a seguir:

a) Para variável inteira m:

para i de 1 até 20 faça $m \leftarrow m + 1$.

b) Para vetor $A[1..n]$ de inteiros:

para i de 1 até n faça $A[i] \leftarrow 0$ {inicialização}.

c) Para vetor $A[1..n]$ de inteiros:

para i de 1 até n faça $A[i] \leftarrow A[i] + 1$ {atualização}.

d) Para variável real s e vetor $R[1..n]$ de reais

para i de 1 até n faça $s \leftarrow s + R[i]$ {somatório}.

e) Para variável real t e matriz quadrada $Q[1..n, 1..n]$ de reais

para i de 1 até n faça $t \leftarrow t + Q[i, i]$ {traço}.

- Na execução, para a entrada **d**, do comando:

para **i** de **j** até **m** faça **S**,

- inicialmente, **S** é executado, com $i = j$, sobre a entrada **d** dando $S(d)$;
- em seguida, **S** é executado, com $i := j + 1$, sobre $S(d)$ dando $S^2(d)$;
-
- finalmente, **S** é executado, com $i := m$, sobre $S^{N(d)-1}(d)$ resultando $S^{N(d)}(d)$

$N(d) = m - j + 1$, com **j** = **j**(d) e **m** = **m**(d).

- Além do esforço de controle dos valores de i , j , m , temos os **esforços computacionais** associados às várias iterações de **S** sucessivamente sobre:
 $d, S(d), S^2(d), \dots, S^{N(d)-1}(d)$.
- Em geral, os esforços computacionais de determinação dos valores de controle são dominados pelos das iterações de **S**.
- Assim, a iteração definida para i de j até m faça **S**, com entrada **d** tem seu desempenho $desemp[\text{para } i \text{ de } j \text{ até } m \text{ faça } S](d)$ dado pelo somatório:

$$\sum_{i=j(d)}^{m(d)} desemp[S](S^{(i-j(d))}(d)).$$

Exemplo 3.2.12. Considere a iteração definida:

para i de 1 até $(n - 1)$ faça Troca($A[i]$, $A[i + 1]$);

onde Troca($A[p]$, $A[q]$) troca de posição os elementos $A[p]$ e $A[q]$ no vetor $A[1..n]$, com complexidade constante $O(1)$.

São executadas $(n - 1)$ trocas, sucessivamente:

Troca($A[1]$, $A[2]$), Troca($A[2]$, $A[3]$), ..., Troca($A[n - 1]$, $A[n]$);
 cada uma delas com mesma complexidade $O(1)$.

- Dada uma iteração definida

para i de j até m faça **S**,

considere $N(n) := \text{Máx} \{N(d) \mid \text{tam}(d) \leq n\}$, onde $N(d) := (m(d) - j(d) + 1)$.

Iteração definida com preservação (assintótica) de tamanho

- Se **S** preserva tamanho assintoticamente, então a complexidade pessimista da iteração definida para i de j até m faça **S** tem cotas:

$$c_P [S](n) \leq c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) \leq N(n) \cdot c_P [S](n).$$

- Assim, a ordem da **complexidade pessimista** de para i de j até m faça **S** é:

$$c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) = O(N(n) \cdot c_P [S](n)).$$

Exemplo 3.2.13. Considere o trecho de algoritmo:

para i de 1 até p faça Buscab(A[i],v);

onde Buscab(a, v) é como no ex. 3.2.5: procura a na lista v, com complexidade $O(\log m)$, para lista v com comprimento m.

Considere como tamanho do problema o máximo entre p e m, chamando $n := \text{Máx}\{p, m\}$.

Exemplo 3.2.14. Considere a iteração definida

para i de j até m faça **S** onde:

S preserva tamanho, i. e., $\text{tam}(S(d)) = \text{tam}(d)$; e

$c_p[S]$ é um polinômio de grau k, i. e., $c_p[S](n) = O(n^k)$.

- Na execução da iteração definida

para i de j até m faça S,

em geral, **S** **pode não preservar o tamanho** (nem mesmo assintoticamente). Neste caso, cada iteração tem que ser considerada e a iteração da função tamanho será usada.

- Dada uma função $s : \mathbb{N} \rightarrow \mathbb{N}$, define-se por indução sobre $i \in \mathbb{N}$:

$$\begin{aligned} s^0(n) &:= n \\ s^{(i+1)}(n) &:= s^i(s(n)). \end{aligned}$$

A execução de **S** é iterada $N(d)$ vezes, onde $N(d) = (m(d) - j(d) + 1)$.

- Usando os valores máximo para $m(d)$ e mínimo para $j(d)$, para entrada de tamanho n :

$$m(n) := \text{Máx} \{ m(d) / tam(d) \leq n \} \text{ e } j(n) := \text{mín} \{ j(d) / tam(d) \leq n \}$$

e função tamanho máximo é

$$s(n) := \text{Máx} \{ tam(S(d)) / tam(d) \leq n \}.$$

Complexidade pessimista da iteração definida (em geral)

- A complexidade pessimista da iteração definida para i de j até m faça **S** tem as cotas:

inferior : Máx { $c_P [S](s^{(i-j(n))}(n))$ / $i = j(n), \dots, m(n)$ };

superior : $\sum_{i=j(n)}^{m(n)} c_P [S](s^{(i-j(n))}(n)).$

- Assim, a ordem da **complexidade pessimista** de para i de j até m faça **S** é:

$$[\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) = O\left(\sum_{i=j(n)}^{m(n)} c_P [S](s^{(i-j(n))}(n))\right).$$

Exemplo 3.2.15. Considere o trecho de algoritmo:

para i de 1 até n faça Buscas(A[i], B[i..n]);

onde Buscas(a, B[i..n]) é similar ao ex. 3.2.13: procura a na parte [i..n] (de i até n) do vetor B, com complexidade $O(m)$ (onde $m := n - i + 1$).

A iteração definida tem complexidade pessimista:

$$\begin{aligned} c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça Buscas(A[i], B[i..n]) }] &= \\ &= O\left(\sum_{i=1}^n c_P [\text{Buscas(A[i], B[i..n]) }](s^{(i-1)}(n))\right). \end{aligned}$$

Neste caso, temos:

$$c_P [\text{Buscas(A[i], B[i..n]) }](m) = O(m);$$

$$s(m) = m - 1 \text{ (dando } s^k(m) = m - k \text{)}.$$

$$\text{Assim, } c_P [\text{Buscas(A[i], B[i..n]) }](s^{(i-1)}(n)) = O(n - i + 1).$$

Então, complexidade pessimista da iteração definida fica:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } \text{Buscas}(A[i], B[i..n])] = O(\sum_{i=1}^n (n - i + 1)).$$

Mas, $\sum_{i=1}^n (n - i + 1) = \sum_{j=1}^n j$ e sabemos que:

$$\sum_{j=1}^n j = \frac{(1+n) \cdot n}{2} = O(n^2).$$

Portanto, a ordem da **complexidade pessimista** desta iteração definida é:

$$\begin{aligned} c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } \text{Buscas}(A[i], B[i..n])] &= \\ &= O(\sum_{j=1}^n j) = \mathbf{O(n^2)}. \end{aligned}$$

Exemplo 3.2.16. Considere a iteração definida

para i de 1 até n faça **S**

onde:

$c_P [S]$ é um polinômio de grau k , i. e., $c_P [S](n) = O(n^k)$; e $\text{tam}(S(d)) = \text{tam}(d) - 1$ (portanto $s(n) = n - 1$ e $s^i(n) = n - i$).

Neste caso, temos:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } S](n) = O\left(\sum_{i=1}^n (n - i)^k\right).$$

Pode-se mostrar que $\sum_{i=1}^n (n - i)^k = O(n^{k+1})$.

Portanto, teremos:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } S](n) = O(n^{k+1}).$$

3.2.5 Iteração indefinida (ou condicional)

enqto **b** faça **S**

Exemplo 3.2.17. Considere os trechos de algoritmos a seguir:

a) Para variável inteira i:

$i \leftarrow 0$ {inicialização} ;
enqto $i \leq 10$ faça $i \leftarrow i + 1$

{iteração}.

b) Inicialização de vetor $A[1..n]$ de inteiros:

$i \leftarrow 0$ {inicialização} ;
enqto $i < n$ faça $i \leftarrow i + 1$; $A[i] \leftarrow 0$ fim-enqto

{iteração}.

c) Atualização de vetor $A[1..n]$ de inteiros:

$i \leftarrow n$ {inicialização} ;
enqto $i > 0$ faça $A[i] \leftarrow A[i] + 1$; $i \leftarrow i - 1$ fim-enqto {iteração}.

d) Somatório de vetor $R[1..n]$ de reais:

$s \leftarrow 0$; $i \leftarrow n$ {inicializações} ;
enqto $i > 0$ faça $s \leftarrow s + R[i]$; $i \leftarrow i - 1$ fim-enqto {iteração}.

e) Traço de matriz quadrada $Q[1..n, 1..n]$ de reais:

$t \leftarrow 0$; $i \leftarrow 1$ {inicializações} ;
enqto $i \leq n$ faça $t \leftarrow t + Q[i, i]$; $i \leftarrow i + 1$ fim-enqto {iteração}.

f) Fatorial de natural n :

$f \leftarrow 1$; $i \leftarrow n$ {inicializações} ;
enqto $i > 1$ faça $f \leftarrow f \cdot i$; $i \leftarrow i - 1$ fim-enqto {iteração}.

A execução da iteração indefinida:

enqto **b** faça **S**,

sobre a entrada **d**, itera a execução de **S** sucessivamente:

- se a entrada **d** **satisfaz b**, então se executa **S** sobre **d**, dando **S(d)**;
- se **S(d)** **satisfaz** a condição **b**, então se executa **S** sobre **S(d)**, dando **S²(d)**;

.....

- finalmente, quando a condição **b deixa de ser satisfeita** sobre **S^{H(d)}(d)**, a execução termina com **S^{H(d)}(d)** como sua saída final.

Esforço computacional:

- avaliação de **b** e execução de **S** para: **d**, **S(d)**, **S²(d)**, ..., **S^{H(d)-1}(d)**;
- avaliação de **b** para **S^{H(d)}**.

- Uma questão **importante** é a determinação de **H(d)**.

- O desempenho de enqto **b** faça **S** sobre entrada **d** é dado por:

$$\text{desemp}[\text{enqto } b \text{ faça } S](d) = \text{aval}[b](S^{H(d)}(d)) + \sum_{i=0}^{H(d)-1} [\text{aval}[b](S^i(d)) + \text{desemp}[S](S^i(d))].$$

- Nos casos em que os testes são dominados pelos das iterações de **S**, podemos desprezá-los. Assim, a expressão anterior se simplifica para:

$$\text{desemp}[\text{enqto } b \text{ faça } S](d) = \sum_{i=0}^{H(d)-1} \text{desemp}[S](S^i(d)).$$

Exemplo 3.2.18. Considere a seguinte iteração indefinida:

enqto ($A[i] = 0 \wedge 1 \leq i \leq n$) faça $A[i] \leftarrow A[i] + 1$; $i \leftarrow i + 1$ fim-enqto.

Aqui, cada iteração mantém fixo o tamanho do vetor $A[1..n]$ de entrada e tem complexidade $O(1)$.

No pior caso, tem-se inicialmente $i = 1$ e são executados todas as iterações, com $i = 1, i = 2, \dots, i = n$;

Assim, essa **iteração indefinida** terá complexidade $n \cdot 1 = O(n)$.

- Agora é preciso **determinar $H(d)$** .

$H(d)$ é o número de vezes que **S** precisa ser executado para que o teste **b** dê o valor falso:

$$H(d) := \text{mín} \{ i \in \mathbb{N} / \neg b(S^i(d)) \}.$$

Como estamos interessados no **pior caso**, introduzimos:

$$h(n) := \text{Máx} \{ H(d) \in \mathbb{N} / \text{tam}(d) \leq n \}.$$

- Em casos razoavelmente freqüentes, a complexidade de **S** domina a do teste **b**, **absorvendo-a**.
- Supondo que $c_P[\mathbf{b}]$ é sempre dominada por $c_P[\mathbf{S}]$:

A definição da função $h : \mathbb{N} \rightarrow \mathbb{N}$ segue a idéia usada para $s : \mathbb{N} \rightarrow \mathbb{N}$, onde

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) \in \mathbb{N} / \text{tam}(d) \leq n \}.$$

Complexidade da iteração indefinida com testes absorvidos

- A complexidade pessimista de enqto **b** faça **S** tem as cotas:

inferior : $\text{Máx} \{ c_P [S](s^i(n)) / i = 0, \dots, h(n) - 1 \} \leq c_P [\text{enqto } b \text{ faça } S](n);$

superior : $c_P [\text{enqto } b \text{ faça } S](n) \leq \sum_{i=0}^{h(n)-1} c_P [S](s^i(n)).$

- Se $c_P [b]$ é $O(c_P [S])$, a **complexidade pessimista** de enqto **b** faça **S** é:

$$c_P [\text{enqto } b \text{ faça } S](n) = O\left(\sum_{i=0}^{h(n)-1} c_P [S](s^i(n)) \right).$$

Exemplo 3.2.19. Considere a seguinte iteração indefinida:

enqto $1 \leq i \leq n$ faça Buscas($A[i]$, $B[i..n]$); $i \leftarrow i + 1$ fim-enqto,
onde Buscas(a , $B[i..n]$) é como no ex. 3.2.15: procura a na parte $[i..n]$ (de i até n) do vetor B , com complexidade $O(m)$.

Desprezando o teste, a iteração indefinida tem complexidade pessimista:

$$c_P [\text{enqto } 1 \leq i \leq n \text{ faça Buscas}(A[i], B[1..i]); i \leftarrow i + 1 \text{ fim-enqto}] = \\ = O\left(\sum_{i=0}^{h(n)-1} c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](s^i(n)) \right).$$

Neste caso, temos: $c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](m) = O(m)$, $h(n) = n$ e $s(m) = m - 1$ (dando $s^i(m) = m - i$).

Logo $c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](s^i(n)) = O(n - i)$.

Procedendo como no ex. 3.2.15, obtemos: $\sum_{i=0}^{n-1} (n - i) = O(n^2)$.

• Portanto, a ordem da **complexidade pessimista** desta iteração indefinida é

$$c_P [\text{enqto } 1 \leq i \leq n \text{ faça Buscas}(A[i], B[1..i]); i \leftarrow i + 1 \text{ fim-enqto}] = \\ = O\left(\sum_{i=0}^{n-1} (n - i) \right) = O(n^2).$$

Complexidade pessimista da iteração indefinida (em geral)

- A complexidade pessimista de enqto **b** faça **S** tem as cotas:

inferior : $\text{Máx} \{ c_P [b](s^{h(n)}(n)), \text{Máx} \{ \text{iter}_i(n) / i = 0, \dots, h(n) - 1 \} \}$
 (onde $\text{iter}_i(n) := \text{Máx} \{ c_P [b](s^i(n)), c_P [S](s^i(n)) \}$);

superior : $c_P [b](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [b](s^i(n)) + c_P [S](s^i(n))]$.

- Assim, a ordem da **complexidade pessimista** de enqto **b** faça **S** é dada por:

$$c_P [\text{enqto } b \text{ faça } S](n) = O(c_P [b](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [b](s^i(n)) + c_P [S](s^i(n))]).$$

Exemplo 3.2.20. Considere a iteração indefinida enqto **b** faça **S** onde:

$c_P [S]$ é um polinômio de grau k , i. e., $c_P [S](n) = O(n^k)$; e h é no máximo linear, i. e., $h(n) \leq n$.

a) Se **S** preserva tamanho, temos $s(n) = n$.

$$c_P [\text{enqto } b \text{ faça } S](n) = O\left(\sum_{i=0}^{h(n)-1} n^k \right) = O\left(\sum_{i=0}^{n-1} n^k \right).$$

b) Se **S** faz o tamanho decrescer de uma unidade, temos $s(n) = n - 1$.

$$c_P [\text{enqto } b \text{ faça } S] = O\left(\sum_{i=0}^{h(n)-1} (n - i)^k \right) = O\left(\sum_{i=0}^{n-1} (n - i)^k \right).$$

Ambos os casos têm a mesma ordem:

$$\sum_{i=0}^{n-1} n^k \text{ e } \sum_{i=0}^{n-1} (n - i)^k \text{ são } O(n^{k+1}).$$

3.3 EXEMPLOS DE APLICAÇÃO

- Algoritmo 3.3.1: **Máximo e mínimo de tabela.**

<u>Função</u> Max_&_min(Tab : D) → Par[Elmord]	<i>{para máximo e mínimo de tabela}</i>
	<i>{Entrada: tabela Tab : D.</i>
	<i>Saída: valores Max, min : Elmord (máximo e mínimo da entrada).}</i>
<u>sorte</u> D := <u>vetor</u> [p..q] <u>de</u> Elmord ;	
1. Max ← Tab[p] ;	<i>{candidato a máximo}</i>
2. min ← Tab[p] ;	<i>{candidato a mínimo}</i>
3. <u>para</u> i <u>de</u> p + 1 <u>até</u> q <u>faça</u>	<i>{varre tabela: de 3 a 6}</i>
4. <u>se</u> Tab[i] ← Max <u>então</u> Max ← Tab[i] ;	<i>{atualiza Max}</i>
5. <u>se</u> Tab[i] ← min <u>então</u> min ← Tab[i] ;	<i>{atualiza min}</i>
6. <u>fim-para</u> ;	<i>{3: i de p + 1 até q}</i>
7. <u>retorne-saída</u> (Max , min) ;	<i>{dá saída Max e min}</i>
8. <u>fim-Função</u> .	<i>{fim do algoritmo Max_&_min: máximo e mínimo de tabela}</i>

Escolhendo como operação fundamental a comparação entre elementos.

- Analisando:

$$c_P [\text{linha 1} ; \text{linha 2}] = O(1).$$

$$c_P [\text{linha 4}] + c_P [\text{linha 5}] = O(1).$$

$$\begin{aligned} c_P [\text{para linha 3 ... linha 6}] &= O(q - 1 - p + 1) \cdot c_P [\text{linha 4; linha 7}] = \\ &= O((n - 1) \cdot 1) = O(n). \end{aligned}$$

$$\begin{aligned} c_P [\text{Máx_ \& _mín}] &= O(c_P [\text{linha 1}] + c_P [\text{linha 2}] + c_P [\text{para linha 3 ... linha 6}]) \\ &= O(n + 0 + n) \\ &= O(n). \end{aligned}$$

○ **Algoritmo 3.3.2: União e interseção de conjuntos.**

<u>Função</u> Uni&Int(S, T : M) → Par[M]	<i>{para a união e interseção de conjuntos}</i>
	<i>{Entrada: conjuntos S, T : M (a operar).}</i>
	<i>Saída: conjuntos Uni, Int : M (Uni, união $S \cup T$, e Int, a interseção $S \cap T$).}</i>
<u>sorte</u> M := Conj[Elm] {conjuntos de elementos de Elm} ;	
<u>var</u> {auxiliar} x : Elm {elemento de Elm} ;	
1. Uni ← S ;	<i>{inicializa Uni com S}</i>
2. Int ← $\emptyset$;	<i>{inicializa Int com vazio}</i>
3. <u>para cada</u> x ∈ T <u>faça</u>	<i>{varre conjunto T: de 3 a 8}</i>
4. <u>se</u> x ∈ S	<i>{testa se x está em S}</i>
5. <u>então</u> Insere(x, Int)	<i>{põe x comum em Int}</i>
6. <u>senão</u> Insere(x, Uni) ;	<i>{põe novo x em Uni}</i>
7. <u>fim-se</u> ;	<i>{4: x ∈ S?}</i>
8. <u>fim-para</u> ;	<i>{3: cada x ∈ T}</i>
9. <u>retorne-saída</u> (Uni, Int) ;	<i>{dá saída Uni e Int}</i>
10. <u>fim-Função</u> .	<i>{fim de Uni&Int: união e interseção de conjuntos}</i>

- Analisando:

$$\begin{aligned}c_P [\text{Uni\&Int}] &= \\&= O(c_P [\text{Uni} \leftarrow S] + c_P [\text{Int} \leftarrow \emptyset] + c_P [\text{para linha 3 ... linha 8}]) \\&= O(|S| + 1 + (|T| \cdot c_P [\text{se linha 4 ... linha 7}])) \\&= O(n + 1 + (n \cdot (c_P [x \in S] + \text{MxAO} (c_P [\text{linha 5}], c_P [\text{linha 6}])))) \\&= O(n + (n \cdot (|S| + \text{MxAO} (1, 1)))) \\&= O(n + (n \cdot (n + 1))) \\&= O(n + n^2) \\&= O(n^2).\end{aligned}$$

○ **Algoritmo 3.3.3: Classificação de vetor.**

Procedimento Classif_Max(A : V)

{classifica vetor em ordem ascendente}

{Entrada: vetor A : V.

Saída: vetor A : V (o vetor de entrada classificado em ordem ascendente).}

sorte V := vetor [1..n] de Elmord ;

var {auxiliar} Temp : Elmord {temporária (para troca)} ;

1. para i de 1 até n - 1 faça

{de 1 a 10}

2. para j de 1 até n - i faça

{de 2 a 9}

3. se A[j] > A[j + 1]

{testa inversão}

4. então

{troca A[j] com A[j + 1]}

5. Temp ← A[j] ;

6. A[j] ← A[j + 1] ;

7. A[j + 1] ← Temp ;

8. fim-se ;

{3: A[j] ← A[j + 1]?}

9. fim-para ;

{2: j de 1 até n - i}

10. fim-para ;

{1: i de 1 até n}

11. retorne-saída(A) ;

{dá saída A}

12. fim-Procedimento .

{fim de Classif_Max: classificação ascendente}

○ **Algoritmo 3.3.4: Loteria esportiva.**

Procedimento Loto(Rslt : V, Apst : M) *{teste da loteria esportiva}*
{Entrada: vetor Rslt : V (resultado de teste), matriz Apst : M (apostador)
Saída: vetor Ptos : vetor [1..n] de IN (número de pontos feitos por apostador).}
 sorte V := vetor [1..13] de { 1..3 } ; M := matriz [1..n, 0..13] de Info ;

1. $i \leftarrow 1$; *{inicializa número de apostadores}*
2. enqto $i \leq n$ faça *{examina apostador: de 2 a 8}*
3. Ptos[i] $\leftarrow 0$; *{inicializa número de pontos do apostador i}*
4. para j de 1 até 13 faça *{examina palpites: de 4 a 6}*
5. se Apst[i,j] = Rslt[j] então Ptos[i] $\leftarrow$ Ptos[i] + 1 .
6. fim-para ; *{4: pontos do apostador i contabilizados}*
7. $i \leftarrow i + 1$; *{novo apostador}*
8. fim-enqto ; *{2: contabilizados os pontos de todos apostadores}*
9. para i de 1 até n faça *{lista ganhadores: de 9 a 11}*
10. se Ptos[i] = 13 então escreva(Apst[i,0], "Ganhador, parabéns")
11. fim-para ; *{9: ganhadores listados}*
12. retorne-saída(Ptos) ; *{dá saída Ptos}*
13. fim-Procedimento . *{fim do algoritmo Loto: loteria esportiva}*

3.4 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é introduzir uma metodologia para analisar a complexidade pessimista de um algoritmo com base em sua estrutura.
- Algumas componentes de um algoritmo são sempre executadas (as conjuntivas), outras (as disjuntivas) definem alternativas, que são executadas conforme o caso.
- As contribuições desses dois gêneros de componentes para o desempenho e a complexidade de um algoritmo **a**, com duas componentes **a₁** e **a₂** é dada por dois princípios básicos.
 - **Princípio das partes conjuntivas:** A complexidade pessimista de um algoritmo **a** com duas componentes **a₁** e **a₂** conjuntivas (de modo que $desemp[a](d) = desemp[a_1](d) + desemp[a_2](d)$, para qualquer entrada **d**) tem as cotas $Máx(c_P^{\leq}[a_1], c_P^{\leq}[a_2]) \leq c_P^{\leq}[a] \leq c_P^{\leq}[a_1] + c_P^{\leq}[a_2]$.

- ▣ **Princípio das partes disjuntivas:** A complexidade pessimista de um algoritmo **a** com duas componentes **a₁** e **a₂** disjuntivas (de modo que $desemp[a](d) = desemp[a_1](d)$ ou $desemp[a](d) = desemp[a_2](d)$, para qualquer entrada **d**) é dada por $c_P^\leq[a] = \text{Máx}(c_P^\leq[a_1], c_P^\leq[a_2])$.
- Os conceitos de absorção e máximo assintótico (em ordem) são motivados pelas idéias de reduzir a complexidade de um algoritmo às ordens das suas partes.
- A complexidade de uma parte pode ser "absorvida" pela complexidade de outra: a idéia é que a segunda domina a primeira, dando a contribuição principal para a soma. Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, dizemos que **f** é absorvida por **g** sse **f** é $O(g)$.
- O princípio seguinte formula a idéia acima: se a função **f** cresce mais rapidamente do que **g**, então sua soma crescerá como **f**, de maneira assintótica.
 - ▣ **Princípio da absorção:** Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, se **f** é absorvida por **g**, sua soma pontual **f + g** é $\Theta(g)$.

- O máximo assintótico em ordem MxAO (f, g), deve ser uma boa cota superior - em sentido assintótico - para as ordens de ambas. Frequentemente, toma-se como máximo assintótico a soma ou o máximo pontuais.
- Complexidades pessimistas das principais estruturas algorítmicas:

▣ **Atribuição $v \leftarrow e$:**

$$c_P [v \leftarrow e] = O(c_P [e] + c_P [\leftarrow_e]);$$

onde $c_P [e]$ e $c_P [\leftarrow_e]$ são as complexidades pessimistas da avaliação da expressão e e da transferência do valor resultante, respectivamente.

▣ **Seqüência $S ; T$:**

$$c_P [S ; T](n) = O(c_P [S](n) + c_P [T](s(n)));$$

onde $s(n)$ dá o tamanho máximo da saída de S para entradas com tamanho até n , i. e.

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) / \text{tam}(d) \leq n \}.$$

□ **Condicional se b então S senão T:**

$$c_P [\text{se } b \text{ então } \mathbf{S} \text{ senão } \mathbf{T}] = O(c_P [b] + \text{MxAO}(c_P [\mathbf{S}], c_P [\mathbf{T}])).$$

onde MxAO dá um máximo assintótico em ordem das complexidades das partes.

□ **Iteração definida para i de j até m faça S:**

$$c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } \mathbf{S}](n) = O(\sum_{i=j(n)}^{m(n)} c_P [\mathbf{S}](s^{(i-j(n))}(n)));$$

onde $m(n) := \text{Máx} \{ m(d) / \text{tam}(d) \leq n \}$, $j(n) := \text{mín} \{ j(d) / \text{tam}(d) \leq n \}$, e a função $s : \mathbb{N} \rightarrow \mathbb{N}$ é como no caso de seqüência.

□ **Iteração indefinida enqto b faça **S**:**

$$c_P [\text{enqto } \mathbf{b} \text{ faça } \mathbf{S}](n) = O(c_P [\mathbf{b}](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [\mathbf{b}](s^i(n)) + c_P [\mathbf{S}](s^i(n))]);$$

onde $s : \mathbb{N} \rightarrow \mathbb{N}$, e $h(n)$ dá o número máximo de iterações para entradas com tamanho até n , i. e.

$h(n) := \text{Máx} \{ H(d) \in \mathbb{N} / \text{tam}(d) \leq n \},$

com $H(d) := \text{mín} \{ i \in \mathbb{N} / \neg b(S^i(d)) \}.$

4. COMPLEXIDADE MÉDIA

- Este capítulo considera a **complexidade média**, examinando casos ilustrativos de análise de algoritmos e de projeto de estruturas.
- Dentre as medidas de complexidade, a complexidade média é, às vezes, o critério de avaliação **mais apropriado**.
- Conforme o capítulo 2, a complexidade média de um algoritmo é uma **média ponderada** do desempenho de cada entrada pela probabilidade de ela ocorrer.
- Dado um algoritmo **a**, sejam:
 - D_n o conjunto de suas entradas com tamanho **n**,
 - $\text{prob}(d)$ a probabilidade de ocorrer a entrada **d** de **D**,
 - $\text{desemp}[\mathbf{a}](d) := \text{custo}(\text{exec}[\mathbf{a}](d))$ o desempenho de **a** com entrada **d**.
- Assim, a complexidade média do algoritmo **a**:

$$c_M[\mathbf{a}](n) := \sum_{d \in D_n} [\text{prob}(d) \cdot \text{desemp}[\mathbf{a}](d)].$$

Visão geral do capítulo

- 4.1 Um exemplo clássico: Quicksort.

Analisa o algoritmo de classificação Quicksort como um exemplo clássico de complexidade média, com distribuição uniforme.

- 4.2 Árvore de busca binária ótima.

Examina árvore de busca binária ótima: um exemplo de armazenamento de dados utilizando um critério que tem como objetivo minimizar a complexidade média de procura.

4.1 UM EXEMPLO CLÁSSICO: QUICKSORT

- O algoritmo *Quicksort* classifica uma lista, particionando-a, classificando suas partes e depois concatenando as partes classificadas.
- A idéia básica é escolher, de alguma maneira, um *pivô* e particionar a lista em relação ao pivô de maneira que todo elemento de uma das partes seja menor que qualquer elemento da outra parte.
- Este algoritmo *classifica* um vetor *A* em ordem *não decrescente*. O pivô será o maior entre os dois primeiros elementos diferentes em *L*, sendo zero se todos os elementos forem iguais entre si.

○ **Algoritmo 4.1.1: Quicksort.**

<u>Função</u> Quicksort ($A : V, m, p : \text{Nat}$) $\rightarrow A : V$	<i>{ordenação por concatenação}</i>
	<i>{Entrada: vetor $A[m..p]$ (a ordenar).}</i>
	<i>{Saída: vetor $A : V$ (o vetor de entrada classificado em ordem ascendente).}</i>
<u>sorte</u> $V :=$ <u>vetor</u> $[m..p]$ <u>de</u> Elmord	
1. $k \leftarrow \text{ind}(\text{pivô}, A);$	<i>{ inicializa k com índice do pivô }</i>
2. <u>se</u> $k = 0$	<i>{ testa se todos os elementos são iguais entre si: 2 }</i>
3. <u>então</u> <u>retorne-saída</u> (A)	<i>{ vetor já está classificado }</i>
4. <u>senão</u>	<i>{ inicia classificação: 4 }</i>
5. Troca($A[m], A[k]$);	<i>{ troca primeiro com pivô }</i>
6. Particionamento(A);	<i>{ particiona na posição j }</i>
7. $L' \leftarrow \text{Quicksort}(A[m .. j - 1]);$	<i>{ classifica início (tamanho j - m) }</i>
8. $L'' \leftarrow \text{Quicksort}(A[j .. p]);$	<i>{ classifica final (tamanho p - j + 1) }</i>
9. $L \leftarrow \text{Concat}(L', L'');$	<i>{ concatena partes classificadas }</i>
10. <u>retorne-saída</u> (A);	<i>{ dá saída concatenada }</i>
11. <u>fim-senão</u> ;	<i>{ 4: inicia classificação }</i>
12. <u>fim-se</u> ;	<i>{ 2: k = 0? }</i>
13. <u>fim-Função</u> .	<i>{ fim do algoritmo Quicksort }</i>

- O desempenho de Quicksort com entrada **A** envolve as seguintes contribuições:
 - **escolha do pivô** e do algoritmo de Particionamento;
 - Quicksort com a **sublista da esquerda** ($A[m..j - 1]$ com tamanho i , $i = m-j$);
 - Quicksort com a **sublista da direita** ($A[j..p]$ com tamanho $n - i$, $n = p-m+1$ e $i = m-j$).

- Assim, o **desempenho** de Quicksort com entrada **A** será dado por:

$$\text{desemp[Quicksort]}(A) = \text{desemp[Pivô_Partic]}(A) +$$

$$+ \text{desemp[Quicksort]}(A[m..j - 1]) +$$

$$+ \text{desemp[Quicksort]}(A[j..p]).$$

onde, $\text{desemp[Pivô_Partic]}(A)$] é o desempenho da escolha do pivô e do algoritmo Particionamento.

- Vamos usar $\text{prob}(i)$ para a **probabilidade** de o pivô ser o $(i + 1)^{\text{o}}$ elemento:

$$c_M [\text{Quicksort}](n) = \sum_{i=1}^{n-1} \{ \text{prob}(i) \cdot [c_M [\text{Pivô_Partic}](n) +$$

$$+ c_M [\text{Quicksort}](i) + c_M [\text{Quicksort}](n - i)] \}.$$

- Como o algoritmo **Particionamento** tem complexidade linear, temos a **complexidade média** $c_M [\text{Pivô_Partic}](n)$ **de ordem $O(n)$** .
- Assim, abreviando $c_M [\text{Quicksort}](n)$ por $T(n)$, a expressão anterior se torna:

$$T(n) = O\left(\sum_{i=1}^{n-1} \{ \text{prob}(i) \cdot [n + T(i) + T(n - i)] \} \right).$$

- Agora, vamos calcular a **probabilidade** $\text{prob}(i)$ de o pivô ser o $(i+1)^{\circ}$ elemento.
- A probabilidade de qualquer elemento $(1^{\circ}, 2^{\circ}, 3^{\circ}, \dots, n^{\circ})$ estar na primeira posição é $\frac{1}{n}$, e a probabilidade de o segundo elemento ser menor que ele é $\frac{i}{n-1}$.
- Assim, a probabilidade de o pivô aparecer na primeira posição e ser o $(i+1)^{\circ}$ elemento será $\frac{1}{n} \cdot \frac{i}{n-1}$.
- De forma análoga, a probabilidade de o pivô estar na segunda posição e ser o $(i+1)^{\circ}$ elemento será $\frac{1}{n} \cdot \frac{i}{n-1}$.

- Logo, a probabilidade de o pivô ser o $(i + 1)^{\text{o}}$ elemento é $\frac{2i}{n \cdot (n-1)}$, e

$$\text{prob}(i) = \frac{2i}{n \cdot (n-1)}.$$

- Assim, para $n > 1$, teremos:

$$T(n) = O\left(\sum_{i=1}^{n-1} \left(\frac{2i}{n \cdot (n-1)}\right) \cdot \left\{ \sum_{i=1}^{n-1} i \cdot [n + T(i) + T(n-1)] \right\}\right).$$

- Realmente, pode-se mostrar, por indução sobre $n > 1$, que:

$$\frac{2i}{(n-1)} \cdot \sum_{i=1}^{n-1} T(i) = O(n \cdot \log n).$$

- Portanto, podemos concluir que:

$$c_M [\text{Quicksort}](n) = O(\log n).$$

4.2 ÁRVORE DE BUSCA BINÁRIA ÓTIMA

- Agora vamos examinar a **árvore de busca binária ótima**, como um exemplo de armazenamento de dados.
- Procura-se uma estrutura de dados que privilegie os dados mais usados, que armazene, nas posições mais próximas da raiz, os **dados mais procurados**.
- A estrutura é uma **árvore binária** e a procura é a partir da raiz, usando a estratégia da busca binária. Isto é, para qualquer nodo: “os elementos da subárvore da esquerda são menores que os elementos da subárvore da direita”.
- A questão é: dada uma seqüência ordenada de elementos, com **probabilidades de procura** conhecidas, obter uma tal árvore de busca binária ótima.

- Para isso, precisamos estabelecer a “**raiz**” da árvore. Conhecida a raiz, sabe-se que os elementos **menores** que a raiz ficarão na subárvore da **esquerda**, e os **maiores**, na subárvore da **direita**.
- Seja $S = \langle x_1, x_2, \dots, x_n \rangle$ uma seqüência ordenada de n elementos a serem alocados.

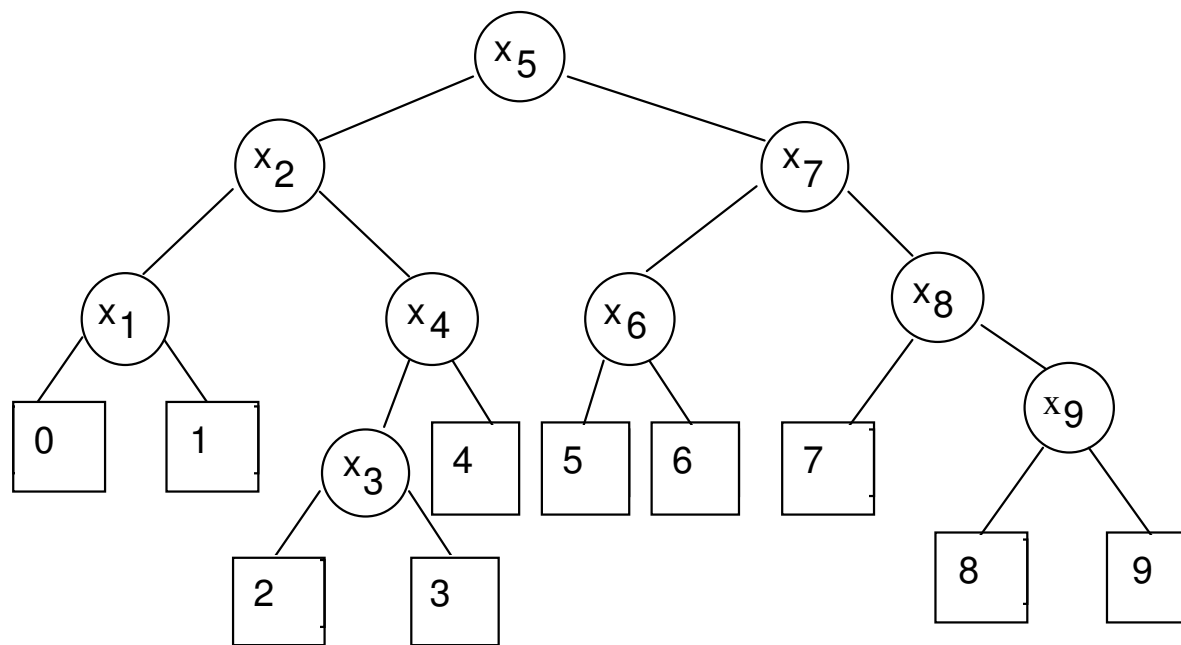


Figura 4.2.1: Árvore de busca binária

- A estrutura de dados é uma árvore de busca binária com nodos **redondos** representando elementos **presentes**, e nodos **quadrados**, elementos **ausentes**.
- Para a árvore representada na figura 4.2.1, tem-se os seguintes custos de procura c:

$$\left. \begin{array}{l} c(x_5) = 1 \\ c(x_2) = c(x_7) = 2 \\ c(x_1) = c(x_4) = c(x_6) = c(x_8) = 3 \\ c(x_3) = c(x_9) = 4 \end{array} \right\} \text{ i. e, } c(x_i) = \text{prof}(x_i) + 1$$

$$\left. \begin{array}{l} c(0) = c(1) = c(4) = c(5) = c(6) = c(7) = 3 \\ c(2) = c(3) = c(8) = c(9) = 4 \end{array} \right\} \text{ i.e, } c(i) = \text{prof}(i)$$

onde prof dá a **profundidade** do nodo na árvore.

- A complexidade média de procura na árvore é:

$$\sum_{x \in S} [\text{prob}(x) \cdot c(x)] + \sum_{x \notin S} [\text{prob}(x) \cdot c(x)]$$

onde $\text{prob}(x)$ é a **probabilidade** de procura de um elemento x e $c(x)$ dá o número de comparações necessárias para achar x ou determinar que $x \notin S$.

- Assim a complexidade média de procura em uma árvore A é dada pela seguinte equação:

$$c_M(A) = \sum_{i=1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \underbrace{\sum_{i=1}^n \text{prob}(x_i)}_{\text{independente da árvore}} + \sum_{i=0}^n (\text{prob}(i) \cdot \text{prof}_A(i))$$

Custo (A)

- Deseja-se obter A que **minimize** $\text{Custo}(A)$:

$$\begin{aligned} \text{Custo}(A) = & \sum_{i=1}^{r-1} (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=0}^n (\text{prob}(i) \cdot \text{prof}_A(i)) + \\ & + \text{prob}(x_r) \cdot \text{prof}_A(x_r) + \sum_{i=r+1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=r}^n (\text{prob}(i) \cdot \text{prof}_A(i)) \end{aligned}$$

Sendo x_r a raiz de A , $\text{prof}_A(x_r) = 0$ e:

$$\begin{aligned} \text{Custo}(A) = & \sum_{i=1}^{r-1} \{\text{prob}(x_i) \cdot (\text{prof}_{AE}(x_i) + 1)\} + \sum_{i=0}^{r-1} \{\text{prob}(i) \cdot (\text{prof}_{AE}(i) + 1)\} + \\ & + \sum_{i=r+1}^n \{\text{prob}(x_i) \cdot (\text{prof}_{AD}(x_i) + 1)\} + \sum_{i=r}^n \{\text{prob}(i) \cdot (\text{prof}_{AD}(i) + 1)\} \quad (4.3.3) \end{aligned}$$

- Conhecida a raiz de A, por exemplo x_r :

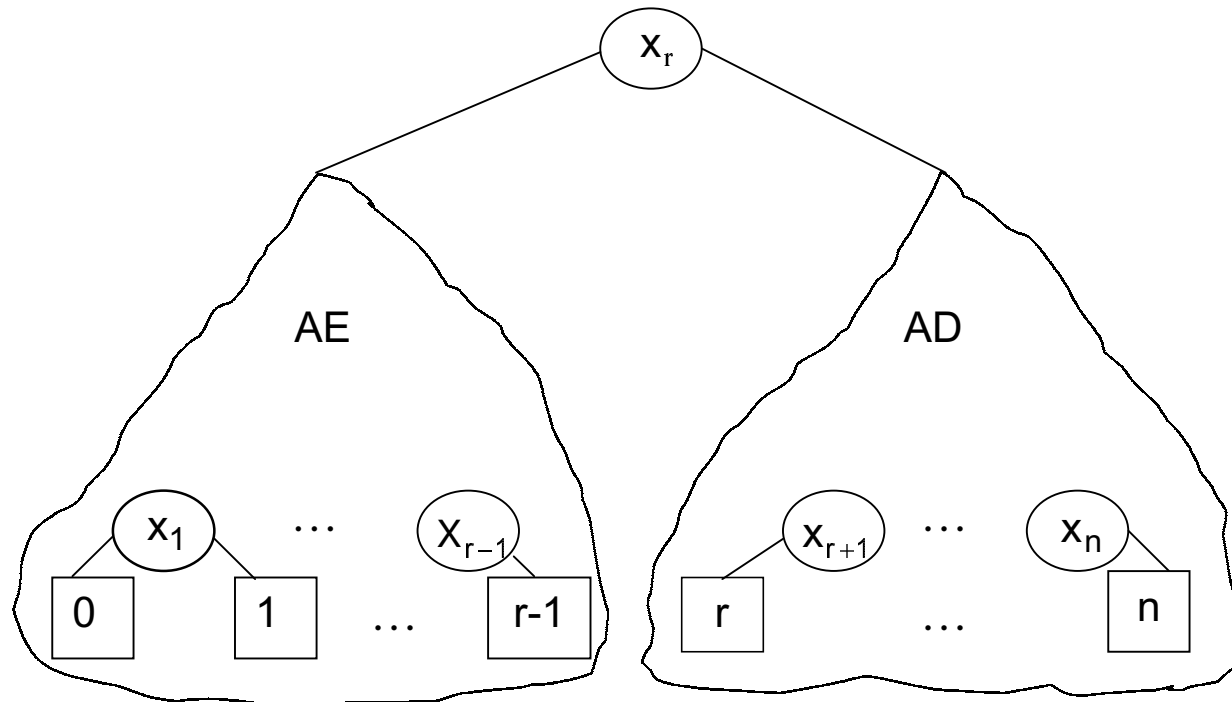


Figura 4.2.2: Subárvore da esquerda e da direita

- **AE** é a subárvore da esquerda
- **AD** é a subárvore da direita

- Temos os **custos** das subárvores dados por:

$$\text{Custo(AE)} = \sum_{i=1}^{r-1} \{\text{prob}(x_i) \cdot (\text{prof}_{\text{AE}}(x_i) + 1)\} + \sum_{i=0}^{r-1} \{\text{prob}(i) \cdot (\text{prof}_{\text{AE}}(i) + 1)\},$$

$$\text{Custo(AD)} = \sum_{i=r+1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=r}^n (\text{prob}(i) \cdot \text{prof}_A(i))$$

- Chamemos:

$$\text{Prob(AE)} := \sum_{i=1}^{r-1} \text{prob}(x_i) + \sum_{i=0}^{r-1} \text{prob}(i),$$

$$\text{Prob(AD)} := \sum_{i=r+1}^n \text{prob}(x_i) + \sum_{i=r}^n \text{prob}(i)$$

- A equação 4.3.3 é reescrita como:

$$\text{Custo (A)} = \text{Custo(AE)} + \text{Prob(AE)} + \text{Custo(AD)} + \text{Prob(AD)}.$$

- Assim, o problema para $S = \langle x_1, x_2, \dots, x_n \rangle$, a partir de uma escolha de raiz x_r , passa a ter **dois subproblemas**, para:
 $S_1 = \langle x_1, x_2, \dots, x_{r-1} \rangle$ e $S_2 = \langle x_{r+1}, x_{r+2}, \dots, x_n \rangle$.
- A questão então é a escolha adequada de r .
- Esse problema será resolvido construindo a solução a partir de um **subcaso** para o caso geral, partindo do caso inicial com a seqüência S vazia.
- Construção da solução passo a passo:

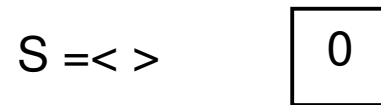


Figura 4.2.3: Árvore inicial

- Em um segundo passo:

$$S_1 = \langle x_1 \rangle, S_2 = \langle x_2 \rangle, \dots, S_n = \langle x_n \rangle;$$

são n seqüências de um elemento e serão obtidas n árvores, conforme a figura 4.2.4.

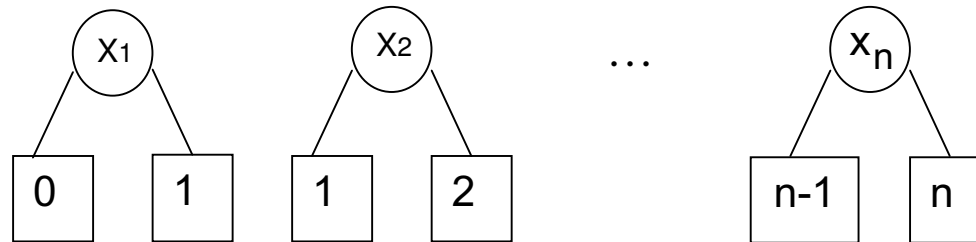


Figura 4.2.4: Primeiro nível de subárvores

- No próximo passo, consideramos seqüências com dois elementos:
 $S_1 = \langle x_1, x_2 \rangle$, $S_2 = \langle x_2, x_3 \rangle$, ..., $S_{n-1} = \langle x_{n-1}, x_n \rangle$.
- Para $S_1 = \langle x_1, x_2 \rangle$, há duas possibilidades, como mostra a figura 4.2.5.

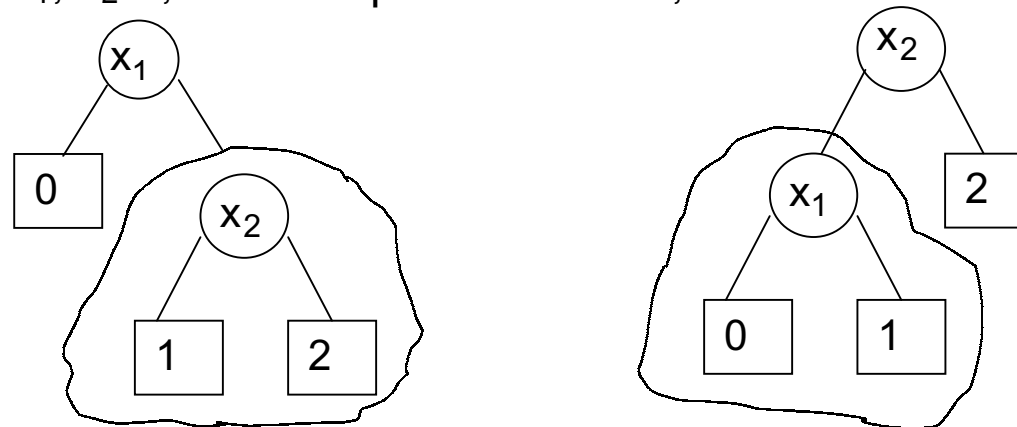


Figura 4.2.5: Subárvore de segundo nível

A de **menor custo** é escolhida.

- Assim, chegamos ao algoritmo a seguir.
- *Algoritmo 4.2.1: **Árvore de busca binária ótima***
- Este algoritmo determina uma árvore de busca binária de complexidade média ótima. Ele usa as probabilidades de procura dos elementos que estão e que não estão na árvore, e fornece as raízes das árvores ótimas em uma matriz.
- O algoritmo determina a melhor raiz, calcula os custos e a soma das probabilidades das subárvores menores (de altura s), guardando-os em raiz, em custo e em P , para utilizar nas próximas iterações. No final, o índice correspondente à raiz da árvore estará em $\text{raiz}[1, n]$ e, a partir daí, a árvore inteira pode ser recuperada.

Procedimento Árvore_Busca_Binária_Ótima { *árvore de busca binária ótima* }

 { **Entrada**: Prob (probab. de procura dos elementos dentro ou fora da árvore).

Saída: matriz raiz : M ($\text{raiz}[i, j]$ é raiz de árvore ótima para $\langle x_i, \dots, x_j \rangle$). }

sorte $M :=$ vetor $[1..n, 1..n]$ de $\mathbb{R}_+$;

var {auxiliar} $P, S : M$ {matrizes}; k {índice};

1. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u>	<i>{ inicializações }</i>
2. $\text{custo}[i, i] \leftarrow 0;$	
3. $P[i, i] \leftarrow \text{Prob}[i];$	
4. <u>fim-para</u> ;	<i>{ 1: i de 1 até n }</i>
5. <u>para</u> s <u>de</u> 1 <u>até</u> n <u>faça</u>	
6. <u>para</u> i <u>de</u> 0 <u>até</u> $n - s$ <u>faça</u>	
7. $j \leftarrow i + s;$	
8. <u>com</u> $S[i, k, j] := \text{custo}[i, k-1] + P[i, k-1] + \text{custo}[k, j] + P[k, j]$ ($i < k \leq j$)	
9. determine k tal que $S[i, k, j]$ seja mínimo para $i < k \leq j$;	
10. $\text{raiz}[i, j] \leftarrow k;$	
11. $\text{custo}[i, j] \leftarrow S[i, j, k];$	
12. <u>fim-com</u>	<i>{ 8: $S[i, k, j]$, para $i < k \leq j$ }</i>
13. $P[i, j] \leftarrow P[i, j - 1] + \text{Prob}[i] + \text{Prob}[j];$	
14. <u>fim-para</u> ;	<i>{ 6: i de 0 até $n - s$ }</i>
15. <u>fim-para</u> ;	<i>{ 5: s de 1 até n }</i>
16. <u>pare-com-saída</u> (custo , raiz);	<i>{ dá a saída }</i>
17. <u>fim-Procedimento</u> .	<i>{ fim de Árvore_Busca_Binária_Ótima }</i>

4.3 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é examinar a complexidade média, considerando casos ilustrativos de análise de algoritmos e de projeto de estruturas.
- Na seção 4.1, foi analisado o algoritmo de classificação Quicksort, como um exemplo clássico de complexidade média com distribuição uniforme. O desempenho desse algoritmo depende crucialmente de qual elemento vem a ser o pivô, por isso teve que ser determinada a probabilidade de posição do pivô.
- Mostrou-se que sua complexidade média é de ordem $O(n \cdot \log n)$. Esse valor é ótimo para algoritmos de classificação, conforme será visto mais tarde.
- A seção 4.2 examinou árvore de busca binária ótima como um exemplo de armazenamento de dados, utilizando um critério que tem como objetivo minimizar a complexidade média de procura.
- A partir de uma análise da influência de cada nodo na complexidade média obteve-se o algoritmo 4.2.1: Árvore de Busca Binária Ótima, que determina uma árvore de busca binária com complexidade média ótima.

5 PROJETO E ANÁLISE DE ALGORITMOS

Vamos examinar estratégias para projeto de algoritmos e sua complexidade.

Problema:

- Estrutura interna aproveitada na sua resolução.
- Relacionamento entre problemas:
 - caso especial;
 - versão abstração;
 - similar.

Exemplos:

Encontrar raízes complexas de polinômios quadráticos com coeficientes reais é um **caso especial** (bem mais simples) do problema de encontrar raízes complexas de polinômios de grau arbitrário com coeficientes reais.

Inserir um objeto em uma estrutura é uma **versão abstrata** do problema de inserir um elemento em uma lista.

A busca de um elemento em uma tabela ou em uma árvore são ditos **similares**.

Idéia: analisar famílias de problemas, cuja solução é alcançada de forma semelhante.

5.1 MÉTODO GULOSO

Vamos examinar algoritmos chamados gulosos. A **estratégia gulosa** é útil principalmente para resolver **problemas de otimização combinatória**, cuja solução pode ser alcançada por uma seqüência de decisões.

A estratégia gulosa pode ser aplicada a problemas de otimização combinatória, como por exemplo:

- intercalação sucessiva ótima de listas;
- caminhos de custo mínimo em grafos orientados;
- árvore geradora (ou de espalhamento) mínima em grafos não orientados;
- códigos para transmissão de mensagens (códigos de Huffman);
- escalonamento de tarefas.

Exemplo introdutório

Um exemplo simples que ilustra bem a estratégia gulosa é o problema da **intercalação sucessiva de listas**. Trata-se de intercalar n listas. Há várias maneiras possíveis de se realizar uma seqüência de intercalações. Ocorre que isso pode afetar o número de comparações necessárias.

Intercalação de Listas.

O número de comparações necessárias para intercalar duas listas, no **pior caso**, é a soma dos seus comprimentos, que também é o comprimento da lista resultante, menos 1.

Sejam três listas L_1 , L_2 e L_3 , com respectivos comprimentos 15, 10 e 5.

Uma maneira de proceder seria: primeiro intercalar L_1 com L_2 e, depois, o resultado $_1L_2$ com L_3

- O número de comparações é $(15+10-1)$, para produzir $_1L_2$, e $(15+10)+5-1$, para intercalá-la com L_3 .

- No total, temos $(15+10-1) + (15+10)+5-1$, ou seja, 53 comparações.

Uma **outra maneira** de proceder é: primeiro intercalar L_2 com L_3 e, depois L_1 com o resultado $_2L_3$.

O número de comparações é $(10+5-1)$, para produzir $_2L_3$, e $15+(10+5-1)$, para intercalar L_1 com $_2L_3$.

No total, temos $(10+5-1) + 15+(10+5-1)$, ou seja 43 comparações.

Então ordem em que são realizadas as intercalações pode influir substancialmente no **número de comparações** requeridas.

Agora vamos examinar mais de perto o efeito da **seqüência escolhida** no número de comparações requeridas para intercalar n listas L_i com comprimentos m_i , para $i = 1, \dots, n$.

Consideremos, no caso de $n = 3$ listas, as duas maneiras acima:

- na primeira maneira, o número total t' de comparações é $_1m_2 + (_1m_2 + m_3)$, ou seja, $2 \cdot (m_1 + m_2) + m_3$.
- na segunda maneira, número total t'' de comparações é $_2m_3 + (m_1 + _2m_3)$, ou seja, $2 \cdot (m_2 + m_3) + m_1$.

Se $_1m_2 \leq _2m_3$ i. e., $(m_1 + m_2) \leq (m_2 + m_3)$, teremos $t' \leq t''$; nesse caso, devemos escolher a primeira maneira.

É isso que sugere a **estratégia gulosa**: escolher o que dá menor contribuição ao resultado final.

De fato, consideremos intercalar as n listas L_i nessa ordem.

1. Inicialmente, intercalamos L_1 com L_2 , usando ${}_1m_2$ comparações e produzindo ${}_1L_2$ com comprimento ${}_1m_2$, onde ${}_1m_2 := (m_1 + m_2)$
- i. No passo i , intercalamos ${}_iL_i$ com L_{i+1} , usando ${}_im_{i+1}$ comparações e produzindo ${}_iL_{i+1}$ com comprimento ${}_im_{i+1}$, onde ${}_im_{i+1} := ({}_im_i + m_{i+1})$
- n-1. Finalmente, intercalamos ${}_nL_{n-1}$ com L_n , usando ${}_nm_n := ({}_nm_{n-1} + m_n)$ comparações

O número total de comparações é dado por ${}_1m_2 + \dots + {}_im_{i+1} + \dots + {}_nm_n$, ou seja,

$$(n-1) \cdot m_1 + (n-1) \cdot m_2 + (n-2) \cdot m_3 + \dots + (n-i) \cdot m_{i+1} + \dots + m_n - (n-1)$$

O primeiro par de listas a ser intercalado dá contribuição maior para o peso total.

Consideremos **5 listas**, com comprimentos 10, 20, 30, 40 e 50, a intercalar:

1. Inicialmente, escolhemos as duas de menores comprimentos (10 e 20) ficando com 4 listas, com comprimentos 30, 30, 40 e 50.
2. Repetindo o processo, escolhemos as de comprimentos 30 e 30 ficando com 3 listas, com comprimentos 60, 40 e 50.
3. Agora, escolhemos as de comprimentos 40 e 50, ficando com 2 listas, com comprimentos 60, 90.
4. Finalmente, substituímos essas duas listas pelo resultado de sua intercalação, ficando com uma lista: o resultado final.

10	20	30	40	50			
1	2	3					
1	4	3					
	2	60	1	2	3		
1	4	4	4	2	4	3	90
				4	150		

Algoritmo : Intercalação ótima de listas

Função Interc_Suc_Lst($L : D$) $\rightarrow R$ {intercalação ótima de listas}

{ *Entrada*: multiconjunto $L : D$ de n listas (com seus comprimentos) a intercalar
Saída: par (intercalação das n listas em L , número mínimo de comparações) }

sorte $D :=$ multiconjunto de $\mathbb{N}$ {comprimentos de listas} ;

var {auxiliares} $ncp : \mathbb{N}$; $L^*, L', L'' : \text{Listas}$;

1. $ncp \leftarrow 0$; {inicializa saída}
2. repita {itera até $|L| = 1:8$ }
3. escolhe as duas menores listas $L' \text{ e } L''$ em L ; {seleção gulosa}
4. $L \leftarrow L - \{ L', L'' \}$; {remove elementos selecionados}
5. $L^* \leftarrow \text{Interc}(L', L'')$; { L^* : intercalação de listas selecionadas}
6. $L \leftarrow L \cup \{L^*\}$; {inclui resultado da intercalação}
7. $ncp \leftarrow ncp + \text{tam}(L') + \text{tam}(L'') - 1$; {atualiza solução}
8. até-que $|L| = 1$; {2: uma única lista}
9. $L^* \leftarrow$ a única lista em L ; {recupera lista encadeada}
10. retorne-saída (L^*, ncp) ; {dá saída recuperada}
11. fim-Função {fim do algoritmo Interc_Suc_Lst}

Idéias básicas

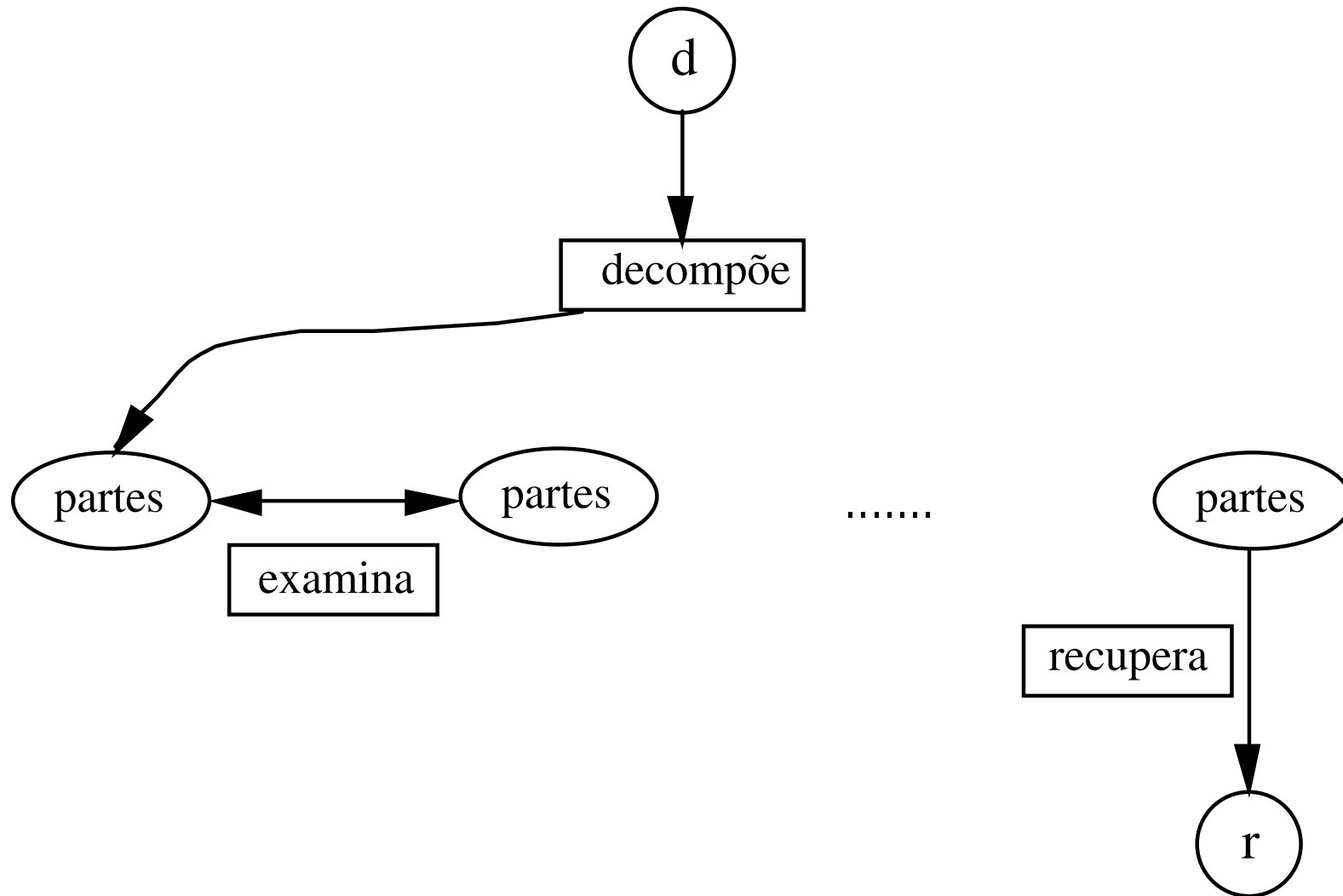
Um algoritmo guloso para seqüência ótima de intercalações seleciona, a cada passo um par com **soma mínima** dentre os presentes para fazer parte da solução.

Em cada passo:

- seleciona um elemento e decide-se se ele é viável - vindo a fazer parte da solução - ou não.
- após uma seqüência de decisões, a solução do problema é alcançada.

Na **seqüência de decisões**, nenhum elemento é examinado mais de uma vez: ou ele fará parte da saída, ou será descartado.

O diagrama a seguir mostra a estrutura geral dessa abordagem.



Estrutura geral do método guloso

A idéia básica da **estratégia gulosa** sugere uma estrutura geral para algoritmos gulosos: uma seqüência, consistindo de um trecho de inicialização, seguido de uma iteração e um trecho de finalização.

Assim, um algoritmo guloso tem a forma

Inicialização ; Iteração ; Finalização.

A **inicialização** prepara a entrada (muitas vezes a entrada é classificada) e inicializa a saída.

A **iteração** seleciona um elemento conforme uma função gulosa, marca-o para não considerá-lo novamente no futuro, atualiza a entrada, examina o elemento selecionado quanto sua viabilidade e decide a sua participação ou não na solução.

A **finalização** recupera a saída.

Dessa maneira, pode-se chegar ao seguinte esquema geral de algoritmo guloso.

A estrutura algorítmica do método guloso é dada pelo seguinte programa abstrato.

Algoritmo : Algoritmo Guloso

Função Alg_Gul($d:D$) $\rightarrow R$ {Algoritmo Guloso (abstrato)}

{ *Entrada-saída*: saída $r:R$ é resposta ótima para entrada $d:D$ }

Inc_z_G ; Iter_G ; Fnl_G {estrutura geral}

onde {componentes}

Inc_z_G: {inicialização}

inicializa parte da entrada a examinar e resposta parcial

Iter_G: itera corpo Crp_G {iteração}

seleciona elemento e da parte da entrada a examinar;

remove elemento e da parte da entrada a examinar;

inclui elemento e na resposta parcial se viável

Fnl_G: {finalização}

recupera a saída a partir da resposta parcial

No algoritmo Interc_Suc_Lst (para intercalação ótima de listas) apresentado no exemplo, podemos estabelecer as seguintes identificações:

Interc_Suc_Lst	Alg_Gul
<u>sorte</u> D := multiconjunto de listas	<u>sorte</u> D {entrada}
1. $ncp \leftarrow 0$;	IncZ_G {inicialização}
2. <u>repita</u>	Iter_G {iteração}
3. $(L', L'') \leftarrow 2 \text{ menores em } L$;	seleciona e da parte da entrada;
4. $L \leftarrow L - \{L', L''\}$;	
5. $L^* \leftarrow \text{Interc}(L', L'')$;	remove e da parte da entrada;
6. $L \leftarrow L \cup \{L^*\}$;	
7. $ncp \leftarrow ncp + \text{tam}(L') + \text{tam}(L'') - 1$;	inclui e na resposta parcial se viável;
8. <u>até-que</u> $ L = 1$;	{fim da iteração}
9. $L^* \leftarrow \text{a única lista em } L$;	FnI_G {finalização}

Tabela : Identificações entre Interc_Suc_Lst e Alg_Gul

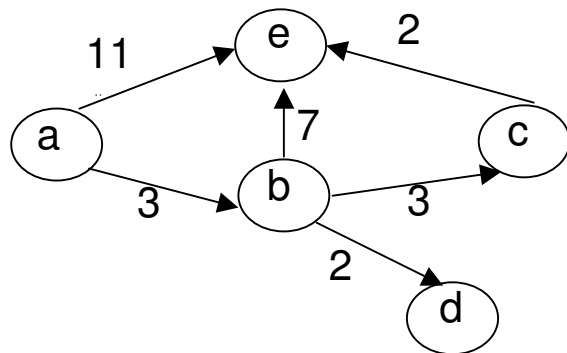
PROJETO DE ALGORITMOS PELA ESTRATÉGIA GULOSA

Agora vamos examinar o emprego da estratégia gulosa para desenvolver algoritmos para alguns **problemas típicos**.

Problema caminhos de custo mínimo em grafo orientado a partir de fonte

O problema consiste em determinar um **caminho de custo mínimo** a partir de um vértice fonte a cada vértice do grafo.

Consideremos um grafo orientado $G = \langle V, E \rangle$ com 5 vértices: $V = \{a, b, c, d, e\}$ e 8 arestas com a seguinte matriz de custos:



custo	a	b	c	d	e
a		3	∞	∞	11
b	∞		3	2	7
c	∞	∞		∞	2
d	∞	∞	∞		∞
e	∞	∞	∞	∞	

A fonte v_0 é o vértice a . Assim, inicialmente o conjunto I de vértices intermediários é vazio, e o vetor de distâncias é a primeira linha da matriz de custos acima, ou seja:

b	c	d	e
3	∞	∞	11

Em **cada passo**, seleciona-se um vértice que tenha distância mínima como novo intermediário, e são atualizadas as distâncias.

Assim, obtém-se a seguinte seqüência de passos:

passo	I	vértice	b	c	d	e
1	$\emptyset$	1: b	3	∞	5	10
2	{b}	2: d	3	6	5	10
3	{b, d}	3: c	3	6	5	8
4	{b, c, d}	4: e	3	6	5	8

Após o passo 4, teremos $I = \{b, c, d, e\}$, sendo os custos mínimos de caminhos da fonte a cada vértice alcançável dados pela última linha, ou seja:

b	c	d	e
3	6	5	8

Vamos construir incrementalmente um conjunto I de vértices e os caminhos a partir da fonte de custo mínimo, usando como intermediários apenas vértices no conjunto I .

Em **cada passo**, seleciona-se um novo vértice com distância mínima e são atualizados os custos mínimos a partir da fonte.

Quando $|I|=n$, teremos todos os caminhos com custo mínimo a partir da fonte.

Assim, podemos obter o seguinte algoritmo (onde a variável p mantém o conjunto de vértices ainda a considerar como possíveis intermediários).

Algoritmo: Custo mínimo de caminhos a partir de fonte em grafo orientado

Função $\text{Dist_fnt}(d:D) \rightarrow R$ {distância a partir da fonte em grafo}

{Entrada: grafo orientado valorado G com fonte v_0 e matriz custo: M (de custos)}

Saída: vetor $\text{dist}:V'$ (custos dos melhores caminhos a partir de v_0)}

sorte $D := \text{Grafos valorados com fonte}; R := V'; M := \underline{\text{matriz}}[0..n, 0..n] \underline{\text{de}} \mathbb{R}^+;$
 $V' := \underline{\text{vetor}}[1..n] \underline{\text{de}} \mathbb{R}^+; Vrt := \text{Vértices}; P := \text{Conj}[Vrt] \text{ {conjuntos de vértices}};$
var {auxiliares} $p:P$ {parte da entrada}; $e:Vrt$ {seleção};
 $V_0:Vrt$ {vértices não fonte}; $c:\mathbb{R}^+$ {distância};

0. $V_0 \leftarrow V - \{v_0\};$ {vértices não fonte: $v_1, \dots, v_n$ }
1. $p \leftarrow V_0;$ {inicializa porção da entrada com $\{v_1, \dots, v_n\}$ }
2. para i de 1 até n faça $\text{dist}[i] \leftarrow \text{custo}[0, i];$ {inicializa resposta parcial}
3. para i de 1 até n faça {itera: 10}
4. $e \leftarrow \text{vértice } v_j \in p \text{ com } \text{dist}[j] \text{ mínimo};$ {seleciona novo vértice}
5. $p \leftarrow p - \{e\};$ {remove vértice selecionado}
6. para cada $v_i \in V_0$ faça {vértices $v_1, \dots, v_n$: 9}
7. $c \leftarrow \text{dist}[j] + \text{custo}[v_j, v_i];$ {distância usando v_j }
8. se $c \leq \text{dist}[i]$ então $\text{dist}[i] \leftarrow c;$ {atualiza distância}
9. fim-para {6: cada $v_i \in V_0$ }
10. fim-para {3: repita}
11. retorne-saída(dist); {dá como saída resposta pronta}
12. fim-Função {fim do algoritmo Dist_fnt: distância a partir da fonte}

Pode-se determinar um **caminho de custo mínimo** a partir da fonte, guardando-se os valores de e , obtidos na linha 4 do algoritmo.

Podemos ver que esse algoritmo `Dist_fnt` foi obtido pela estratégia gulosa observando o que segue:

- as linhas de 0 a 2 fazem a inicialização: as linhas 0 e 1 preparam a porção da entrada e a linha 2 inicializa a saída;
- a iteração está nas linhas de 3 a 10: a linha 4 seleciona um elemento, que é removido na linha 5, sendo incluído na resposta parcial se viável nas linhas de 6 a 9;
- a finalização está na linha 11, quando se recupera a solução.

Problema de árvore geradora (ou de espalhamento) mínima

Uma **árvore geradora** (ou de espalhamento) de um grafo G é um subgrafo acíclico (i. e. sem ciclos) que contém todos os vértices (nodos) do grafo.

Se o grafo não é conexo, cada componente terá uma árvore geradora.

Uma **estratégia** sugerida pelo método guloso para determinar uma árvore geradora de custo mínimo é a seguinte:

As arestas serão consideradas em ordem não decrescente de seus custos, e vamos construir uma floresta de árvores.

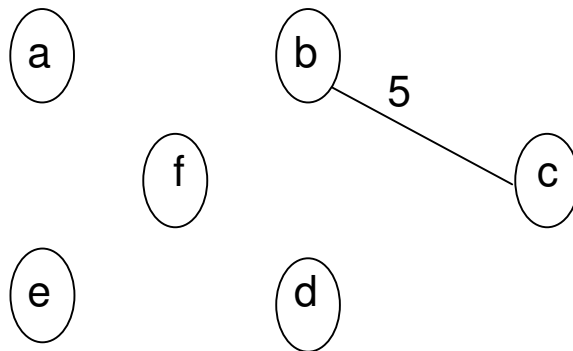
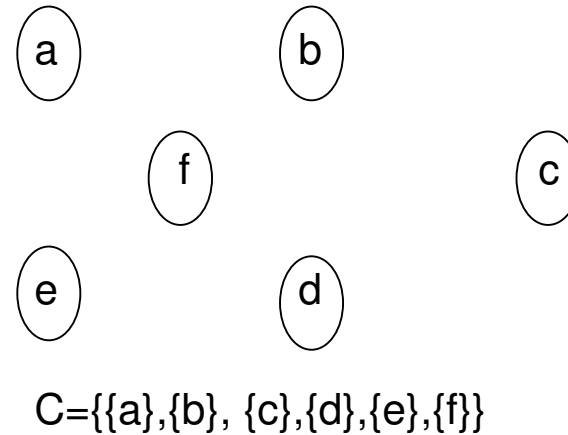
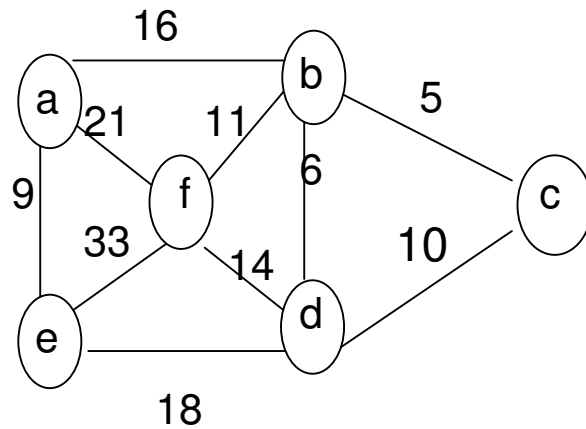
A cada passo, seleciona-se uma **aresta de custo mínimo** dentre as ainda não examinadas. Se sua inclusão criar um ciclo, ela é descartada; caso contrário, ela é incluída.

Consideremos um grafo não orientado $G = \langle V, E \rangle$ com

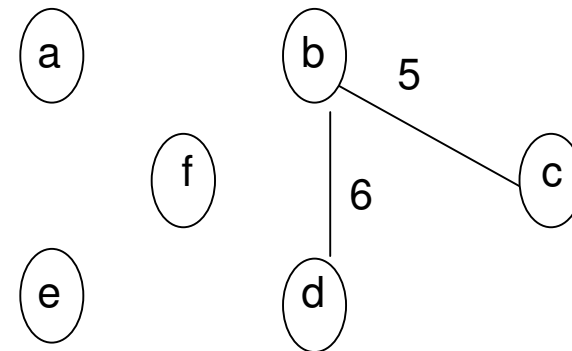
- 6 vértices: $V = \{a, b, c, d, e, f\}$;
- 10 arestas: $E = \{(a, b), (a, e), (a, f), (b, c), (b, d), (b, f), (c, d), (d, e), (d, f), (e, f)\}$;
- custos:

$\text{custo}(a, b) = 16,$	$\text{custo}(a, e) = 9,$	$\text{custo}(a, f) = 21,$
$\text{custo}(b, c) = 5,$	$\text{custo}(b, d) = 6,$	$\text{custo}(b, f) = 11,$
$\text{custo}(c, d) = 10,$	$\text{custo}(d, e) = 18,$	$\text{custo}(d, f) = 14,$
$\text{custo}(e, f) = 33.$		

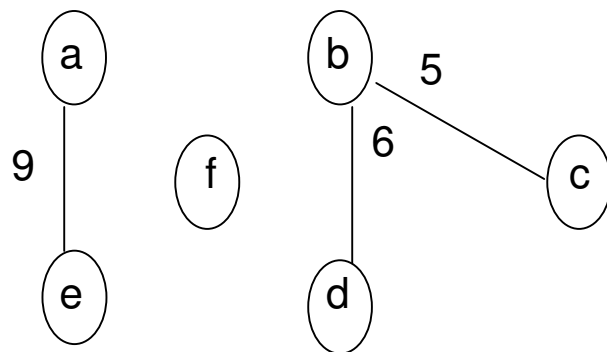
Uma **árvore espalhada mínima** para G pode ser construída de acordo com a estratégia acima.



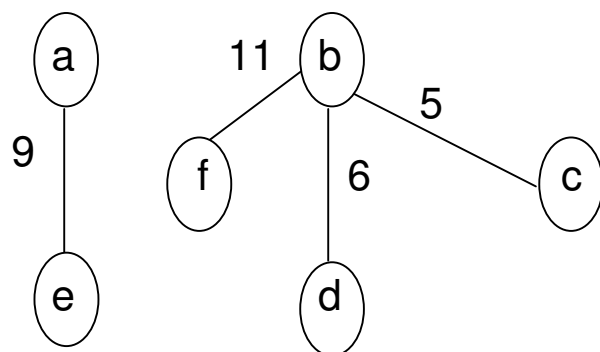
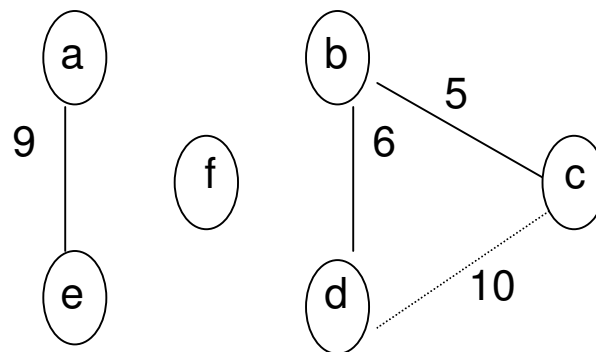
$$C = \{\{a\}, \{b, c\}, \{d\}, \{e\}, \{f\}\}$$



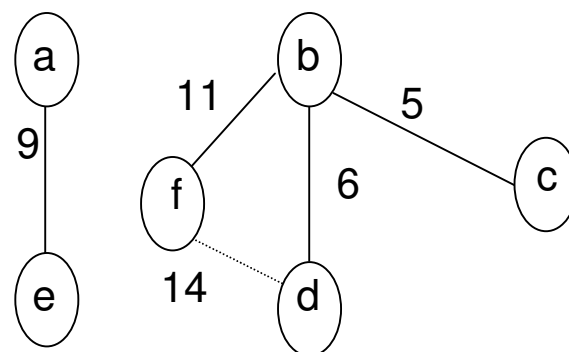
$$C = \{\{a\}, \{b, c, d\}, \{e\}, \{f\}\}$$

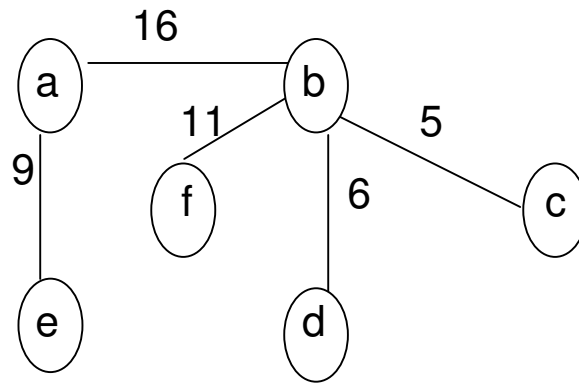


$$C = \{\{a, e\}, \{b, c, d\}, \{f\}\}$$



$$C = \{\{a, e\}, \{b, c, d, f\}\}$$





$$C = \{\{a, e, b, c, d, f\}\}$$

Construção de árvore espalhada mínima

Agora, vejamos como essas idéias levam ao projeto de um algoritmo para esse problema de árvore geradora (ou de espalhamento) mínima para um **grafo** (não orientado) pela estratégia gulosa.

A estratégia gulosa sugere selecionar arestas em **ordem crescente de custos**.

Construir um floresta usando as arestas dadas: cada aresta selecionada será incluída se não originar ciclo, podendo levar à fusão de componentes. Quando a floresta só tiver único componente, teremos uma **árvore geradora mínima**.

Algoritmo: Árvore geradora mínima de grafo

Função Árv_ger_mín($d:D$) $\rightarrow R$ {árvore geradora mínima de grafo conexo}

{*Entrada*: grafo não orientado (conexo) $d:D$ com m arestas valoradas}

Saída: árvore geradora mínima $r:R$ do grafo (conexo) $d:D$ }

sorte $D :=$ Grafos valorados; $R :=$ Árvores (valoradas); $E :=$ Arestas valoradas;

$P :=$ Filas de arestas; $S :=$ Florestas de árvores;

var $p:P$ {parte da entrada}; $s:S$ {parte da saída}; $e:E$ {elemento selecionado};

1. $p \leftarrow \langle e_1, \dots, e_m \rangle$; {fila com arestas em ordem de custos}
2. $s \leftarrow \{ \langle \{v\}, \emptyset \rangle / v \in V \}$; {floresta com componentes unitários}
3. repita {itera até $|s.V| = 1$: 9}
4. $(u, v) \leftarrow \text{prim}(p)$; {seleciona aresta com custo mínimo}
5. $p \leftarrow \text{rem_P}((u, v), p)$; {remove aresta selecionada da fila}
6. $A' \leftarrow \text{comp}(u, s)$; $A'' \leftarrow \text{comp}(v, s)$; {componentes dos vértices}
7. se $A' \neq A''$ {testa criação de ciclo}
8. então substitua em s : $A' \notin A''$ por $A' \cup A''$; {inclui aresta}
9. até-que $|s.V| = 1$; {3: floresta com um único componente}
10. $r \leftarrow \text{árv}(s)$; {recupera árvore de floresta pronta}
11. retorne-saída(r); {dá saída r }
12. fim-Função {fim do algoritmo Árv_ger_mín: de grafo conexo}

Note que esse algoritmo `Árv_ger_mín` consegue determinar uma árvore geradora mínima (de um grafo conexo) sem ter necessidade de examinar todas as suas arestas. Essa é a razão de se utilizar uma iteração indefinida.

- as linhas 1 e 2 fazem a **inicialização**: a linha 1 prepara a porção da entrada (ordenada) e a linha 2 inicializa a saída;
- a **iteração** está nas linhas de 3 a 9: a linha 4 seleciona um elemento, que é removido na linha 5, sua viabilidade de estar ou não na solução é testada na linha 7, sendo incluído na resposta parcial se viável na linha de 8;
- a **finalização** está na linha 10, quando se recupera a solução.

Vimos três exemplos de algoritmos desenvolvidos pela estratégia gulosa

Cada um desses três algoritmos gulosos tem a estrutura geral de seqüência

Inicialização ; Iteração ; Finalização.

No algoritmo `Dist_fnt`, a iteração (nas linhas de 3 a 10) é definida: para ... faça Nos outros dois primeiros algoritmos (`Interc_Suc_Lst` e `Árv_ger_mín`) a iteração principal é indefinida: repita ... até-que Mas, já sabemos que uma iteração definida pode ser simulada por uma iteração indefinida (usando um contador como variável de controle).

COMPLEXIDADE DE ALGORITMOS GULOSOS

Complexidade de distância de fonte em grafo orientado

Considere o algoritmo `Dist_fnt` para distância (custo mínimo de caminhos) a partir de fonte em grafo orientado.

Como só é usada a operação de **retirada de elemento do conjunto**, pode-se usar uma representação com complexidade de retirada constante (linhas 0 e 1).

A linha 2 é executada n vezes. Assim, a complexidade da inicialização (linhas 0, 1 e 2) tem ordem **$O(n)$** .

A complexidade da linha 4 é o tamanho de p , que começa com n ($n = |V_0|$) e diminui de 1 a cada iteração.

A iteração das linhas de 3 a 10 é executada n vezes. O trecho das linhas 4 e 5 tem complexidade n . Assim, o corpo da iteração (linhas de 4 a 9) tem complexidade $|p| + n$, que varia durante a iteração ($|p| = n - i$, ou seja $(n - i) + n$).

Logo, a complexidade da iteração é dada pela somatória.

$$\sum_{i=0}^n [(n-i) + n] = O(n + n^2). \text{ Assim, } c_p^{\leq} [\text{Dist_fnt}](n) = O(n + n^2) = O(n^2).$$

Agora, vamos fazer uma **análise** mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse algoritmo Dist_fnt recebe como **entrada** $d:D$

um grafo orientado valorado G com fonte v_0 e matriz de custos

e fornece como **saída**

um vetor dist (com os custos dos melhores caminhos a partir de v_0).

Consideremos um grafo orientado G , com conjunto $V = \{v_0, v_1, \dots, v_n\}$ de vértices. Tomemos, como é natural, para tamanho da entrada o número n de vértices não fonte.

Nesse caso, parece natural tomar como **operações fundamentais** as manipulações com conjuntos (de vértices) e matrizes.

Como o algoritmo `Dist_fnt` é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de `Dist_fnt` sobre uma entrada $d:D$ tem contribuições dadas por suas componentes: inicialização, iteração e finalização.

Para determinar essas contribuições, devemos obter os valores das entradas de cada uma dessas três componentes.

A inicialização (linhas de 0 a 2) fornece valores iniciais às variáveis:

- $p:P$ {parte da entrada}, $p_0 := \{v_1, \dots, v_n\}$ {tamanho $|p_0| = n$ }, e
- $\text{dist}:V'$ {custos dos melhores caminhos a partir de v_0 },
 $\text{dist}_0 := [\text{custo}[0, 1], \dots, \text{custo}[0, n]]$ {tamanho $\dim(\text{dist}_0) = n$ }

Assim, a saída da inicialização é $\text{Incz_Dist_fnt}(G) = (p_0, \text{dist}_0)$.

O desempenho da inicialização tem as seguintes contribuições:

comando	desempenho
0. $V_0 \leftarrow V - \{v_0\};$	1
1. $p \leftarrow v_0;$	n
2. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u> $\text{dist}[i] \leftarrow \text{custo}[0, i];$	n

Assim, temos:

$$\text{desemp}[\text{Incz_Dist_fnt}](G) = 1 + n + n \cdot n = 2 \cdot n + 1.$$

Agora, a iteração para i de 1 até n (linhas de 3 a 10) repete n vezes o corpo, selecionando um elemento, removendo-o e incluindo-o na resposta parcial se viável (linhas de 4 a 9).

São gerados novos valores para as variáveis $p:P$ e $\text{dist}:V'$ através de valores da variável $e:E$ {seleção}

$e:E$	$e_1 \in p_0$	...	$e_k \in p_{k-1}$	...	$e_n \in p_{n-1}$
$p:P$	$p_1 := p_0 - \{e_1\}$		$p_k := p_{k-1} - \{e_k\}$		$p_n = p_{n-1} - \{e_n\}$
$\text{dist}:V'$	dist_1	...	dist_k	...	dist_n

Assim, os valores sucessivos das variáveis $p:P$ e $\text{dist}:V$ têm tamanhos como se segue:

l	0	1	$\dots$	k	$\dots$	n
$ p_i $	n	$n-1$	$\dots$	$n-k$	$\dots$	0
$\text{Dim}(\text{dist}_i)$	n	n	$\dots$	n	$\dots$	n

Ao início da $i^{\text{ésima}}$ iteração, as variáveis $p:P$ têm tamanhos $|p_i| = n-i+1$ e $\text{dist}:V$ têm tamanhos $\text{dim}(\text{dist}_i) = n$.

Agora, o desempenho do corpo **Crp_Dist_fnt** (linhas de 4 a 9) em cada iteração é dado pela soma das contribuições das linhas de 4 a 5 e das linhas de 6 a 9.

Na $i^{\text{ésima}}$ iteração, as linhas 4 e 5 atuam sobre variáveis $p:P$ e $\text{dist}:V$ com tamanhos $|p_i| = n-i+1$ e $\text{dim}(\text{dist}_i) = n$. Assim, temos as seguintes cotas superiores para o desempenho das linhas 4 e 5:

Comando	cota superior
4. $e \leftarrow$ vértice $v_j \in p$ com $\text{dist}[j]$ mínimo;	$n-i+1$
5. $p \leftarrow p - \{e\}$;	$n-i+1$

Assim, $\text{desemp}[\text{linhas 4-5}](p_i, \text{dist}_i) \leq (n-i+1) + (n-i+1) \leq 2 \cdot (n+1)$.

Na $i^{\text{ésima}}$ iteração, as linhas de 6 a 9 atuam sobre variáveis $p:P$ e $\text{dist}:V$ com tamanhos $|p_{i+1}| = n-i$ e $\text{dim}(\text{dist}_i) = n$.

Assim, temos as seguintes **cotas superiores** para desempenho das linhas 7 e 8:

comando	cota superior
7. $c \leftarrow \text{dist}[j] + \text{custo}[v_j, v_i];$	1
8. <u>se</u> $c \leq \text{dist}[i]$ <u>então</u> $\text{dist}[i] \leftarrow c$	1
linhas 7-8	$1 + 1 = 2$

Como na $i^{\text{ésima}}$ iteração, as linhas 7 e 8 são executadas com $n = |V_0|$ vezes, temos:

$$\text{desemp}[\text{linhas 6-9}](p_{i+1}, \text{dist}_i) \leq n \cdot 2 = 2 \cdot n.$$

Uma **cota superior** para o desempenho do corpo Crp_Dist_fnt na $i^{\text{ésima}}$ iteração é dado pela soma das cotas para as contribuições conforme a tabela seguinte:

$\text{desemp}[\text{linhas 4-5}](p_i, \text{dist}_i)$	$\leq$	$2 \cdot (n+1) = 2 \cdot n + 2$
+		
$\text{desemp}[\text{linhas 6-9}](p_{i+1}, \text{dist}_i)$	$\leq$	$2 \cdot n$
$\text{desemp}[\text{Crp_Dist_fnt}](p_i, \text{dist}_i)$	$\leq$	$2 \cdot (n+1) + 2 \cdot n = 4 \cdot n + 2$

Agora, como a iteração `Iter_Dist_fnt` (linhas de 3 a 10) repete n vezes o corpo `Crp_Dist_fnt`, uma cota superior para o desempenho da iteração `Iter_Dist_fnt` é $n \cdot (4 \cdot n + 2) = 4 \cdot n^2 + 2 \cdot n$.

Assim, temos para o desempenho de `Iter_Dist_fnt`

$$desemp[Iter_Dist_fnt](l_0, dst_0) \leq 4 \cdot n^2 + 2 \cdot n.$$

Agora, o desempenho do algoritmo 5.1.3: `Dist_fnt` é a soma das duas contribuições:

$desemp[Incz_Dist_fnt](G)$ e $desemp[Iter_Dist_fnt][Incz_Dist_fnt](G)$:

$$\begin{array}{rcl} desemp[Incz_Dist_fnt](G) & = & 2 \cdot n + 1 \\ + & & \\ desemp[Iter_Dist_fnt](p_0, dist_0) & \leq & 4 \cdot n^2 + 2 \cdot n \\ \hline desemp[Dist_fnt](G) & \leq & 4 \cdot n^2 + 4 \cdot n + 1 \end{array}$$

Portanto, temos para o desempenho de `Dist_fnt`

$$desemp[Dist_fnt](G) \leq 4 \cdot n^2 + 4 \cdot n + 1.$$

Assim, para a complexidade pessimista de `Dist_fnt` temos

$$c_p^{\leq} [\text{Dist_fnt}](n) = O(4 \cdot n^2 + 4 \cdot n + 1) = O(n^2).$$

Logo, o algoritmo: `Dist_fnt` (para distância de fonte em grafo orientado) tem complexidade pessimista quadrática:

$$c_p^{\leq} [\text{Dist_fnt}](n) = O(n^2).$$

Complexidade de intercalação ótima de listas

Considere o algoritmo `Interc_Suc_Lst` (para intercalação ótima de listas).

O algoritmo efetua operações sobre o multiconjunto L , portanto a complexidade do algoritmo pode variar conforme a estrutura de dados escolhida para representar L .

Vamos considerar a representação do multiconjunto L por uma lista encadeada, em que cada elemento guarda tanto uma lista quanto seu tamanho.

- para determinar as duas menores listas, a lista é percorrida, com complexidade $O(n)$;

- **para remoção** a complexidade é $O(n)$;
- **para a inclusão** a complexidade é constante (pois, como um multiconjunto pode ter diversas ocorrências de um mesmo elemento, não se necessita verificar a repetição de elementos).

Assim, podemos estimar a complexidade pessimista do algoritmo Interc_Suc_Lst, desprezando a inicialização (linha 1) e a finalização (linhas 9 e 10), nos concentrando na iteração (linhas de 2 e 8).

A cada iteração do corpo (linhas de 3 e 7), o tamanho de L diminui de 1, logo a iteração é executada para $|L|$ variando de n a 1.

A **complexidade do corpo da iteração** tem ordem

$$O(n) + O(1) + O(m' + m'' - 1),$$

onde m' e m'' são os tamanhos das duas listas selecionadas.

A ordem da complexidade da iteração é dada pela somatória

$$\sum_{i=0}^{n-2} (n + m' + m'').$$

Mas $\sum_{i=0}^{n-2} n = O(n^2)$ e $\sum_{i=0}^{n-2} (m' + m'') = O(n^2 \cdot m)$, onde m é o máximo dos comprimentos das n listas em L (pois $m' + m'' \leq n \cdot m$).

Portanto, a **complexidade do algoritmo Interc_Suc_Lst** tem ordem

$$c_p^{\leq} [\text{Interc_Suc_Lst}](n) = O(n^2 + n^2 \cdot m) = O(n^2 \cdot m).$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, como antes usando as idéias vistas na seção 3.2 do capítulo 3.

Esse algoritmo Interc_Suc_Lst recebe como **entrada** $d:D$

multiconjunto L de n listas (com seus comprimentos)

e fornece como **saída**

o par (intercalação das n listas em L , número mínimo de comparações).

Nesse caso, parece natural tomar como **operação fundamental** a comparação entre elementos das listas.

Também, parece natural tomar como tamanho da entrada o par formado pelo tamanho n do multiconjunto L e pelo comprimento m da maior lista em L .

Como o algoritmo `Interc_Suc_Lst` é uma seqüência, seu desempenho sobre uma entrada $L:D$ tem contribuições dadas por suas componentes:

Inicialização, Iteração e Finalização.

A **inicialização** (linha 1) e a **finalização** (linhas 9 e 10) não envolvem comparações.

Assim:

$desemp[Inc_{Interc_Suc_Lst}](L) = 0$ e $desemp[Fnl_Interc_Suc_Lst](L) = 0$.

Agora, examinemos a **iteração**:

repita até-que $|L| = 1$ (linhas de 2 a 8).

A cada **iteração** do corpo (linhas de 3 a 7), as duas menores listas em L são substituídas por sua intercalação, fazendo o tamanho de L diminuir de 1, logo a iteração é executada para $|L|$ variando de n a 1: n vezes.

Realmente, os valores sucessivos da variável $L:D$ têm tamanhos como se segue

i	0	1	...	k	...	$n-1$
$ L_i $	n	$n-1$	...	$n-k$	...	1

Na i ésima **iteração**, a variável $L:D$ tem tamanho $|L_i| = n-i$. Assim, temos as seguintes cotas superiores para o desempenho das linhas de 3 e 7:

comando	cota superior
3. $(L', L'') \leftarrow 2 \text{ menores em } L;$	$n-i$
4. $L \leftarrow L - \{L', L''\};$	$n-i$
5. $L^* \leftarrow \text{Interc}(L', L'');$	$\text{compr}(L') + \text{compr}(L'') - 1$
6. $L \leftarrow L \cup \{L^*\};$	0
7. $\text{ncp} \leftarrow \text{ncp} + \text{compr}(L') + \text{compr}(L'') - 1;$	0

Assim, com m'_i e m''_i para os tamanhos das duas listas selecionadas na i ésima iteração, temos

$$\begin{aligned} \text{desemp}[\text{Crp_Interc_Suc_Lst}](L_i, \text{ncp}_i) &\leq n-i + n-i + m'_i + m''_i + 0, \text{ logo,} \\ \text{desemp}[\text{Crp_Interc_Suc_Lst}](L_i, \text{ncp}_i) &= 2 \cdot (n-i) + m'_i + m''_i \leq 2 \cdot n + m'_i + m''_i. \end{aligned}$$

Portanto, uma cota superior para o desempenho da iteração $\text{Iter_Interc_Suc_Lst}$ é a somatória das $n-1$ contribuições:

$$2 \cdot n + m'_0 + m''_0 + \dots + 2 \cdot n + m'_i + m''_i + \dots + 2 \cdot n + m'_{n-2} + m''_{n-2}$$

Essa **cota superior para a iteração** é:

$$\sum_{i=0}^{n-2} (2 \cdot n + m'_i + m''_i) = 2 \cdot \sum_{i=0}^{n-2} n + \sum_{i=0}^{n-2} (m'_i + m''_i).$$

Agora, $\sum_{i=0}^{n-2} n = (n-1) \cdot n$. Por outro lado, tomando m como o máximo dos comprimentos das n listas em L , temos (já que $m'_i + m''_i \leq n \cdot m$)

$$\sum_{i=0}^{n-2} (m'_i + m''_i) \leq \sum_{i=0}^{n-2} (n \cdot m) = \sum_{i=0}^{n-2} m = (n-1) \cdot n \cdot m.$$

Assim, temos para o **desempenho de Iter_Interc_Suc_Lst**

$$desemp[Iter_Interc_Suc_Lst](L) \leq (n-1) \cdot n + (n-1) \cdot n \cdot m.$$

Agora, o desempenho do algoritmo Interc_Suc_Lst é a soma das três contribuições devidas:

à inicialização, à iteração e à finalização.

Portanto, temos para o **desempenho de Interc_Suc_Lst**

$$desemp[Interc_Suc_Lst](L) \leq 0 + (n-1) \cdot n \cdot (1+m) + 0.$$

Assim, para a complexidade pessimista de Interc_Suc_Lst temos

$$c_p^{\leq}[\text{Interc_Suc_Lst}](n, m) = O((n-1) \cdot n \cdot (1+m)) = O(n^2 \cdot m).$$

Portanto, o algoritmo **Interc_Suc_Lst** (para intercalação ótima de listas) tem **complexidade pessimista**

$$c_p^{\leq}[\text{Interc_Suc_Lst}](n, m) = O(n^2 \cdot m).$$

5.2 PROGRAMAÇÃO DINÂMICA

A estratégia da programação dinâmica costuma ser aplicada a certos **problemas de otimização**, resultando, em geral algoritmos mais eficientes que os triviais, pois a cada passo são **eliminadas subsoluções** que certamente não farão parte da **solução ótima do problema**.

Esse método é especialmente útil quando não é fácil chegar a uma **seqüência ótima de decisões** sem testar todas as seqüências possíveis para então escolher a melhor.

Reduz drasticamente o número total de **seqüências viáveis** através de um mecanismo que evita aquelas que sabidamente não podem resultar ótimas.

Muitas vezes, quando o algoritmo direto tem **complexidade exponencial**, o algoritmo desenvolvido por programação dinâmica é polinomial, reduzindo drasticamente o **tempo de execução**.

Outras vezes, a complexidade continua exponencial, mas de ordem mais baixa.

De qualquer forma, a programação dinâmica é uma estratégia para projeto de **algoritmos eficientes** bastante usada (cf. exercícios adicionais propostos no fim do capítulo).

A programação dinâmica pode ser aplicada a problemas de **otimização combinatória**, como por exemplo:

- multiplicação de várias matrizes;
- caminhos de custo mínimo em grafos orientados;
- projeto de sistemas confiáveis;
- problema do caixeiro viajante (cf. exercício adicional);
- problema da partição (cf. exercício adicional);
- problema da mochila (cf. exercício adicional).

Multiplicação de matrizes.

$$M := M_1 \times M_2 \times \dots \times M_n,$$

onde cada matriz M_i tem b_{i-1} linhas e b_i colunas, para $1 \leq i \leq n$.

Para multiplicar uma matriz A $p \times q$ por outra, B $q \times r$,

$$[A \times B]_{ij} := \sum_{k=1}^q A_{ik} \cdot B_{kj} \quad (\text{para } i = 1, \dots, p \text{ e } j = 1, \dots, r)$$

requer $p \times q \times r$ multiplicações de elementos.

Considere o seguinte exemplo:

$$M := \begin{matrix} M_1 & \times & M_2 & \times & M_3 & \times & M_4 & \times & M_5 \\ 100 \times 3 & & 3 \times 10 & & 10 \times 50 & & 50 \times 30 & & 30 \times 5. \end{matrix}$$

Calcular o produto, $\{[(M_1 \times M_2) \times M_3] \times M_4\} \times M_5$, requer:

$$(100 \times 10 \times 3) + (100 \times 10 \times 50) + (100 \times 50 \times 30) + (100 \times 30 \times 5) = 218.000 \text{ operações.}$$

E calcular o produto $M_1 \times \{M_2 \times [(M_3 \times M_4) \times M_5]\}$, requer:

$$(10 \times 50 \times 30) + (10 \times 30 \times 5) + (3 \times 10 \times 5) + (100 \times 3 \times 5) = 18.150 \text{ operações}$$

A **ordem** em que são realizadas as multiplicações pode influir substancialmente no número **total de operações** requeridas.

O problema consiste, então, em determinar uma **seqüência ótima** de multiplicações, i. e. com custo mínimo.

Um algoritmo que enumere todas as seqüências possíveis de multiplicações, calcule os respectivos números totais de operações requeridas, para, só então, escolher uma seqüência ótima terá complexidade exponencial em n (número de matrizes).

Utilizando a **programação dinâmica**, poderemos diminuir a ordem de complexidade para uma ordem polinomial.

Estamos interessados no **custo mínimo** do produto das n matrizes, i. e., no número mínimo de multiplicações necessárias para calcular:

$$M := M_1 \times M_2 \times \dots \times M_n$$

Para $1 \leq i < j \leq n$, consideremos o subproblema de calcular o produto:

$$M_{ij} = \begin{matrix} M_i & & M_{i+1} & & \dots & & M_j \\ b_{i-1} \times b_i & & b_i \times b_{i+1} & & & & b_{j-1} \times b_j \end{matrix};$$

com custo (número de multiplicações) mínimo, dado por m_{ij} .

O **problema original**, correspondente à matriz M_{1n} com custo mínimo m_{1n} .

Para $i \leq k < j$, vamos considerar os produtos:

$$M_{ik} := M_i \times M_{i+1} \times \dots \times M_k \text{ e}$$

$$M_{(k+1)j} := M_{k+1} \times M_{k+2} \times \dots \times M_j.$$

Notemos que essas matrizes M_{ik} e $M_{(k+1)j}$ correspondem a **subproblemas** do problema correspondente à matriz M_{ij} e, portanto, subproblemas do problema original correspondente à matriz M .

Essas duas matrizes M_{ik} e $M_{(k+1)j}$, respectivamente, têm:

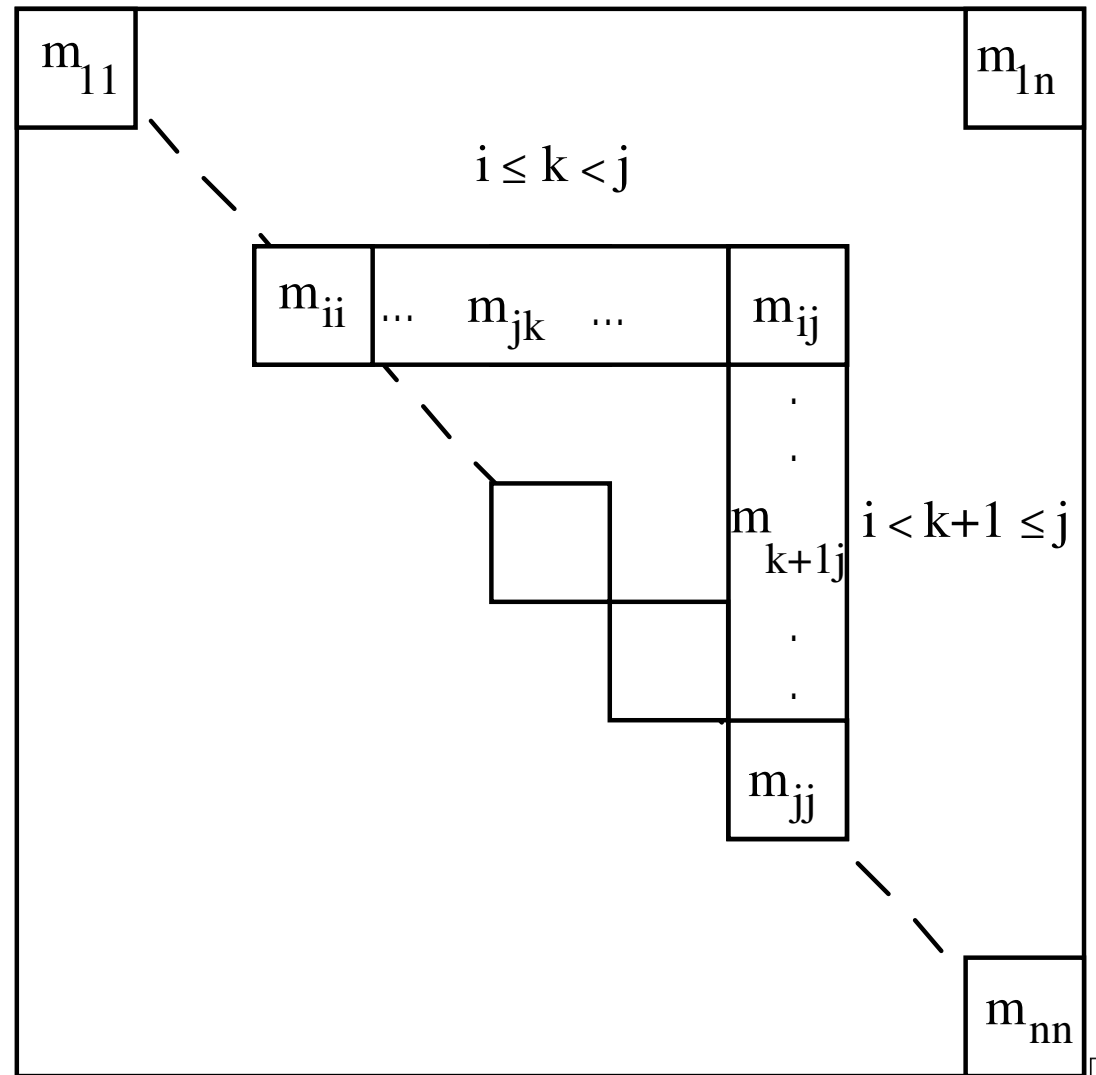
- custo mínimo m_{ik} e dimensões $b_{i-1} \times b_k$ (para M_{ik}) e
- custo mínimo $m_{(k+1)j}$ e dimensões $b_k \times b_j$ (de $M_{(k+1)j}$).

O produto M_{ij} pode ser obtido, fazendo $M_{ik} \times M_{(k+1)j}$, e o custo será dado por
 $(m_{ik} + m_{(k+1)j}) + (b_{i-1} \times b_k \times b_j)$.

Para o custo ser mínimo, tem-se que escolher um k apropriado, ou seja:

$$m_{ij} = \min_{i \leq k < j} \{ (m_{ik} + m_{(k+1)j}) + (b_{i-1} \times b_k \times b_j) \}.$$

Como $i \leq k < j$, tem-se $k-i < j-i$ e $j-(k+1) < j-1$, portanto os **custos mínimos** m_{ik} e $m_{(k+1)j}$ já terão sido obtidos antes. Assim, como mostra a figura 5.3.3, não há necessidade de recursão.



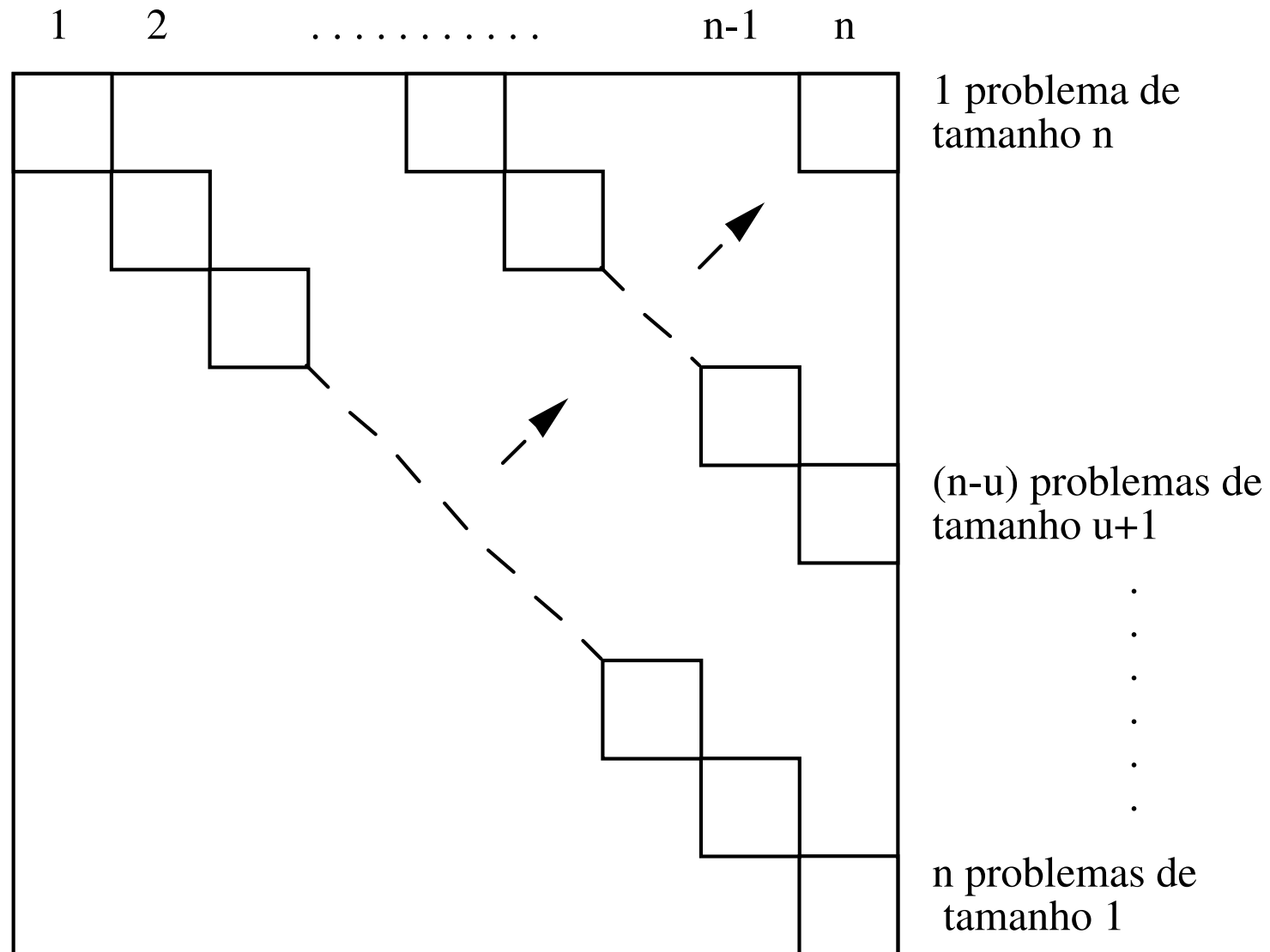
Processo de cálculo de custo mínimo de produto de matrizes

Um algoritmo para resolver esse problema de custo mínimo do produto de n matrizes por programação dinâmica utiliza como área auxiliar para armazenar os **custos intermediários** uma matriz quadrada.

O algoritmo funciona da seguinte maneira:

1. Num primeiro passo, a diagonal dessa matriz é inicializada: $m_{ii} := 0$, para $1 \leq i \leq n$.
2. Passo a passo, são obtidos valores correspondentes às diagonais superiores da matriz.
3. Finalmente, o processo termina com o valor m_{1n} no canto superior direito.

Desse modo, podemos formular um algoritmo para determinar o custo mínimo para calcular o produto de várias matrizes.



Hierarquia de instâncias na programação dinâmica

Função: Multi_Mat($b:D \rightarrow \mathbb{IN}$) {custo mínimo de produto de matrizes}
 { *Entrada:* vetor $b:D$ (para $i = 1, \dots, n$, das n matrizes $M_i : b[i-1] \times b[i]$)
Saída: $m[1, n] : \mathbb{IN}$ (n^o mín. de mult. para o produto $M_1 \times M_2 \times \dots \times M_n$) }
sorte $D :=$ vetor $[0..n]$ de $\mathbb{IN}$ {entrada} ; $M :=$ matriz $[1..n, 1..n]$ de $\mathbb{IN}$ {auxiliar} ;
var {auxiliares} $m:M$ {matriz auxiliar: $m[i, j]$ ($1 \leq i \leq j \leq n$) (custo mínimo para o produto $M_i \times \dots \times M_j$)} ; $j : \mathbb{IN}$ {índice de matriz} ;

1. para i de 1 até n faça $m[i, i] \leftarrow 0$; {inicializa diagonal principal}
2. para u de 1 até $n-1$ faça {deslocamento da diagonal: 7}
3. para i de 1 até $n-u$ faça {posição na diagonal: 6}
4. $j \leftarrow i+u$; { $u=j-i$ } ;
5. $m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j]) \}$;
6. fim-para {3: i de 1 até $n-u$ }
7. fim-para {2: u de 1 até $n-1$ }
8. retorne-saída($m[1, n]$) ; {dá saída extraída}
9. fim-Função {fim do algoritmo Multi_Mat: custo mínimo de produto}

A **seqüência ótima das multiplicações** pode ser obtida, guardando-se os valores de k , obtidos na linha 5 do algoritmo.

A **complexidade pessimista** do algoritmo Multi_Mat tem **ordem cúbica**, portanto a programação dinâmica tornou possível resolver com uma complexidade de ordem polinomial um problema para o qual o algoritmo imediato teria complexidade de ordem exponencial.

Lembrando que uma iteração definida tem comportamento equivalente a uma iteração definida inicializada, podemos simular a iteração definida

para u de 1 até $n - 1$ (das linhas de 2 a 7)

de Multi_Mat por:

$u \leftarrow 0$; enqto $u < n$ faça $u \leftarrow u + 1$; corpo {linhas de 4 a 5} fim-enqto.

Assim, podemos reescrever o texto de Multi_Mat como se segue:

1. para i de 1 até n faça $m[i, i] \leftarrow 0$; {inicializa diagonal principal}
- 1.a $u \leftarrow 0$; {inicializa controle u}
2. enqto $u < n$ faça {deslocamento da diagonal: 7}
- 2.a $u \leftarrow u + 1$; {incrementa controle u}
3. para i de 1 até $n - u$ faça {posição na diagonal: 6}
4. $j \leftarrow i + u$; { $u = j - i$ }
5. $m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k + 1, j]) + (b[i - 1] \times b[k] \times b[j]) \}$;
6. fim-para {3: i de 1 até $n - u$ }
7. fim-enqto {2: $u < n$ }
8. retorne-saída($m[1, n]$) ; {dá saída extraída}

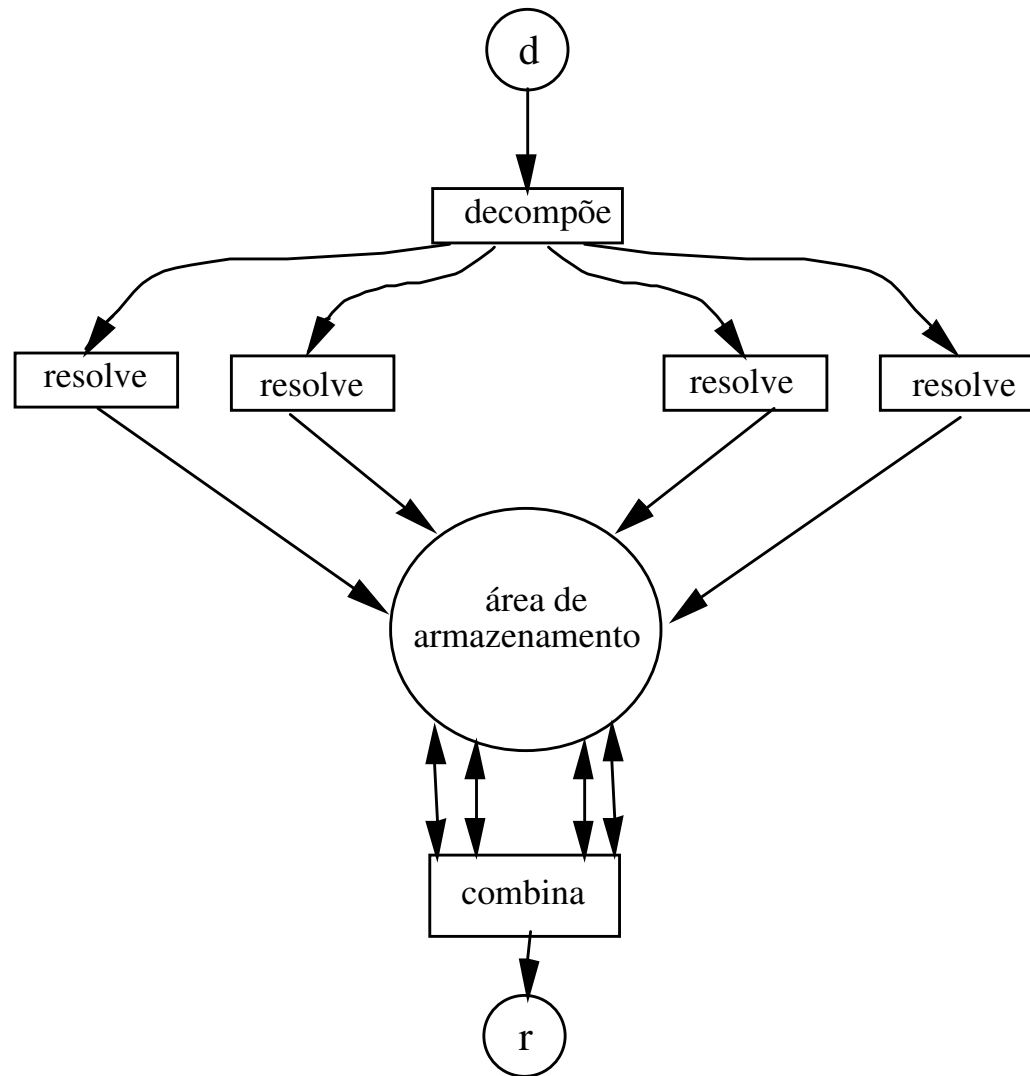
Idéias básicas da programação dinâmica

A **idéia básica** da programação dinâmica é **construir por etapas** uma resposta ótima combinando respostas já obtidas para **partes menores**.

- Inicialmente, entrada é decomposta partes mínimas, para as quais são obtidas respostas.
- Em cada passo, sub-resultados são combinados dando respostas para partes maiores, até que se obtenha uma resposta para o problema original.
- A decomposição é feita uma única vez e, além disso, os casos menores são tratados antes dos maiores.

Assim, esse método é chamado ascendente, ao contrário dos métodos recursivos, que são chamados descendentes.

O diagrama mostra a estrutura geral dessa abordagem.



Estrutura geral da programação dinâmica

A programação dinâmica **reduz** o número de possíveis combinações eliminando as que sabidamente não poderão fazer parte de uma solução ótima do problema.

É aplicável aos problemas de otimização com a seguinte **propriedade de otimalidade**.

"Uma solução não ótima de um subproblema não pode ser subsolução de solução ótima do problema".

Isso significa que só **soluções ótimas** dos subproblemas podem compor uma solução ótima do problema.

Na procura de uma solução ótima, as soluções não ótimas de subproblemas podem ser descartadas, e é esse o processo que torna os algoritmos desenvolvidos por programação dinâmica mais eficientes que os algoritmos diretos.

Além disso, os subproblemas são resolvidos uma única vez, e suas **soluções** são **guardadas** para serem usadas tantas vezes quantas for necessário.

A idéia básica da programação dinâmica sugere uma estrutura geral para algoritmos projetados por essa estratégia:

Inicialização ; Iteração ; Finalização.

Na **inicialização**, a entrada é decomposta em partes mínimas para as quais são obtidas respostas diretas.

A **iteração** vai aumentando tamanho das partes e obtendo respostas correspondentes a partir das já geradas, até que as partes atingem o tamanho da entrada original

Quando então sua **solução** é recuperada, e o processo finalizado.

Assim, chega-se ao seguinte esquema geral de algoritmo de programação dinâmica.

Algoritmo: Programação Dinâmica

Função Alg_PD($d:D$) $\rightarrow R$ {Algoritmo de programação dinâmica (abstrato)}

{ *Entrada-saída*: saída $r:R$ é resposta ótima para entrada $d:D$ }

 Incz_PD ; Iter_PD ; Fnl_PD {estrutura geral}
 onde {componentes}

 Incz_PD := {inicialização}
 entrada é decomposta em partes mínimas;
 partes mínimas dão diretamente partes da saída

 Iter_PD: itera corpo Crp_PD {iteração}
 escolhe elemento e da parte da entrada;
 combina parte da entrada com elemento e ;
 atualiza parte da saída com elemento e

 Fnl_PD: {finalização}
 extraí a saída final para a entrada original

No algoritmo Multi_Mat (para custo mínimo de produto de matrizes) apresentado no exemplo, podemos estabelecer as seguintes identificações:

Multi_Mat	Alg_PD
<u>sorte</u> D := vetor b:D de naturais	<u>sorte</u> D {entrada}
1. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u> m[i, i] ← 0;	IncZ_PD {inicialização}
2. <u>para</u> u <u>de</u> 1 <u>até</u> n-1 <u>faça</u>	Iter_PD {iteração}
3. <u>para</u> i <u>de</u> 1 <u>até</u> n-u <u>faça</u>	Crp_PD {corpo da iteração}
4. j ← i+u;	
5. m[i, j] ← melhor valor;	{atualiza parte de saída}
6. <u>fim-para</u>	
7. <u>fim-para</u>	{fim da iteração}
8. <u>retorne-saída</u> (m[1, n]);	FnI_G {finalização}

Tabela : Identificações entre Multi_Mat e Alg_PD

Projeto de algoritmos por programação dinâmica

Vamos examinar um problema de **caminhos de custo mínimo em grafo orientado**. O problema consiste em determinar um caminho de custo mínimo para cada par de vértices do grafo.

Para aplicar a estratégia de programação dinâmica podemos usar um conjunto I de **vértices intermediários**, que é incrementado em cada passo.

A idéia é determinar, passo a passo, os caminhos de custo mínimo com a restrição de usar como vértices intermediários apenas os do conjunto I .

Claramente, quando o conjunto I tiver todos os vértices do grafo, não há mais restrição e teremos resolvido o problema.

Para essa idéia funcionar, devemos saber inicializar e atualizar.

Inicialmente, vejamos um exemplo concreto para ilustrar as idéias envolvidas.

Caminhos de custo mínimo em grafo orientado.

Consideremos um grafo orientado $G = \langle V, E \rangle$ com 3 vértices $V = \{v_1, v_2, v_3\}$ e a seguinte matriz de custos:

cst	v_1	v_2	v_3
v_1	0	3	6
v_2	∞	0	2
v_3	7	∞	0

Inicialmente o conjunto I de vértices intermediários é vazio e $\text{dst}_0 := \text{custo}$.

No passo 1, selecciona-se um vértice como novo vértice intermediário, digamos o vértice v_1 . Com isso obtemos um caminho de v_3 a v_2 passando por v_1 com comprimento $7 + 4 = 11 < \infty$.

dst_1	v_1	v_2	v_3
v_1	0	3	6
v_2	∞	0	2
v_3	7	11	0

No passo 2, escolhendo o vértice v_2 como novo vértice intermediário, obtemos um novo caminho de v_1 a v_3 passando por v_2 com comprimento $3+2=5 < 6$. E:

dst_2	v_1	v_2	v_3
v_1	0	3	5
v_2	∞	0	2
v_3	7	11	0

Finalmente, tomando o vértice v_3 como terceiro vértice intermediário, obtemos um caminho de v_2 a v_1 passando por v_3 com comprimento $2+7=9 < \infty$.

Assim, a **matriz de distâncias** passa a ser:

dst_3	v_1	v_2	v_3
v_1	0	3	5
v_2	9	0	2
v_3	7	11	0

Essa matriz fornece o custo mínimo de caminhos entre cada par de vértices.

Distância entre vértices em grafo orientado.

Consideremos um grafo orientado G , com conjuntos

- conjunto $V = \{v_1, \dots, v_n\}$ de vértices e
- matriz de custo (dando arestas valoradas).

Desejamos determinar a distância (custo do caminho mínimo) entre cada par de vértices.

Usaremos um conjunto $I \subseteq V$ (de possíveis vértices intermediários), que será incrementado em cada passo.

Inicializamos o conjunto I de vértices intermediários como vazio ($I := \emptyset$) e a matriz de distâncias com a matriz de custos: $\text{dist}_I := \text{custo}$.

Em cada passo, seleciona-se um novo vértice $w \in (V - I)$ como novo vértice intermediário ($I := I \cup \{w\}$) e atualiza-se a matriz de distâncias (comparando cada valor atual com o de um caminho passando pelo novo vértice intermediário w).

Esse processo prossegue enquanto houver possíveis novos vértices intermediários ($I \neq V$).

Ao final $I = V$ e teremos o custo do caminho mínimo entre cada par de vértices.

O **algoritmo Dist_vrt** foi obtido pela de programação dinâmica observando:

- As linhas de 1 a 7 fazem a inicialização: a linha 1 inicializa as partes mínimas da entrada (sem vértices intermediários) e a linhas de 2 a 7 inicializam a saída correspondente.
- A iteração (aumentando o conjunto de possíveis vértices intermediários) está nas linhas de 8 a 16: a linha 9 seleciona um novo elemento, que é usado para atualizar a parte I da entrada (na linha 10) e a saída correspondente (nas linhas de 11 a 15).
- A finalização está na linha 17, quando se extrai a solução.

Algoritmo: Custo mínimo de caminhos entre vértices de grafo orientado

Função Dist_vrt($d:D$) $\rightarrow R$ {distância entre vértices de grafo}

{ *Entrada:* grafo orientado G com conjunto V de vértices e matriz $cst:M$ (custos)

Saída: matriz $dst:R$ (custos dos melhores caminhos entre vértices de G) }

sorte $D :=$ Grafos valorados com custos ; $R := M$; $Vrt :=$ Vértices ;

$P :=$ Conj[Vrt] {conjuntos de vértices} ; $M :=$ matriz [$u \in V, v \in V$] de $\mathbb{R}^*$;

var {auxiliares} $I:P$ {parte da entrada} ; $w:Vrt$ {vértice} ;

1. $I \leftarrow \emptyset$;	{inicializa conjunto I sem intermediários}
2. <u>para cada</u> $u \in V$ <u>faça</u>	{inicializa distância sob I: 7}
3. $\text{dst}[u, u] \leftarrow 0$;	{inicializa diagonal com zero}
4. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	{inicializa não diagonal: 7}
5. $\text{dst}[u, v] \leftarrow \text{cst}[u, v]$;	{distância inicial como custo}
6. <u>fim-para</u>	{4: cada $v \in (V - \{u\})$ }
7. <u>fim-para</u>	{2: cada $u \in V$ }
8. <u>enqto</u> $I \neq V$ <u>faça</u>	{itera: 16}
9. escolha $w \in (V - I)$;	{seleciona novo vértice}
10. $I \leftarrow I \cup \{w\}$;	{atualiza I com novo intermediário}
11. <u>para cada</u> $u \in V$ <u>faça</u>	{atualiza distância sob I: 15}
12. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	{atualiza não diagonal: 14}
13. $\text{dst}[u, v] \leftarrow \min(\text{dst}[u, v], \text{dst}[u, w] + \text{dst}[w, v])$;	
14. <u>fim-para</u>	{12: cada $v \in (V - \{u\})$ }
15. <u>fim-para</u>	{11: cada $u \in V$ }
16. <u>fim-enqto</u>	{8: $I \neq V$ (todos os vértices)}
17. <u>retorne-saída</u> (dst) ;	{dá como saída resposta pronta}
18. <u>fim-Função</u>	{fim do algoritmo Dist_vrt: distância entre vértices}

Pode-se determinar um caminho de custo mínimo, guardando-se os vértices correspondentes ao mínimo na linha 13 do algoritmo.

O problema do projeto de sistemas confiáveis (PSC).

O objetivo é construir um sistema com **máxima confiabilidade**, restrito aos recursos orçamentários disponíveis e pode ser enunciado como segue.

O sistema desejado consiste da ligação em série de **n** dispositivos:

$$D_1, D_2, \dots, D_n.$$

Cada dispositivo D_i tem uma confiabilidade b_i , para $i = 1, \dots, n$.

Para aumentar a confiabilidade do sistema, serão usadas **cópias** reservas de cada dispositivo, as quais entram em ação imediatamente quando o dispositivo em uso deixar de funcionar.

Cada cópia de um dispositivo D_i tem um custo $\text{custo}(i)$, e o montante **M** de reserva orçamentária para construir o sistema é um limitante do problema.

O problema consiste em **maximizar a confiabilidade** do sistema sem ultrapassar o limite de custo pré-determinado M .

O sistema está constituído de n estágios, cada estágio se referente a um dispositivo. Consideremos o dispositivo D_i (no estágio i).

Sejam:

- n_i o número de cópias disponíveis do dispositivo;
- b_i a confiabilidade (probabilidade de funcionamento) do dispositivo.

Assim, temos

- $(1 - b_i)$ é a probabilidade de D_i não funcionar, e
- $(1 - b_i)^{n_i}$ é a probabilidade de n_i cópias do dispositivo i não funcionarem.

Portanto, a **confiabilidade $cfb(i)$ do estágio i** pode ser dada por $cfb(i) = 1 - (1 - b_i)^{n_i}$.

O sistema terá confiabilidade $\prod_{i=1}^n cfb(i)$ e custo $\sum_{i=1}^n [custo(i) \cdot n_i]$.

O problema é maximizar a confiabilidade do sistema, sujeito ao limite de custo M , isto é

$$\text{Máx } \prod_{i=1}^n cfb(i) \quad \text{sujeito a } \sum_{i=1}^n [custo(i) \cdot n_i] \leq M.$$

Trata-se de um **problema de otimização com restrições**. A programação dinâmica é um método apropriado para resolver de forma exata e com bom desempenho tais problemas.

Um algoritmo trivial analisaria todos os casos possíveis. A programação dinâmica vai eliminar aqueles casos que sabidamente não estarão na solução ótima.

O montante M é a principal limitação. É preciso comprar pelo menos um exemplar de cada dispositivo, e o restante pode ser gasto em exemplares extras de dispositivos.

Assim, pode-se calcular o número máximo m_i de exemplares de cada dispositivo da seguinte forma:

$$m_i = \left\lfloor \frac{\left(M - \sum_{j=1}^n \text{custo}(j) \right) + \text{custo}(i)}{\text{custo}(i)} \right\rfloor$$

O problema PSC com n estágios é decomposto em problemas de 1 estágio, isto é, em que só existe o dispositivo D_1 , que pode ter no máximo m_1 exemplares, o que significa m_1 problemas: cada um, com um custo e uma confiabilidade.

A cada passo, o tamanho do problema **crescerá de 1** (mais um dispositivo é considerado).

A solução de problemas, num certo passo, usa a solução de passos anteriores. Assim, em $n-1$ passos, é obtido e solucionado o problema original.

Para cada $i = 1, \dots, n$, consideremos:

S_i , consistindo do conjunto dos possíveis pares confiabilidade e custo do projeto com i estágios, usando j exemplares do dispositivo i ($j = 1, 2, \dots, m_i$), com custo que não ultrapasse o montante M .

Uma configuração com maior confiabilidade e menor custo é preferível desde que viável.

Assim, temos a seguinte regra de dominância para preterir configurações.

Sendo (x', y') e (x'', y'') pares viáveis ($y' \leq M$ e $y'' \leq M$): (x', y') domina (x'', y'') sempre que $x' > x''$ e $y' \leq y''$.

Aplicando essa **regra de dominância**, podemos otimizar S_i , eliminando pares dominados, para obter C_i . Ou seja:

$$C_i := \{(x, y) \in S_i / \nexists (x', y') \in S_v \text{ tal que } (x', y') \text{ domina } (x, y)\}.$$

Assim, esse conjunto otimizado C_i é obtido em dois passos:

- no primeiro passo, constrói-se S_i , com todos os pares (x, y) de confiabilidade e correspondente custo, considerados i estágios e o limitante M ;
- no segundo passo, S_i é otimizado, por eliminação dos pares dominados.

Os conjuntos S_i de pares, para $i = 1, \dots, n$, podem ser calculados indutivamente da seguinte forma.

Para $i = 1$, temos S_1 diretamente, pois

$$S_1 = \{(1 - (1 - b_i))^j, j \cdot \text{custo}(1) / j = 1, 2, \dots, m_1\}.$$

Para $i > 1$, podemos obter S_i a partir de C_{i-1} , pois

$$S_i := \{(x \cdot \text{cfb}(i))^j, y + j \cdot \text{custo}(i) / (x, y) \in C_{i-1} \text{ tal que } y + j \cdot \text{custo}(i) \leq M, j = 1, \dots, m_i\}.$$

Consideremos o **PSC** com montante $M=190$ e $n=4$ dispositivos, tendo confiabilidades e custos dados pela tabela abaixo

i	1	2	3	4
b_i	0.9	0.8	0.7	0.9
$\text{custo}(i)$	30	35	40	22

Nesse caso, o número máximo de exemplares de cada dispositivo resulta $m_1=3$; $m_2=2$; $m_3=2$ e $m_4=3$.

Obtemos as seguintes seqüências de conjuntos de pares confiabilidade e custo.

Para $i=1$, temos:

$S_1 = \{(0.9, 30), (0.99, 60), (0.999, 90)\}$., como não há dominâncias, obtemos

$C_1 = S_1 = \{(0.9, 30), (0.99, 60), (0.999, 90)\}$.

Para $i=2$,

$S = \{(0.72, 65), (0.792, 95), (0.7992, 125), (0.864, 100), (0.9504, 130), (0.9599, 160)\}$

Notamos que há dominância de pares: de $(0.7992, 125)$ por $(0.864, 100)$, assim:

$C_2 = \{(0.72, 65), (0.792, 95), (0.864, 100), (0.9504, 130), (0.9599, 160)\}$.

Para $i=3$,

$$S_3 = \{(0.504, 105), (0.6552, 145), (0.5544, 135), (0.66528, 175), (0.6048, 140), (0.78624, 180), (0.66528, 170)\}.$$

Como não há dominâncias, temos:

$$C_3 = S_3 = \{(0.504, 105), (0.6552, 145), (0.5544, 135), (0.66528, 175), (0.6048, 140), (0.78624, 180), (0.66528, 170)\}.$$

Para $i=4$,

$$S_4 = \{(0.4536, 127), (0.49896, 149), (0.5039496, 171), (0.49896, 157), (0.548856, 179), (0.54432, 162), (0.598752, 184), (0.58968, 167), (0.698648, 189)\}.$$

Notamos as seguintes dominância de pares:

- de $(0.5039496, 171)$ e $(0.548856, 179)$ por $(0.58968, 167)$;
- de $(0.49896, 157)$ por $(0.49896, 149)$.

Eliminando esses pares dominados, obtemos:

$$C_4 = \{(0.4536, 127), (0.49896, 149), (0.54432, 162), (0.598752, 184), (0.58968, 167), (0.698648, 189)\}.$$

Portanto, o **sistema ótimo** é o que tem **confiabilidade 0.698648** e **custo 189**.

Agora, podemos obter um algoritmo de programação dinâmica para o problema PSC.

Algoritmo: Projeto de sistemas com confiabilidade máxima

Função PSC($b: \text{Cnf}$, $\text{custo}: \text{Cst}$, $M: \mathbb{R}$) $\rightarrow [0, 1]$ {confiabilidade máxima}
 { *Entrada*: vetores b (com a confiabilidade) e custo (com o custo) de cada dispositivo D_i (para $i = 1, \dots, n$) e montante (cota superior para custo)
Saída: r (confiabilidade máxima de sistema com n dispositivos) }

sorteCnf := vetor [1..n] de [0, 1]; Cst := vetor [1..n] de $\mathbb{R}$
 Par := [0, 1] $\times$ $\mathbb{R}$ {auxiliar: confiabilidade e custo};
var {auxiliares} cst_min: $\mathbb{R}$ {custo mínimo}; m : vetor [1..n] de $\mathbb{N}$
 {número máximo de cópias}; S, C: Conj[Par] {conjuntos S_i e C_i }; x' : [0, 1] {confiabilidade}; y' : $\mathbb{R}$ {custo};

1. $\text{cst_min} \leftarrow \sum_{j=1}^n \text{custo}[j]$; {custo mínimo}
2. para i de 1 até n faça $m[i] \leftarrow \left\lfloor \frac{M - \text{cst_min}}{\text{custo}[i]} \right\rfloor + 1$; {nº máx. de cópias do dispos. i }

3. $S \leftarrow \{ ((1 - (1 - b[1]))^j), j \cdot \text{custo}[1] \} / j = 1, 2, \dots, m[1] \};$	{conjunto S_1 }
4. $C \leftarrow S;$	{conjunto C_1 }
5. $i \leftarrow 1;$	{inicializa controle i }
6. <u>enqto</u> $i < n$ <u>faça</u>	{estágios: de 6 a 17}
7. $i \leftarrow i + 1;$	{incrementa controle i }
8. $S \leftarrow \emptyset;$	{inicializa S_{i-1} }
9. <u>para cada</u> $(x, y) \in C$ <u>faça</u>	{varre C_{i-1} : de 9 a 15}
10. <u>para</u> j <u>de</u> 1 <u>até</u> $m[i]$ <u>faça</u>	{número de cópias: de 10 a 14}
11. $x' \leftarrow x \cdot (1 - (1 - b[1]))^j;$	{confiabilidade}
12. $y' \leftarrow y + j \cdot \text{custo}[i];$	{custo}
13. <u>se</u> $y' \leq M$ <u>então</u> $\text{Insere}((x', y'), S);$	{atualiza S_j }
14. <u>fim-para</u>	{10: j de 1 até $m[i]$ }
15. <u>fim-para</u>	{9: cada $(x, y) \in C$ }
16. $C \leftarrow S$ - pares dominados;	{obtem C_i de S_i }
17. <u>fim-enqto</u>	{6: $i \leq n$ }
18. $r \leftarrow \text{Máx}\{x / \exists y: (x, y) \in C\};$	{confiabilidade máxima em C^n }
19. <u>retorne-saída</u> (r);	{dá saída}
20. <u>fim-Função</u>	{fim do algoritmo PSC: confiabilidade máxima de sistema}

Complexidade de algoritmos de programação dinâmica

Vamos ver como se pode efetuar a **análise de complexidade pessimista** de alguns algoritmos típicos desenvolvidos por programação dinâmica.

Complexidade de custo mínimo de produto de matrizes.

Considere o algoritmo **Multi_Mat** (custo mínimo de produto de matrizes).

- A iteração de inicialização na linha 1 é executada n vezes.
- A iteração nas linhas de 2 a 7 é executada $n-1$ vezes.
- Já o número de vezes que é executada a iteração nas linhas de 3 a 6 (para i de 1 até $n-u$) dependendo de u (o índice da iteração externa de 2 a 7).
- O mínimo na linha 5 é tomado sobre $j-i$ (i. e. u) elementos.

Assim, a complexidade da iteração das linhas de 2 a 7 é dada por $\sum_{u=1}^{n-1} [u \cdot (n-u)]$.

Mas, $\sum_{u=1}^{n-1} [u \cdot (n-u)]$ tem ordem $\Theta(n^3)$.

Portanto, a complexidade do algoritmo Multi_Mat tem ordem

$$c_p[\text{Multi_Mat}](n) = O(n + n^3) = \mathbf{O(n^3)}.$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse algoritmo `Multi_Mat` recebe como **entrada**

um vetor b : vetor $[0..n]$ de $\mathbb{N}$ (com as dimensões das n matrizes a multiplicar)

e fornece como **saída**

um natural $m[1, n]$ (o número mínimo de multiplicações para o produto),

usando como área auxiliar de armazenamento

uma matriz m : M , sendo $M :=$ matriz $[1..n, 1..n]$ de $\mathbb{N}$.

Aqui, é natural tomar como tamanho da entrada o número n de matrizes a multiplicar.

Nesse caso, parece natural tomar como fundamental as operações aritméticas sobre naturais, incluindo inicializações com 0.

Como o algoritmo `Multi_Mat` é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de **Multi_Mat** sobre uma entrada b tem contribuições dadas por suas componentes: **Inicialização, Iteração e Finalização**.

Para determinar essas contribuições, devemos obter os valores das entradas de cada trecho.

A inicialização (linha 1) fornece valores iniciais à diagonal principal da variável $m:M$ {custos das partes mínimas da entrada}

$$[m_0[1,1], \dots, m_0[n,n]] := [0, \dots, 0] \quad \{\text{tamanho } \dim(m_0) = n\}$$

A inicialização (linha 1) executa n vezes o comando $m[i,i] \leftarrow 0$.

Esse comando $m[i,i] \leftarrow 0$ tem desempenho $\text{desemp}[m[i,i] \leftarrow 0] = 1$.

Assim, **inicialização Incz_Multi_Mat** tem

$$\text{desemp}[\text{Multi_Mat}](b) = n \cdot 1 = n;$$

o que resulta em complexidade pessimista

$$c_p^{\leq}[\text{Incz_Multi_Mat}](n) = n.$$

A seguir, a iteração nas linhas de 2 a 7 (para u de 1 até $n-1$ faça) repete $n-1$ vezes o corpo (tratando das diagonais superiores e aumentando o número de matrizes que serão multiplicadas).

Desse modo, são gerados novos valores para a variável $m:M$ {custos das partes das diagonais superiores}.

Os valores sucessivos da variável $m:M$ têm tamanhos como se segue:

$$\begin{array}{c|cccccc} u & 0 & 1 & \dots & u & \dots & n-1 \\ \dim(m_u) & n & n-1 & \dots & n-u & \dots & 1 \end{array}$$

Assim, ao início da $u^{\text{ésima}}$ iteração, a variável $m:M$ tem tamanho $\dim(m_u) = n - u$.

Agora, o corpo **Crp_Multi_Mat** é a iteração definida nas linhas de 3 a 6 varrendo cada posição na diagonal. Aqui, só contribui para o desempenho a linha 5

$$m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j]) \}.$$

Essa linha 5 envolve uma **minimização** sobre k variando de i até $j-1 = i+u-1$ (cf. linha 4): uma minimização sobre um conjunto C_{ij} com $(j-1) - (i-1) = u$ elementos.

O k^{o} elemento desse conjunto C_{ij} é o natural dado pela expressão $(m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j])$; assim sua determinação envolve 2 somas e 2 produtos de naturais.

Portanto, a determinação de todos os u elementos desse conjunto C_u envolve $4 \cdot u$ operações aritméticas.

Assim, o desempenho da minimização da linha 5 na $u^{\text{ésima}}$ iteração é dado por

$$\text{desemp}[\text{linha 5}](m_u) = 4 \cdot u.$$

Desse modo, na $u^{\text{ésima}}$ iteração, as linhas 4 e 5 têm os seguintes desempenhos:

Comando	desempenho
4. $j \leftarrow i + u;$	1
5. $m[i, j] \leftarrow \text{melhor valor};$	$4 \cdot u$

Assim, temos $\text{desemp}[\text{linhas 4-5}](m_u) = 1 + 4 \cdot u$.

A iteração definida nas linhas de 3 a 6 (para de 1 até $n-u$ faça) repete $n-u$ as linhas 4 e 5.

Portanto, o desempenho do corpo **Crp_Multi_Mat** na $u^{\text{ésima}}$ iteração é dado pela somatória das $n-u$ contribuições:

$$\text{desemp}[\text{Crp_Multi_Mat}](m_u) = (1 + 4 \cdot u) \cdot (n - u).$$

A iteração `Iter_Multi_Mat` (linhas de 2 a 7) repete $n-1$ vezes o corpo `Crp_Multi_Mat`.

Portanto, o desempenho da iteração `Iter_Multi_Mat` é dado pela somatória das $n-1$ contribuições:

$$\begin{array}{c} u=1 \\ (1+4.1).(n-1) \end{array} + \dots + \begin{array}{c} U \\ (1+4.u).(n-u) \end{array} + \dots + \begin{array}{c} u=n-1 \\ (1+4.(n-1)).1 \end{array}$$

Assim, o desempenho da iteração `Iter_Multi_Mat` é dado por

$$desemp[Iter_Multi_Mat](m_0) = \sum_{u=1}^{n-1} [(1+4.u).(n-u)].$$

Desse modo, pode-se ver que a complexidade pessimista da iteração `Iter_Multi_Mat` tem ordem cúbica.

$$c_P^{\leq}[Iter_Multi_Mat](n) = \Theta(n^3).$$

Agora, a ordem da **complexidade pessimista** do algoritmo Multi_Mat é a soma das três contribuições devidas à inicialização, à iteração e à finalização.

$$\begin{array}{rcl}
 c_p^{\leq} [\text{Incz_Multi_Mat}](n) & = & n \\
 + & & \\
 c_p^{\leq} [\text{Iter_Multi_Mat}](n) & = & \Theta(n^3) \\
 + & & \\
 c_p^{\leq} [\text{Fnl_Multi_Mat}](n) & = & 0 \\
 \hline
 c_p^{\leq} [\text{Multi_Mat}](n) & = & \Theta(n+n^3)
 \end{array}$$

Assim, a complexidade pessimista de Multi_Mat tem a ordem

$$c_p^{\leq} [\text{Multi_Mat}](n) = \Theta(n^3).$$

Logo, o algoritmo **Multi_Mat tem complexidade pessimista** cúbica:

$$c_p^{\leq} [\text{Multi_Mat}](n) = \Theta(n^3).$$

Complexidade de distância entre vértices em grafo orientado.

Considere o algoritmo Dist_vrt para distância (custo mínimo de caminhos) entre pares de vértices em grafo orientado.

- A inicialização do conjunto I ($I \leftarrow \emptyset$ na linha 1) tem complexidade constante.
- As linhas de 2 a 7 inicializam cada um dos n^2 elementos da matriz dst.

Assim, a complexidade da inicialização (linhas de 1 a 7) tem ordem $O(n^2)$.

- A iteração das linhas de 8 a 16 é executada até $I=V$: n vezes.
- A parte dominante de seu corpo (linhas de 9 a 15) é devida às linhas de 11 a 15, atualizando cada um dos n^2 elementos da matriz dst, com complexidade de ordem $O(n^2)$.

Logo, a complexidade da iteração tem ordem $O(n \cdot n^2) = O(n^3)$.

Portanto, a **complexidade** do algoritmo Dist_vrt tem ordem

$$c_p^{\leq}[\text{Dist_vrt}](n) = O(n^2 + n^3) = \mathbf{O(n^3)}.$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse **algoritmo Dist_vrt** recebe como **entrada** $d:D$

um grafo orientado G com conjunto V de vértices e matriz cst de custos e fornece como **saída**

uma matriz dst (com os custos dos melhores caminhos entre vértices de G)

Consideremos um grafo orientado G , com conjunto $V = \{v_1, \dots, v_n\}$ de vértices.

É natural, para tamanho da entrada o número n de vértices de G .

E parece natural tomar como **operações fundamentais** as manipulações com conjuntos (de vértices) e matrizes.

Como o algoritmo $Dist_vrt$ é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de $Dist_vrt$ sobre uma entrada $d:D$ tem contribuições dadas por suas componentes:

Inicialização, Iteração e Finalização.

Para determinar essas contribuições, devemos obter os valores das entradas de cada trecho.

A inicialização (linhas de 1 a 7) fornece valores iniciais às variáveis $l:P$ {parte da entrada} e $dst:R$ {custos dos melhores caminhos entre vértices de G }

$$l_0 := \emptyset \quad \{\text{tamanho} \mid |l_0| = 0\}$$

$$dst_0[u,v] := \begin{cases} 0 & (\text{se } u=v) \\ cst[u,v] & (\text{se } u \neq v) \end{cases} \quad \{\text{tamanho } \dim(dst_0) = n^2\}$$

Assim, a saída da inicialização é $Inc_{z_Dist_vrt}(G) = (l_0, dst_0)$.

O **desempenho da inicialização** tem as seguintes contribuições:

comando	nº de vezes	esforço
1. $I \leftarrow \emptyset;$	1	1
2. <u>para cada</u> $u \in V$ <u>faça</u>	n	
3. $\text{dst}[u, u] \leftarrow 0;$	1	1
4. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	$(n - 1)$	
5. $\text{dst}[u, v] \leftarrow \text{cst}[u, v];$	1	1
6. <u>fim-para</u>		
7. <u>fim-para</u>		

Assim,

$$\text{desemp}[\text{Incz_Dist_vrt}](G) = (1 \cdot 1) + (n \cdot 1) + (n \cdot (n - 1) \cdot 1) = n^2 + 1.$$

Agora, examinemos a iteração enqto $I \neq V$ (linhas de 8 a 16).

A cada iteração do corpo (linhas de 9 a 15), um novo vértice $w \in (V - I)$ é selecionado e incluído em I (linhas 9 e 10), sendo atualizados os valores da matriz $\text{dst} : R$ (linhas de 11 a 15).

Desse modo, são gerados novos valores para as variáveis $I:P$ e $dst:R$ através de valores da variável $w:Vrt \{seleção\}$

$$\begin{array}{l|l}
 w:Vrt & w_1 \in (V - I_0) \quad \dots \quad w_k \in (V - I_{k-1}) \quad \dots \quad w_n \in (V - I_{n-1}) \\
 I:P & I_1 := I_0 \cup \{w_1\} \quad \quad \quad I_k := I_{k-1} \cup \{w_k\} \quad \quad \quad I_n := I_{n-1} \cup \{w_n\} \\
 dst:R & dst_1 \quad \quad \quad \dots \quad dst_k \quad \quad \quad \dots \quad dst_n
 \end{array}$$

Assim, os valores sucessivos das variáveis $I:P$ e $dst:R$ têm tamanhos como se segue:

$$\begin{array}{l|l}
 j & 0 \quad 1 \quad \dots \quad k \quad \dots \quad n \\
 |I_j| & 0 \quad 1 \quad \dots \quad k \quad \dots \quad n \\
 \dim(dst_j) & n^2 \quad n^2 \quad \dots \quad n^2 \quad \dots \quad n^2
 \end{array}$$

Logo, a iteração é executada para $|I|$ variando de 0 a $n-1$: n vezes.

Ao início da $k^{ésima}$ iteração, as variáveis $I:P$ e $dst:R$ têm tamanhos $|I_k| = k$ e $\dim(dst_k) = n^2$.

O desempenho do corpo **Crp_Dist_vrt** (linhas de 9 a 15) em cada iteração é dado pela soma das contribuições das linhas 9 e 10 e das linhas de 11 a 15.

Na k ésima iteração, as linhas 9 e 10 atuam sobre as variáveis

- $I:P$ e $dst:R$ com tamanhos $|I_k|=k$ e
- $\dim(dist_i)=n^2$ e gerando novo valor para $I:P$ com tamanho $|I_{k+1}|=k+1$.

Assim, as linhas 9 e 10 dão as seguintes contribuições para o desempenho:

comando	desempenho
9. escolha $w \in (V-I)$;	1
10. $I \leftarrow I \cup \{w\}$;	k

Assim, temos $desemp[linhas\ 9,10](I_k, dst_k) = 1 + k = k + 1$.

Na $k^{\text{ésima}}$ iteração, as linhas de 11 a 15 atuam sobre variáveis:

- $p:P$ e $\text{dist}:V$ com tamanhos $|I_{k+1}| = k+1$ e
- $\dim(\text{dst}_k) = n^2$.

Assim, as linhas de 11 a 15 dão as seguintes contribuições para o desempenho:

comando	nº de vezes	esforço
11. <u>para cada</u> $u \in V$ <u>faça</u>	n	
12. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	$(n - 1)$	
13. $\text{dst}[u, v] \leftarrow \min(\text{dst}[u, v], \text{dst}[u, w] + \text{dst}[w, v])$		1
14. <u>fim-para</u>		
15. <u>fim-para</u>		

Assim, $\text{desemp}[\text{linhas 11-15}](I_{k+1}, \text{dst}_k) = n \cdot (n - 1) \cdot 1 = n^2 - n$.

Desse modo, o desempenho do corpo Crp_Dist_vrt na $k^{\text{ésima}}$ iteração é dado por:

$$\begin{aligned} \text{desemp}[\text{Crp_Dist_vrt}](l_k, \text{dst}_k) &= \\ &= \text{desemp}[\text{linhas 9,10}](l_k, \text{dst}_k) + \text{desemp}[\text{linhas 11-15}](l_{k+1}, \text{dst}_k) \\ &= (k+1) + n \cdot (n-1) = n \cdot (n-1) + (k+1). \end{aligned}$$

Agora, como a iteração enqto $l \neq V$ (linhas de 8 a 16) é executada n vezes, para $|l|$ variando de 0 a $n-1$, seu desempenho é a somatória das n contribuições:

$$\begin{array}{ccccccc} j=0 & & & j=k & & & j=n-1 \\ n \cdot (n-1) + 1 & + & \dots & + & n \cdot (n-1) + (k+1) & + & \dots & + & n \cdot (n-1) + n \end{array}$$

Assim, para o desempenho da iteração **Iter_Dist_vrt** temos:

$$\begin{aligned} \text{desemp}[\text{Iter_Dist_vrt}](l_0, \text{dst}_0) &= \\ &= [n \cdot (n-1) + 1] + \dots + [n \cdot (n-1) + (k+1)] + \dots + [n \cdot (n-1) + n] = \\ &= n \cdot [n \cdot (n-1)] + \sum_{k=0}^{n-1} (k+1) = [n^2 \cdot (n-1)] + [n \cdot (n-1)] / 2. \end{aligned}$$

Agora, o desempenho do algoritmo Dist_vrt é a soma das duas contribuições:

$\text{desemp}[\text{Incz_Dist_vrt}](G)$ e $\text{desemp}[\text{Iter_Dist_vrt}](\text{Incz_Dist_vrt}(G))$:

$$\begin{aligned} \text{desemp}[\text{Dist_vrt}](I_k, \text{dst}_k) &= \\ &= \text{desemp}[\text{Incz_Dist_vrt}](G) + \text{desemp}[\text{Iter_Dist_vrt}](I_0, \text{dst}_0) \\ &= [n^2 + 1] + [n \cdot (n-1) \cdot (n+1/2)]. \end{aligned}$$

Assim, para a complexidade pessimista de Dist_vrt temos

$$c_p^{\leq}[\text{Dist_vrt}](n) = O([n^2 + 1] + [n^3 - n^2/2 - n/2]) = \Theta(n^3).$$

Logo, o **algoritmo Dist_vrt tem complexidade pessimista** cúbica:

$$c_p^{\leq}[\text{Dist_vrt}](n) = \Theta(n^3).$$

5.3 DIVISÃO E CONQUISTA

A estratégia de **divisão e conquista** para a resolução de problemas - através de **decomposições e recombinações** - fornece um método poderoso e versátil para o projeto de algoritmos.

Alguns exemplos, ilustrando a grande aplicabilidade de divisão e conquista, podem ser dados por algoritmos:

- de **busca** em tabelas, como buscas seqüencial e binária;
- de **classificação**, como classificação por seleção (selectionsort), por intercalação (mergesort) e por particionamento (quicksort);
- para **multiplicação** (de matrizes e de números binários, por exemplo);
- para **seleção** (para determinar máximo e mínimo, o k^o máximo, etc.).

A estratégia opera **decompondo** um problema em subproblemas independentes, resolvendo-os e **combinando** as soluções obtidas em uma solução para o problema original.

Isso dá um processo recursivo de decomposições e recombinações.

Exemplos Introdutórios

Soma e ocorrência em lista.

Consideremos alguns problemas simples sobre listas.

- a) Desejamos achar a somatória dos elementos de uma lista de números. Podemos aplicar a idéia de **dividir para conquistar**.

Se a lista L é curta (digamos, comprimento no máximo 1), já sabemos calcular sua somatória.

Caso contrário, podemos: partir a lista L em sublistas L_1 e L_2 e aplicar recursivamente o mesmo processo até que tenhamos só sublistas curtas, das quais já sabemos tratar.

E então bastará **somar as saídas** correspondentes a essas sublistas.

A solução esboçada acima pode ser expressa pelo algoritmo recursivo

$$\begin{aligned} \text{smt}(L) &:= \underline{\text{se}} \text{ curta}(L) \underline{\text{então}} \text{imd}(L) \\ &\quad \underline{\text{senão}} \text{soma}(\text{smt}(\text{sublist}_1(L), \text{smt}(\text{sublist}_2(L))). \end{aligned}$$

Esse algoritmo usa como **auxiliares**

- o teste unário curta (para ver se L tem comprimento no máximo 1);
- a função unária imd (dando 0 ou o único elemento da lista);
- as funções unárias sublist₁ e sublist₂ (partindo L em sublistas);
- a função binária soma (somando resultados: $\text{soma}(r_1, r_2) := r_1 + r_2$).

b) Analogamente, o problema de contar quantas vezes um objeto obj ocorre em uma lista L de objetos poderia ser resolvido por divisão e conquista da seguinte maneira.

Se a lista L é curta, basta saber se obj ocorre em L , dando saída 1, ou não, dando saída 0.

Caso contrário, partimos L em sublistas L_1 e L_2 e aplicamos recursivamente o mesmo processo até que tenhamos só sublistas curtas, das quais já sabemos tratar.

E então bastará somar as saídas correspondentes a essas sublistas.

Agora, vejamos um exemplo clássico: um algoritmo de classificação projetado através de divisão e conquista.

Classificação de listas por intercalação de sublistas.

Desejamos um algoritmo que receba como entrada uma lista e devolva essa lista classificada.

Vamos aplicar a idéia de divisão e conquista.

Se a lista de entrada é curta já está classificada.

Caso contrário a entrada será dividida em duas listas aproximadamente do mesmo tamanho a primeira metade $\text{Prim}(d)$ e a segunda metade $\text{Find}(d)$ (cf. exemplo 3.2.6 na seção 3.2 do capítulo 3).

Essas duas **sublistas** serão recursivamente classificadas e intercaladas para assim se obter uma versão classificada da **lista** de entrada.

Pensando dessa maneira, podemos chegar ao algoritmo seguinte.

Algoritmo: Classificação de listas por intercalação de sublistas

Função Mergesort($d:D$) $\rightarrow R$ {classifica lista por intercalação de sublistas}
 { Entrada-saída: clssf (a saída r é a lista d de entrada classificada) }
 sorte $D := \text{Seq}[\text{Elmord}]$ {dados}; $R := \text{Seq}[\text{Elmord}]$ {resultados};
 var $d_1, d_2:D$; $r_1, r_2:R$; $r:R$ {saída}; $n:IN$ {comprimento da lista};
 0. $n \leftarrow \text{compr}(d)$; {comprimento da entrada}
 1. se $n \leq 1$ {testa se a entrada é curta ou não: 12}
 2. então { $n \leq 1$ (saída direta): 5}
 3. $r \leftarrow d$; {entrada curta já está classificada}
 4. retorne-saída(r); {dá saída direta d }
 5. fim-então {2: entrada curta (saída direta)}
 6. senão { $n > 1$ (recursão): 11}
 7. $d_1 \leftarrow \text{Prim}(d)$; $d_2 \leftarrow \text{Fin}(d)$; {parte em duas metades}
 8. $r_1 \leftarrow \text{Mergesort}(d_1)$; $r_2 \leftarrow \text{Mergesort}(d_2)$; {recursão}
 9. $r \leftarrow \text{Intercl}(r_1, r_2)$; {intercala partes classificadas}
 10. retorne-saída(r); {dá saída intercalada r }
 11. fim-senão {6: entrada não curta (recursão)}
 12. fim-se {1: $n \leq 1$?}
 13. fim-Função {fim do algoritmo Mergesort: classificação por intercalação}

A idéia básica da divisão e conquista é decompor um **problema** em **subproblemas menores** independentes, resolvê-los e combinar as soluções obtidas em uma solução para o problema original.

Esse **processo recursivo** funciona da seguinte maneira:

- dada uma entrada, se ela é suficientemente simples, obtemos diretamente uma saída correspondente;
- caso contrário, ela é decomposta em entradas mais simples, para as quais aplicamos o mesmo processo, obtendo saídas correspondentes, que são então combinadas em uma saída para a entrada original.

De acordo com a **estratégia**, cada entrada será decomposta em, digamos, duas partes, cujas saídas serão depois combinadas, fornecendo uma saída para a entrada original. Para isso, usamos procedimentos (funções) como se segue:

- duas funções unárias $part_1$ e $part_2$ para decompor entradas;
- uma função binária $cmbn_2$ para combinar saídas;
- um procedimento unário $smpl$ para testar se a entrada é simples ou não; e
- uma função unária drt dando saídas para entradas simples.

Formulação da divisão e conquista binária

Função: $\text{Div_Conq_2} (d:D) \rightarrow R$ {Divisão e Conquista binária}
 {Sortes D (entrada); R (resultados)} com a seguinte **formulação recursiva**.

$$\text{Div_Conq_2}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{smp}(d) \\ \text{combn_2}(\text{Div_Conq_2}(\text{part}_1(d), \text{Div_Conq_2}(\text{part}_2(d))) & \text{caso contrário} \end{cases}$$

Instâncias de divisão e conquista binária.

Procedimento smt

	Div_Conq_2	smt
Sortes		
entrada	$d:D$	L : Lista de números
saída	R	Número
Teste		
simples	$\text{smp}(d)$	$\text{curta}(L)$
Funções		
direto	$\text{drt}(d)$	$\text{imd}(L)$
primeira parte	$\text{part}_1(d)$	$\text{sublist}_1(L)$
segunda parte	$\text{part}_2(d)$	$\text{sublist}_2(L)$
combinação	$\text{combn_2}(r_1, r_2)$	$\text{soma}(r_1, r_2)$

Algoritmo: MergesortMergesort(d) := Div_Conq_2(d)onde $D := \text{Seq}[\text{Elmord}]; R := \text{Seq}[\text{Elmord}]$ $\text{smpl}(d) := \text{compr}(d) \leq 1$ $\text{drt}(d) := d$ $\text{part}_1(d) := \text{Prim}(d)$ $\text{part}_2(d) := \text{Fin}(d)$ $\text{cmbn_2}(r_1, r_2) := \text{Intercl}(r_1, r_2)$

{invoca Div_Conq_2}

{definições}

{sortes};

{teste smpl};

{operação drt};

{primeira parte};

{segunda parte};

{combinação}.

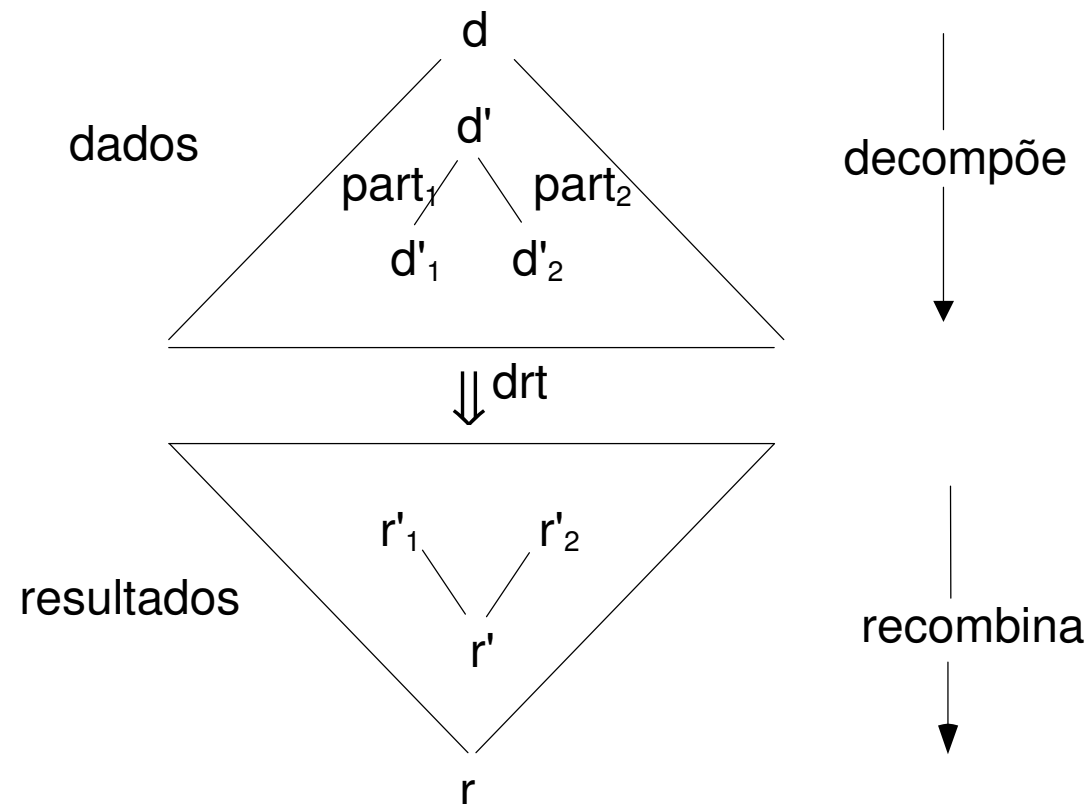
Outros exemplos de algoritmos de divisão e conquista binária são algoritmos:

- de classificação por seleção "selectionsort";
- para determinar máximo e mínimo de tabela.

Algoritmos de divisão e conquista em geral são recursivos. A **recursão** desempenha importante papel na divisão e conquista.

O funcionamento da recursão na divisão e conquista pode ser visualizado considerando-se as árvores que o procedimento abstrato Div_Conq_2 (constrói, em três estágios.

1. **árvore de dados**: é construída da raiz em direção à folhas (por decomposições repetidas das instâncias de dados) até que as folhas sejam simples.
2. aplica-se drt para transferir as folhas do sorte de dados para o de resultados.
3. **árvore de resultados**: é construída das folhas em direção à raiz.



Agora, a formulação binária anterior se generaliza para sua **versão m-ária**.

Função: $\text{Div_Conq_m} (d:D) \rightarrow R$ {Divisão e Conquista m-ária}
 {Sortes D (de entrada); R (de saída)} com a seguinte formulação recursiva

$$\text{Div_Conq_m}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{smpl}(d) \\ \text{combn_m}(\text{Div_Conq_m}(\text{part}_1(d), K, \text{Div_Conq_m}(\text{part}_m(d)))) & \text{caso contrário} \end{cases}$$

Uma formulação mais liberal contempla o caso em que a combinação pode utilizar, além dos resultados das partes, também a entrada que foi decomposta.

Neste caso a operação de combinação é **(m+1)-ária**.

Um exemplo imediato de divisão e conquista ternária ($m=3$) pode ser obtido, modificando-se o algoritmo Quicksort (cf. seção 4.1 do capítulo 4).

Essa nova versão dividiria a entrada em três por comparação com o **pivô**:

- a primeira com os elementos menores do que o pivô,
- a segunda com os elementos iguais ao pivô, e
- a terceira com os elementos maiores do que o pivô. Sua combinação seria a concatenação.

Ainda outro exemplo de divisão e conquista não binária é o **algoritmo de Strassen** para multiplicação de matrizes quadradas.

Para multiplicar duas matrizes $n \times n$, esse algoritmo decompõe cada uma delas em 4 blocos $(n/2) \times (n/2)$ e calcula 7 produtos de matrizes, dos quais os 4 blocos da matriz produto podem ser obtidos.

Examinemos agora o caso de **divisão e conquista unária ($m=1$)**.

Função: $\text{Div_Conq_1} (d:D) \rightarrow R$ {Divisão e Conquista unária}
 {Sortes D {dados de entrada}; R (resultados de saída)}

com a seguinte formulação recursiva

$$\text{Div_Conq_1}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{simpl}(d) \\ \text{combn_1}(\text{Div_Conq_1}(\text{part}_1(d))) & \text{caso contrário} \end{cases}$$

No caso de divisão e conquista unária ($m=1$) o termo 'divisão' não parece apropriado, 'redução' sendo talvez mais adequado.

Projeto de algoritmos por divisão e conquista

A fim de aplicar a estratégia de divisão e conquista, devemos formular os **procedimentos auxiliares** de maneira apropriada. Problemas de classificação podem ser usados para ilustrar essa idéia.

Divisão e conquista para classificação.

Consideremos algoritmos de classificação, cujas entradas e saídas consistem de seqüências de elementos (de um conjunto ordenado).

Podemos definir o teste smpl e a função unária drt , sobre dados, por

$$\text{smpl}(d) \text{ sse } \text{compr}(d) \leq 1 \text{ e } \text{drt}(d) := d.$$

Duas possíveis definições para decomposições e recombinações são as seguintes.

1. Podemos definir $\text{part}_1(d)$ e $\text{part}_2(d)$ como a primeira e segunda metades de d e $\text{cmbn_2}(r_1, r_2)$ como a intercalação de r_1 com r_2 .
2. Também podemos definir $\text{part}_1(d)$ como a seqüência consistindo do valor mínimo em d , $\text{part}_2(d)$ como o resultado de remover esse valor mínimo de d , e $\text{cmbn_2}(r_1, r_2)$ como a concatenação de r_1 seguida por r_2 .

Essas **duas** maneiras de implementar dão origem a algoritmos de classificação familiares:

- por intercalação (mergesort) no caso 1, e
- seleção (straight selectionsort) no caso 2.

Em ambos os casos cada entrada é decomposta em duas menores, mas com distintas relações de tamanho.

Essa distinção entre as duas relações de tamanho influencia a **altura da árvore** de execução.

Considerando a entrada $d := [1, 2, 3, 4]$, teremos as seguintes árvores de dados:

1. na classificação por intercalação (mergesort)

$$\begin{array}{c} [1,2,3,4] \\ \hline [1,2] \quad [3,4] \\ \hline [1] \quad [2] \quad [3] \quad [4] \end{array}$$

2. na classificação por seleção (straight selectionsort)

$$\begin{array}{c} [1,2,3,4] \\ \hline [1] \quad [2,3,4] \\ \quad \quad \hline [2] \quad [3,4] \\ \quad \quad \quad \hline [3] \quad [4] \end{array}$$

O exemplo ilustra uma idéia importante na estratégia de divisão e conquista de que entrada em partes (aproximadamente) iguais leva a árvores menores.

O chamado *princípio de equipartição* aconselha partir a entrada em partes (aproximadamente) iguais a fim de diminuir a profundidade das árvores, assim tendendo a melhorar o desempenho do algoritmo resultante.

Máximo e mínimo de tabela por divisão e conquista

Algoritmo: Máximo e mínimo de tabela (divisão e conquista binária)

Função Max_min(Tab:D) $\rightarrow$ R {determina máximo e mínimo de tabela}

{ Saída: valores r_M :Elmord(máximo) e r_m :Elmord (mínimo) da entrada}
sorte D := vetor [p..q] de Elmord {dados}; R := Par[Elmord] {resultados};
var {auxiliares} p_s, q_i : IN {índices de subtabelas}; s_M, s_m, t_M, t_m : Elmord
 {máximo e mínimo de partes}; n, k : IN {tamanhos de tabela};

```

0.  $n \leftarrow q - p + 1$ ;                                {tamanho da tabela}
1. se  $n \leq 2$                                          {testa se a tabela é pequena ou não: 23}
2. então                                             { $n \leq 2$  (saída direta): 14}
3.   se  $n = 1$                                        { $n = 1$ ?: 12}
4.   então                                         { $n = 1$ : 7}
5.      $r_M \leftarrow \text{Tab}[p]$ ;                    {máximo}
```

6.	$r_m \leftarrow \text{Tab}[p] ;$	{mínimo}
7.	<u>fim-então</u>	{4: n=1}
8.	<u>senão</u>	{n=2:11}
9.	$r_M \leftarrow \text{Máx}(\text{Tab}[p], \text{Tab}[q]) ;$	{máximo}
10.	$r_m \leftarrow \text{mín}(\text{Tab}[p], \text{Tab}[q]) ;$	{mínimo}
11.	<u>fim-senão</u>	{8: n=2}
12.	<u>fim-se</u>	{3: n=1?}
13.	<u>retorne-saída</u> (r_M, r_m) ;	{dá saída direta}
14.	<u>fim-então</u>	{2: tabela pequena (saída direta)}
15.	<u>senão</u>	{n>2 (recursão): 22}
16.	$k \leftarrow \lceil n/2 \rceil ; q_i \leftarrow p+k ; p_s \leftarrow q_i - 1 ;$	{duas metades}
17.	$(s_M, s_m) \leftarrow \text{Max_min}(\text{Tab}[p..p_s]) ;$	{recursão: 1ª metade}
18.	$(t_M, t_m) \leftarrow \text{Max_min}(\text{Tab}[q_i..q]) ;$	{recursão: 2ª metade}
19.	$r_M \leftarrow \text{Máx}(s_M, t_M) ;$	{combinação: máximo}
20.	$r_m \leftarrow \text{mín}(s_m, t_m) ;$	{combinação: mínimo}
21.	<u>retorne-saída</u> (r_M, r_m) ;	{dá saída combinada}
22.	<u>fim-senão</u>	{15: tabela não pequena (recursão)}
23.	<u>fim-se</u>	{1: n≤2?}
24.	<u>fim-Função</u>	{fim do algoritmo Max_min: máximo e mínimo de tabela}

E algoritmo **Max_min** é uma instância da formulação acima da divisão e conquista binária Div_Conq_2.

Examinemos agora exemplos de algoritmos projetados por divisão e conquista unária.

Ocorrência de chave em tabela por divisão e conquista.

Este algoritmo determina se uma **chave** ocorre ou não uma tabela, armazenada como vetor tab, por meio de divisão e conquista unária.

Caso o elemento procurado ainda não tenha sido encontrado, a busca original é reduzida a uma numa tabela com um elemento a menos.

Algoritmo: Ocorrência em tabela (por divisão e conquista unária)

Função Oc_seq(ch:Elm, tab:Cnj) $\rightarrow$ Bool {determina se chave ocorre em tabela}

{Entrada: par consistindo de chave ch:Elm e tabela tab:Cnj
Saída: V (se ch ocorre em tab) e F (se ch não ocorre em tab)}

sorte Cnj := vetor[p..q] de Elm;

```

    var {auxiliares} achou:Bool; n:IN {tamanho de tabela};
0. n ← q - p + 1;                                {tamanho da tabela}
1. se n=1                                         {testa se a tabela é pequena ou não: 17}
2. então                                         {n=1 (saída direta): 7}
3.     se tab[p]=ch                               {tab[p]=ch?: 6}
4.         então achou ← V ;                      {tab[p]=ch}
5.         senão achou ← F ;                      {tab[p]≠ch}
6.     fim-se                                    {3: tab[p]=ch?}
7.     retorne-saída(achou) ;                    {dá saída direta}
8. fim-então                                    {2: tabela pequena (saída direta)}
9. senão                                         {n>1 (recursão): 16}
10.    se tab[p]=ch                               {tab[p]=ch?: 13}
11.        então q ← p ;                          {tab[p]=ch}
12.        senão p ← p + 1 ;                      {tab[p]≠ch}
13.    fim-se                                    {9: tab[p]=ch?}
14.    achou ← Oc_seq(ch:Elm, tab:Cnj) ;           {recursão: parte}
15.    retorne-saída(achou) ;                    {retorna saída}
16. fim-senão                                    {9: tabela não pequena (recursão)}
17. fim-se                                       {1: n=1?}
18. fim-Função    {fim do algoritmo Oc_seq: ocorrência em tabela (seqüencial)}

```

Este algoritmo simula uma busca seqüencial, reduzindo a busca original a uma numa tabela com um elemento a menos, caso o elemento procurado ainda não tenha sido encontrado.

Esse algoritmo foi obtido especializando-se a formulação **Div_Conq_1 da divisão e conquista unária**. Podemos identificar a seguinte estrutura.

Oc_seq(d) := Div_Conq_1 (d)	{invoca Div_Conq_1}
<u>onde</u>	{definições}
D := Par[Elmx Cnj]; R := Bool	
<u>com</u> Cnj := <u>vetor</u> [p..q] <u>de</u> Elm	{sortes};
smp1(ch, tab) := compr(tab) = 1	{teste smp1};
drt(ch, tab) := linhas de 2 a 6	{operação drt};
part ₁ (ch, tab) := linhas de 10 a 13	{parte};
cmbn_1(r ₁) := r ₁	{combinação}.

Essa estrutura exhibe o algoritmo Oc_seq como uma instância da formulação da divisão e conquista unária.

Examinemos um exemplo de algoritmo projetados por divisão e conquista liberal.

Consideremos o problema de determinar o mínimo de uma tabela, um problema análogo ao visto na seção 2.4 do capítulo 2 (cf. algoritmo 2.1 no exemplo 2.4.1).

Mínimo de tabela por divisão e conquista.

Algoritmo: Mínimo de tabela (por divisão e conquista unária liberal)

Função El_mín(tab:D) → R {determina mínimo de tabela}
 { Entrada: tabela tab:D representando elementos (de conjunto ordenado)
 Saída: valor r :Elmord (mínimo da tabela tab:D de entrada) }
 sorte D := vetor [p..q] de Elmord {dados}; R := Elmord {resultados};
 var {auxiliares} d₁:D {parte de tabela}; r, r₁:Elmord {mínimo (de parte)};
 n:IN {tamanho de tabela};

0. n ← q - p + 1; {tamanho da tabela}
1. se n ≤ 1 {testa se a tabela é pequena ou não: 9}
2. então retorne-saída(tab[p]); {dá saída direta}
3. senão {n > 1 (recursão): 8}
4. d₁ ← tab[p + 1..q]; {parte}
5. r₁ ← El_mín(d₁); {recursão: parte}
6. r ← mín(tab[p], r₁); {combinação (liberal)}
7. retorne-saída(r); {retorna saída}

- | | |
|-----------------------|---|
| 8. <u>fim-senão</u> | {3: tabela não pequena (recursão)} |
| 9. <u>fim-se</u> | {1: $n=1?$ } |
| 10. <u>fim-Função</u> | {fim do algoritmo El_mín: mínimo de tabela} |

O algoritmo Mínimo de tabela foi obtido especializando-se a formulação Div_Conq_1L da **divisão e conquista unária liberal**.

El_mín(tab) := Div_Conq_1L(tab)	{invoca Div_Conq_1L}
<u>onde</u>	{definições}
D := <u>vetor</u> [p..q] <u>de</u> Elmord; R := Elmord	{sortes};
smp1(tab) := compr(tab) ≤ 1	{teste smp1};
drt(tab) := tab[p]	{operação drt};
part ₁ (tab) := tab[p+1..q]	{parte};
cmbn_1L(r ₁ , tab) := mín(tab[p], r ₁)	{combinação liberal}.

Um exemplo interessante de divisão e conquista ternária ($m=3$): um algoritmo para multiplicar números inteiros em uma base (e. g. em base 2).

Produto de números binários por divisão e conquista.

Calcula o produto de dois números binários p e q , com n bits, escrevendo-os na forma $p = p_1 * 2^{(n/2)} + p_2$ e $q = q_1 * 2^{(n/2)} + q_2$, onde p_1, p_2, q_1 e q_2 têm $n/2$ bits cada um.

Algoritmo: Produto de dois números binários (divisão e conquista ternária)

Função Prod_bin($p, q: \text{Bin}$) $\rightarrow \text{Bin}$ {calcula o produto de 2 números binários}
 { Entrada: números binários $p, q: \text{Bin}$ (na forma $p = p_1 * 2^{(n/2)} + p_2$ e $q = q_1 * 2^{(n/2)} + q_2$)
 Saída: valor $r: \text{Bin}$ (produto dos números binários $p, q: \text{Bin}$ de entrada) }
 var {auxiliares} $x, y, z, t, u, v: \text{Bin}; n: \mathbb{N}$ {números de bits};

1. se $n = 1$ {testa se os números são pequenos ou não: 15}
2. então { $n = 1$ (saída direta): 5}
3. $r \leftarrow p.q;$ {multiplica seus (únicos) bits}
4. retorne-saída(r) {dá saída direta}
5. fim-então {2: números pequenos (saída direta)}
6. senão { $n > 1$ (recursão): 14}
7. $x \leftarrow p_1 + p_2; y \leftarrow q_1 + q_2;$ {soma números com $n/2$ bits}
8. $z \leftarrow \text{Prod_bin}(x_2, y_2);$ {recursão com $n/2$ bits}
9. $t \leftarrow (x_1 * y_1) * 2^n + (x_1 * y_2 + x_2 * y_1) * 2^{(n/2)} + z;$
10. $u \leftarrow \text{Prod_bin}(p_1, q_1);$ {recursão com $n/2$ bits}
11. $v \leftarrow \text{Prod_bin}(p_2, q_2);$ {recursão com $n/2$ bits}
12. $r \leftarrow u * 2^n + (t - u - v) * 2^{(n/2)} + v;$ {combinação}
13. retorne-saída(r) {dá saída combinada}
14. fim-senão {6: números não pequenos (recursão)}

15. fim-se {1: n=1?}
16. fim-Função {fim do algoritmo Prod_bin: produto de números binários}

O algoritmo Prod_bin é uma instância da formulação acima da divisão e conquista ternária Div_Conq_3 (com $m:=3$).

Análise da complexidade de algoritmos de divisão e conquista

Agora vamos ver como se pode efetuar a **análise de complexidade pessimista** de alguns algoritmos típicos desenvolvidos pela estratégia de divisão e conquista.

Em cada caso, faremos inicialmente uma **estimativa** da complexidade, passando a seguir a efetuar uma análise mais cuidadosa e detalhada

Algoritmos desenvolvidos por divisão e conquista geralmente são recursivos, tendo análise menos simples do que no caso de algoritmos iterativos.

Aqui, uma questão **fundamental** é a determinação do número de chamadas recursivas (similar à questão do número de iterações quando indefinidas).

Complexidade de ocorrência em tabela.

Consideremos o algoritmo Oc_seq (para determinar se uma chave ocorre numa tabela por busca seqüencial)

Esse **algoritmo Oc_seq** foi obtido especializando-se a formulação Div_Conq_1 da divisão e conquista unária.

Podemos estimar a complexidade pessimista do algoritmo Oc_seq da seguinte maneira:

- a cada invocação recursiva o tamanho da tabela diminui de 1, assim serão feitas n chamadas recursivas para uma tabela de tamanho n;
- no caso simples (linhas de 2 a 8), é feito um acesso à tabela (linha 3);
- no caso não simples (linhas de 9 a 16), é feito um acesso à tabela (linha 10).

Portanto, a **complexidade do algoritmo Oc_seq** tem ordem

$$c_p^{\leq} [\text{Oc_seq}](n) = \Theta(n).$$

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo `Oc_seq` recebe como entrada d um par ordenado (registro), consistindo de:

uma chave $ch:Elm$ e uma tabela $tab:\underline{vetor}[p..q]$ de Elm .

Como é natural, para tamanho $tam(d)$ da entrada d é tomada a dimensão $compr(tab)$ da tabela tab , calculado no algoritmo `Oc_seq` pela linha

0. $n \leftarrow q - p + 1$; {tamanho da tabela}

e seu teste de simplicidade é se $n=1$ (linha 1).

Vemos que o teste de simplicidade do algoritmo `Oc_seq` é caracterizado por

$simpl(d)$ sse $tam(d) \leq 1$ (cf linha 1 se $n=1$).

Afim de determinar quantas invocações recursivas são feitas, ou seja quantas vezes a **entrada d** deve ser partida (por aplicação de $part_1$) até que o teste $simpl$ seja satisfeito, examinemos o trecho das linhas 10 a 13:

10. <u>se</u> $tab[p]=ch$	{ $tab[p]=ch?$: 13}
11. <u>então</u> $q \leftarrow p$	{ $tam[part_1(d)]=q-q+1$ }
12. <u>senão</u> $p \leftarrow p+1$	{ $tam[part_1(d)]=q-(p+1)+1$ }
13. <u>fim-se</u>	{9: $tab[p]=ch?$ }

Assim, temos $tam[part_1(d)] = 1$ na linha 11 e $tam[part_1(d)] = q - p$ na linha 12. Logo:

$$tam[part_1(d)] \leq tam(d) - 1.$$

Pelo visto acima, os dados dessa seqüência têm tamanhos como se segue

dados	d_0	d_1	$\dots$	d_j	$\dots$	$d_{H(d)}$
tamanhos	n_0	$n_0 - 1$	$\dots$	$n_0 - j$	$\dots$	$n_0 - H(d)$

Quanto vale $H(d_0)$?

A **altura da árvore** de execução: a primeira vez que se consegue satisfazer o teste `smpl`.

$$H(d_0) = \min\{j \in \mathbb{N} / n_0 - j \leq 1\}.$$

Portanto, a árvore de execução tem altura $H(d_0) \leq n_0 - 1$, ou seja

$$H(d) \leq tam(d) - 1.$$

Prosseguindo de modo similar ao caso da iteração indefinida (cf. 3.2.5 na seção 3.2 do capítulo 3), o desempenho do algoritmo `Oc_seq` sobre entrada d é:

- Se a entrada d satisfaz o teste $\text{smpl}(d)$ (i. e. $\text{tam}(d) \leq 1$):

$$\text{desemp}[\text{Oc_seq}](d) = \text{ava}[\text{smpl}](d) + \text{desemp}[\text{drt}](d).$$

- Se a entrada d não satisfaz o teste $\text{smpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{Oc_seq}](d) = & \text{ava}[\text{smpl}](d) + \text{desemp}[\text{part}_1](d) + \\ & + \text{desemp}[\text{Oc_seq}](\text{part}_1(d)) + \\ & + \text{desemp}[\text{cmbn_1}](\text{Oc_seq}(\text{part}_1(d))). \end{aligned}$$

Nesse caso, é natural tomar como **operação fundamental** a comparação $\text{tab}[i] = \text{ch}$ de elemento acessado na tabela com a chave ch . Assim, temos

Processo	linhas	comparações
smpl	0 a 1	0
drt	2 a 6	1
part_1	10 a 13	1
cmbn_1	15	0

Então, em termos de número de comparações, obtemos:

- para a entrada simples (com $tam(d) \leq 1$), $desemp[Oc_seq](d)$ tem as contribuições

$$\begin{array}{rcl} ava[smpl](d) & = & 0 \\ + & & + \\ desemp[drt](d) & = & 1 \end{array}$$

i. e. $desemp[Oc_seq](d) = 1$;

- para a entrada não simples (com $tam(d) > 1$), $desemp[Oc_seq](d)$ tem as contribuições

$$\begin{array}{rcl} ava[smpl](d) & = & 0 \\ + & & + \\ desemp[part_1](d) & = & 1 \\ + & & + \\ desemp[Oc_seq](part_1(d)) & = & desemp[Oc_seq](part_1(d)) \\ + & & + \\ desemp[cmbn_1](Oc_seq(part_1(d))) & = & 0 \end{array}$$

i. e. $desemp[Oc_seq](d) = 1 + desemp[Oc_seq](part_1(d))$.

Assim, o desempenho do algoritmo Oc_seq sobre entrada d é descrito pela recorrência;

$$desemp[Oc_seq](d) = \begin{cases} 0 & \text{se } tam(d) \leq 1 \\ desmp[Oc_seq](part_1(d)) + 1 & \text{se } tam(d) > 1 \end{cases}$$

E o desempenho do algoritmo Oc_seq sobre entrada d é dado pela somatória

$$\frac{\begin{array}{c} \text{rec.} \\ 1 \quad + \quad \dots \quad + \quad 1 \end{array}}{H(d)} + \frac{\begin{array}{c} \text{fim} \\ 1 \end{array}}$$

Assim, obtemos a seguinte cota superior para o **desempenho do algoritmo Oc_seq**

$$desemp[Oc_seq](d) = H(d) + 1 \leq tam(d).$$

O pior caso, para entradas com tamanho até n , vem a ser

$$h_{\leq}(n) := \text{Máx}\{H(d) \in \mathbb{N} / tam(d) \leq n\}.$$

Agora, a complexidade pessimista do algoritmo Oc_seq vem a ser

$$c_p^{\leq}[Oc_seq](n) := \text{Máx}\{desemp[Oc_seq](d) / tam(d) \leq n\};$$

Logo, é cota superior para a complexidade pessimista de Oc_seq

$$c_p^{\leq} [\text{Oc_seq}](n) \leq n.$$

Portanto, a **complexidade pessimista do algoritmo Oc_seq** tem ordem

$$c_p^{\leq} [\text{Oc_seq}](n) = \Theta(n).$$

Essa análise mostra que a complexidade pessimista do algoritmo Oc_seq, em termos de número de comparações com a chave, tem ordem linear no tamanho da entrada.

Examinemos agora um caso de divisão e conquista unária liberal.

Complexidade de mínimo de tabela.

Consideremos o **algoritmo El_mín** (para determinar o elemento mínimo de um conjunto ordenado, armazenado como vetor tab por busca seqüencial).

Esse algoritmo El_mín foi obtido especializando-se a formulação Div_Conq_1L da divisão e conquista unária liberal.

Podemos estimar a complexidade pessimista do algoritmo El_mín como se segue.

- A cada invocação recursiva o tamanho da tabela diminui de 1, assim serão feitas n chamadas recursivas para uma tabela de tamanho n .
- O caso simples (linha 2) não envolve nenhuma comparação.
- No caso não simples (linhas de 3 a 8) envolve uma comparação (linha 6).

Portanto, a **complexidade do algoritmo El_mín** tem ordem

$$c_p^{\leq}[\text{El_mín}](n) = \Theta(n).$$

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo El_mín recebe como entrada d uma tabela $\text{tab} : \text{vetor}[p..q]$ de Elmord.

para $\text{tam}(d)$ é tomada a dimensão $\text{compr}(\text{tab})$ da tabela tab .

Esse tamanho é calculado na linha 0 do algoritmo El_mín e seu teste de simplicidade é se $n \leq 1$ (linha 1), logo

$$\text{simpl}(d) \text{ sse } \text{tam}(d) \leq 1.$$

Queremos saber quantas invocações recursivas são feitas até que o teste simpl seja satisfeito.

No algoritmo El_mín a operação de partir é

$$\text{part}_1(\text{tab}) := \text{tab}[p+1..q] \text{ (linha 4).}$$

Assim, temos $\text{tam}[\text{part}_1(d)] = q - (p+1) + 1$. Logo

$$\text{tam}[\text{part}_1(d)] = \text{tam}(d) - 1.$$

O algoritmo El_mín aplicado a uma entrada d fará uma seqüência de chamadas recursivas até que o teste smpl seja satisfeito, i. e. até obter

$$d_{H(d)} \text{ com } \text{tam}[d_{H(d)}] \leq 1.$$

Mas, $\text{tam}(d_j) = \text{tam}(d) - j$.

A altura $H(d_0)$ da árvore de execução é

$$H(d_0) = \text{mín} \{j \in \mathbb{N} / \text{tam}(d_0) - j \leq 1\}.$$

Portanto, a **árvore de execução** tem altura $H(d) = \text{tam}(d) - 1$.

Com $H(d)$ o desempenho do algoritmo El_mín sobre entrada d é como se segue:

- Se a entrada d satisfaz o teste $\text{smpl}(d)$ (i. e. $\text{tam}(d) \leq 1$):

$$\text{desemp}[\text{El_mín}](d) = \text{ava}[\text{smpl}](d) + \text{desemp}[\text{drt}](d).$$

- Se a entrada d não satisfaz o teste $\text{simpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{El_mín}](d) = & \text{aval}[\text{simpl}](d) + \text{desemp}[\text{part}_1](d) + \\ & + \text{desemp}[\text{El_mín}](\text{part}_1(d)) + \\ & + \text{desemp}[\text{cmbn_1L}](d, \text{El_mín}(\text{part}_1(d))). \end{aligned}$$

O **natural** é tomar como operação fundamental $\text{mín}(r, \text{tab}[i])$ que compara um elemento r de Elmord com o elemento $\text{tab}[i]$ acessado na tabela.

Mas, só $\text{cmbn_1L}(r_1, \text{tab}) := \text{mín}(\text{tab}[p], r_1)$ usa essa operação (uma vez).

Então, em termos de número de comparações, o desempenho do algoritmo El_mín sobre entrada d é descrito pela recorrência

$$\text{desemp}[\text{El_mín}](d) = \begin{cases} 0 & \text{se } \text{tam}(d) \leq 1 \\ \text{desemp}[\text{El_mín}](\text{part}_1(d)) + 1 & \text{se } \text{tam}(d) > 1 \end{cases}$$

Portanto, o **desempenho** do algoritmo El_mín sobre entrada d é dado por

$$\frac{1 \quad + \quad \dots \quad + \quad 1}{H(d)}$$

Assim, obtemos

$$desemp[El_mín](d) = H(d) = tam(d).$$

Como antes, vemos que o **pior caso** da profundidade de recursão do algoritmo `El_mín` sobre entradas com tamanho até n , é dado por

$$h_{\leq}(n) = n; \text{ logo } h_{\leq}(n) = \Theta(n).$$

Portanto, obtemos $c_p^{\leq}[El_mín](n) = n$, logo

$$c_p^{\leq}[El_mín](n) = \Theta(n).$$

Complexidade de classificação de listas por intercalação.

Consideremos o algoritmo Mergesort(para classificar listas por intercalação de sublistas), obtido especializando-se a formulação `Div_Conq_2` da divisão e conquista binária.

Estimativa da **complexidade pessimista do algoritmo Mergesort**:

- A cada invocação recursiva a lista é dividida aproximadamente ao meio, assim serão feitas $\Theta(\log_2 n)$ chamadas recursivas para uma tabela de tamanho n .
- Só $r \leftarrow \text{Intercl}(r_1, r_2)$ (linha 9) envolve comparações: $O(n)$.

Assim, podemos ver que $c_p^{\leq} [\text{Mergesort}](n) = O(n \cdot \log_2 n)$.

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo Mergesort recebe como entrada uma lista $d: \text{Seq}[\text{Elmord}]$.

para $\text{tam}(d)$ tomemos o comprimento $\text{compr}(d)$ da lista d .

Esse comprimento é determinado na linha 0 e seu teste de simplicidade é

se $n \leq 1$ (linha 1), e o teste de simplicidade $\text{smpl}(d)$ sse $\text{tam}(d) \leq 1$.

Agora surge a questão de quantas invocações recursivas são feitas, ou seja quantas vezes a **entrada d** deve ser partida até que o teste smpl seja satisfeito.

As operações part_1 e part_2 do algoritmo Mergesort são efetuadas pela linha 7:

$$\text{part}_1(d) := \text{Prim}(d) \qquad \{\text{compr}[\text{Prim}(d)] = \lfloor \text{compr}(d)/2 \rfloor\}$$

$$\text{part}_2(d) := \text{Fin}(d) \qquad \{\text{compr}[\text{Fin}(d)] = \lceil \text{compr}(d)/2 \rceil\}.$$

Assim, temos $\text{tam}[\text{part}_1(d)] = \lfloor \text{tam}(d)/2 \rfloor$ e $\text{tam}[\text{part}_2(d)] = \lceil \text{tam}(d)/2 \rceil$.

Consideremos o tamanho máximo das decomposições de entrada d (não simples):

$$S(d) := \text{Máx}\{\text{tam}[\text{part}_1(d)], \text{tam}[\text{part}_2(d)]\}.$$

Obtemos $S(d) = \lceil \text{tam}(d)/2 \rceil$.

O algoritmo **Mergesort** aplicado a uma entrada d_0 fará uma seqüência de chamadas recursivas gerando (por aplicação de part_1 e part_2) uma árvore de dados até que o teste smpl seja satisfeito, ou seja até obter $d_{H(d)}$ com $\text{tam}[d_{H(d)}] \leq 1$.

Os níveis dessa árvore têm tamanhos máximos como se segue:

nível d_i	d_0	d_1	...	d_j	...	$d_{H(d)}$
tamanho $S(d_j)$	n_0	$\lceil n_0/2 \rceil$	...	$\lceil n_0/(2^j) \rceil$	...	$\lceil n_0/(2^{H(d)}) \rceil$

Quanto vale $H(d_0)$? Trata-se da altura da árvore de execução: a primeira vez em que todas as folhas satisfazem o teste *smpl*. Assim, temos

$$H(d_0) = \min\{j \in \mathbb{N} / \lceil \text{tam}(d_0)/(2^j) \rceil \leq 1\}.$$

A árvore de execução tem altura $H(d_0) = \lceil \log_2 n_0 \rceil$, ou seja

$$H(d) = \lceil \log_2 [\text{tam}(d)] \rceil.$$

De posse da altura $H(d)$ o **desempenho** do algoritmo sobre entrada d é:

- Se a entrada d satisfaz o teste *smpl*(d) (i. e. $\text{tam}(d) \leq 1$):
 $\text{desemp}[\text{Mergesort}](d) = \text{ava}[\text{tam}(d) \leq 1] + \text{desemp}[r \leftarrow d].$

- Se a entrada d não satisfaz o teste $\text{simpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{Mergesort}](d) = & \text{aval}[\text{tam}(d) \leq 1] + \\ & + \text{desemp}[d_1 \leftarrow \text{Prim}(d)] + \text{desemp}[d_2 \leftarrow \text{Fin}(d)] + \\ & + \text{desemp}[\text{Mergesort}](\text{Prim}(d)) + \text{desemp}[\text{Mergesort}](\text{Fin}(d)) + \\ & + \text{desemp}[\text{Intercl}](\text{Mergesort}(\text{Prim}(d)), \text{Mergesort}(\text{Fin}(d))). \end{aligned}$$

Nesse caso, é natural tomar como **operação fundamental** a comparação entre elementos de Elmord.

Mas só $\text{cmbn_2}(r_1, r_2) := \text{Intercl}(r_1, r_2)$ envolve tais comparações.

O desempenho de $\text{Intercl}(r_1, r_2)$ é dado por

$$\text{desemp}[\text{Intercl}](r_1, r_2) = \text{tam}(r_1) + \text{tam}(r_2) - 1.$$

Então, em termos de número de comparações, obtemos as expressões:

Para a entrada **simples** (com $\text{tam}(d) \leq 1$):

$$\begin{aligned} \text{desemp}[\text{Mergesort}](d) = & \text{aval}[\text{tam}(d) \leq 1] + \text{desemp}[r \leftarrow d] = \\ & = 0 + 0 = 0. \end{aligned}$$

Para a entrada **não simples** (com $tam(d) > 1$):

$$\begin{aligned}
 desemp[Mergesort](d) &= aval[tam(d) \leq 1] + \\
 &+ desemp[d_1 \leftarrow Prim(d)] + desemp[d_2 \leftarrow Fin(d)] + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ desemp[Intercl](Mergesort(Prim(d)), Mergesort(Fin(d))) = \\
 &= 0 + 0 + 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam[Mergesort(Prim(d))] + tam[Mergesort(Fin(d))] - 1.
 \end{aligned}$$

Assim, já que Mergesort preserva tamanho: $tam(Mergesort(d)) = tam(d)$, temos

$$\begin{aligned}
 desemp[Mergesort](d) &= 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam(Prim(d)) + tam(Fin(d)) - 1.
 \end{aligned}$$

Uma vez que $tam(Prim(d)) + tam(Fin(d)) = tam(d)$, obtemos

$$\begin{aligned}
 desemp[Mergesort](d) &= 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam(d) - 1.
 \end{aligned}$$

Assim, o desempenho do algoritmo Mergesort sobre entrada d é descrito pela recorrência

$$\text{desemp[Mergesort]}(d) = \begin{cases} 0 & \text{se } \text{tam}(d) \leq 1 \\ \text{desemp[Mergesort]}(\text{Prim}(d)) + \\ \text{desemp[Mergesort]}(\text{Fin}(d)) + \text{tam}(d) - 1 & \text{se } \text{tam}(d) > 1 \end{cases}$$

Agora, consideremos o **pior caso**, para entradas com tamanho até $n > 1$.

O tamanho máximo das decomposições de tais entradas vem a ser

$$s(n) := \text{Máx}\{S(d) \in \mathbb{N} / \text{tam}(d) \leq n\} \text{ e } s(n) = \lceil n/2 \rceil, \text{ logo } s(n) = \Theta(n/2).$$

A altura máxima das árvore de execução de tais entradas vem a ser

$$h^{\leq}(n) := \text{Máx}\{H(d) \in \mathbb{N} / \text{tam}(d) \leq n\} \text{ e } h^{\leq}(n) = \lceil \log_2 n \rceil, \text{ logo } h^{\leq}(n) = \Theta(\log_2 n).$$

Como $\text{part}_1(d) := \text{Prim}(d)$ e $\text{part}_2(d) := \text{Fin}(d)$, para $j = 1, 2$, temos

$$\text{desemp[Mergesort]}(\text{part}_j(d)) \leq c_{\bar{p}}^{\leq}[\text{Mergesort}](\text{part}_j(d)) \leq c_{\bar{p}}^{\leq}[\text{Mergesort}](s(n)).$$

Assim, uma **cota superior** $CS^{\leq}(n)$ para a ordem da complexidade pessimista de Mergesort pode ser dada pela seguinte recorrência

$$CS^{\leq}(n) = \begin{cases} 0 & \text{se } n \leq 1 \\ n + 2 \cdot CS^{\leq}\left(\frac{n}{2}\right) & \text{se } n > 1 \end{cases}$$

Agora, essa cota superior $CS^{\leq}(n)$ é dada por:

$$\begin{aligned} CS^{\leq}(n) &= n + 2 \cdot CS^{\leq}(n/2) = \\ &= n + 2 \cdot (n/2 + 2 \cdot CS^{\leq}(n/2^2)) = \\ &= n + n + 2^2 \cdot (n/2^2 + 2 \cdot CS^{\leq}(n/2^3)) = \\ &\quad \dots \\ &= n + n + \dots + n + \dots + 2^{h^{\leq}(n)} \cdot CS^{\leq}(n/2^{h^{\leq}(n)}). \end{aligned}$$

Então, o valor da cota superior $CS^{\leq}(n)$ é dado pela somatória:

$$\frac{n + \dots + n}{h^{\leq}(n)} + 0$$

Assim, obtemos $CS^{\leq}(n) = n \cdot h^{\leq}(n) = n \cdot \lceil \log_2 n \rceil$, logo

$$CS^{\leq}(n) = \Theta(n \cdot \log_2 n).$$

Portanto, para a complexidade pessimista do algoritmo Mergesort temos

$$c_p^{\leq}[\text{Mergesort}](n) \leq CS^{\leq}(n) = O(n \cdot \log_2 n).$$

Essa análise mostra que a complexidade pessimista do algoritmo Mergesort, em termos de número de comparações, tem ordem

$$c_p^{\leq}[\text{Mergesort}](n) = O(n \cdot \log_2 n).$$

6. COMPLEXIDADE DO PROBLEMA

- Até aqui, neste trabalho, o centro das atenções foi o algoritmo. Neste capítulo, o ponto de referência deixará de ser o algoritmo para ser o **problema**.
- O objetivo é estudar as limitações dos problemas com respeito à complexidade dos algoritmos que os resolvem.
- O que se deve pensar antes de codificar um problema?
 - o O problema é computável?
 - o Será que existe solução (algoritmo) eficiente para este problema?
- A questão da computabilidade foi proposta por Hilbert em 1900, que conjecturava que todo problema bem posto seria demonstrável.
- Em 1931, Gödel apresentou o “**Teorema da Completude**”, mas ainda havia a questão formal da definição de computabilidade, que foi fechada em 1936, com a apresentação e a aceitação da “**Tese de Church**”.

- A questão $P \times NP$ surgiu em 1971, com o artigo “The complexity of theorem-proving procedures”, e tem sido intensamente estudada. A conjectura que existe é, “ $P \neq NP$ ”. A partir dessa conjectura, foi criada toda a teoria, na qual a figura principal é a classe NP-Completa.

Visão geral do capítulo

- 6.1 Limite de complexidade de um problema.

Apresenta os conceitos referentes à complexidade, como limite superior de complexidade de um problema e complexidade mínima de problema.

- 6.2 Intratabilidade.

Define, relaciona e ilustra com exemplos as principais classes de problemas, como : P , NP , NP -Completo, Co - NP e NP -Difícil.

6.1 LIMITE DE COMPLEXIDADE DE UM PROBLEMA

- O **limite superior** de complexidade de um problema refere-se ao **melhor algoritmo que o resolve**. Por exemplo, para o problema de Classificação, no capítulo 3, apresenta-se o algoritmo 3.3.3: Classificação de Vetor, com complexidade $O(n^2)$. Já no capítulo 5, o algoritmo do exemplo 5.2.3: *Mergsort* tem complexidade $O(n \log_2 n)$, e $n \log_2 n$ é um limite superior para a Classificação, no sentido que é possível classificar qualquer lista com uma complexidade $O(n \log_2 n)$.
- O **limite inferior** de um problema refere-se à **melhor complexidade possível**. Os problemas têm suas dificuldades, que não permitem o desenvolvimento de algoritmos melhores do que um certo limite de complexidade. Um limite inferior **é um resultado teórico** que determina não ser possível desenvolver um algoritmo para o problema com complexidade melhor que uma certa função. Esta função então é um *limite inferior de complexidade do problema*.
- A técnica mais simples de **cálculo do limite inferior** é simplesmente **contar as entradas e as saídas produzidas**, pois o algoritmo precisa, no mínimo, ler a entrada e dar as saídas.

- Um limite inferior pode ser obtido sem, entretanto, conhecer-se um algoritmo que realmente atinja essa complexidade. Nesse caso, o limite inferior conhecido não é igual ao limite superior conhecido.
- Enquanto houver diferença entre os limites de complexidade inferior e superior para o problema, a questão pode progredir nos dois sentidos:
 - o Um algoritmo melhor que o limite superior pode ser desenvolvido, ou um cálculo mais apertado da complexidade mínima pode ser alcançada, diminuindo a diferença entre esses dois limites.
 - o Quando essa **diferença desaparece**, a complexidade mínima do problema é conhecida. Então, a complexidade mínima do problema é a melhor complexidade que um algoritmo que resolve esse problema pode atingir. O problema do cálculo da complexidade, nesse caso, é dito **fechado**.

Problema de Classificação

- O **problema de classificação** por comparação é um problema com limite de complexidade determinada, **$n \log n$** .
- Não existe algoritmo de classificação baseado em comparações de complexidade melhor que $n \log n$.
- Consideremos três elementos a , b , c , que desejamos classificar.
 - o A única informação disponível a respeito é uma relação de ordem ($<$) entre os elementos.
 - o Nesse caso, os únicos algoritmos possíveis são baseados em comparação.
 - o A árvore a seguir é um exemplo de árvore de decisão para o problema.

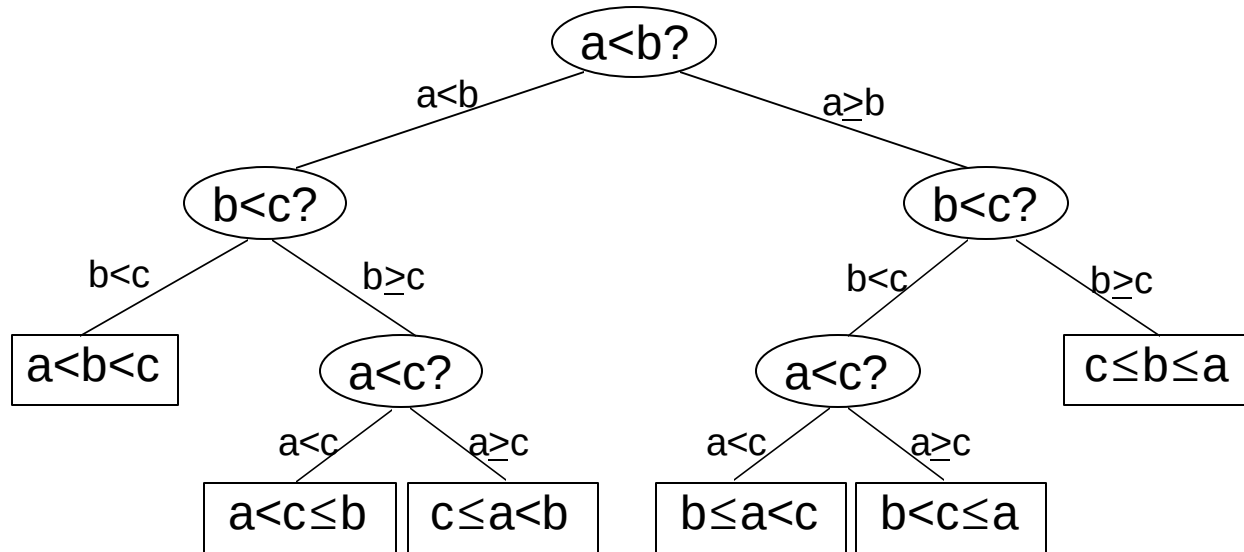


Figura 6.1.1: Árvore de comparações

- Qualquer **algoritmo de classificação** baseado em comparação pode ser representado por uma **árvore de decisão**, cujas folhas são as possíveis classificações dos elementos, isto é, todas as $n!$ permutações possíveis, onde n é o número de elementos.
- O número de comparações necessárias para classificar n elementos é a profundidade, na árvore de decisão, da folha que ilustra o caso. A complexidade no pior caso é a maior profundidade de folha na referida árvore.

Complexidade no Pior Caso

- Árvore binária completa possui relação: folhas = 2^{altura}
- No problema em questão:
 - o **folhas**: são $n!$ permutações possíveis;
 - o **altura**: no pior caso, $c(n)$
- Neste caso a relação passa a ser:

$$n! \leq 2^{c(n)}$$

$$\log_2 n! \leq \log_2 2^{c(n)}$$

$$c(n) \geq \lceil \log_2 n! \rceil$$

- Usando, a aproximação de Stirling

$$[\log_2 n!] = n \log_2 n - \frac{n}{(\ln 2)} + \frac{1}{2} \log_2 n + O(1)$$

como, $\ln 2 = \log_e 2$ é constante e $n \log_2 n$ absorve os outros termos, então:

$[\log_2 n!]$ é $O(n \log_2 n)$, e $c(n)$ é $O(n \log_2 n)$

- Para qualquer **problema de classificação** onde a única informação dos elementos a serem classificados é a **relação de ordem** entre eles, o **limite de complexidade no pior caso** é $n \log_2 n$, ou seja, **$n \log n$** .

Complexidade Média

- No caso médio com distribuição uniforme, mostra-se um resultado análogo.
- Suponha que as $n!$ entradas têm a mesma probabilidade: $\frac{1}{n!}$.
- A cada uma das $n!$ permutações corresponde uma folha na árvore de decisão, e qualquer um dos caminhos da raiz a uma folha é igualmente provável de ser percorrido.
- O número médio de comparações é a profundidade média das $n!$ folhas.
- Seja $S(A, k)$ a soma das profundidades de todas as folhas da árvore de decisão A , com k folhas.
- Mínimo da soma das profundidades: $m(k) = \min_A \{ S(A, k) \}$
- Profundidade média: $\frac{m(k)}{k}$

- Prova-se, por indução no número de nodos da árvore, que esse limite é $\log k$, ou $m(k) \geq k \log k$.

o **BI**: Para um nodo, a árvore é um nodo isolado, e $m(k) = 0$.

Por outro lado, $\log k = \log 1 = 0$, e $k \log k = 0$

o **HI**: Suponha $m(k') \geq k' \log k'$ para todo $k < k'$.

o **PI**: Seja uma árvore de decisão A . Sejam A_1 e A_2 as duas subárvores de A , imediatamente abaixo da raiz, com, respectivamente, k_1 e k_2 folhas, tal que $k_1 + k_2 = k$. A_1 e A_2 são árvores de decisão com menos nodos, portanto vale a hipótese de indução. A profundidade de um nodo qualquer nas subárvores A_1 e A_2 é uma unidade menor que a profundidade desse mesmo nodo em A , i.e,

$$S(A, k) = k_1 + S(A_1, k_1) + k_2 + S(A_2, k_2), \text{ e}$$

$$m(k) = \min_{k_1+k_2=k} \{k + m(k_1) + m(k_2)\}$$

$$= k + \min_{k_1+k_2=k} \{m(k_1) + m(k_2)\}$$

o **Hipótese de Indução**, $m(k_1) \geq k_1 \log k_1$ e $m(k_2) \geq k_2 \log k_2$, então

$$m(k) \leq k + \min_{k_1+k_2=k} \{ k_1 \log k_1 + k_2 \log k_2 \}$$

o Para resolver

$$\min_{k_1+k_2=k} \{ k_1 \log k_1 + k_2 \log k_2 \}$$

substitua k_1 por x e k_2 por $k-x$, e defina

$$f(x) = x \log x + (k-x) \cdot \log (k-x).$$

o Derivando tem-se:

$$f'(x) = \log x + \frac{x}{x \ln 2} + (-1) \log(k-x) + (k-x) \frac{-1}{(k-x) \ln 2}$$

$$= \log x + \frac{1}{\ln 2} - \log(k-x) - \frac{1}{\ln 2} \quad (\ln 2 = \log_e 2)$$

$$= \log x - \log(k-x)$$

$$= \log\left(\frac{x}{k-x}\right), \quad \text{mas}$$

$$\log\left(\frac{x}{k-x}\right)=0 \Leftrightarrow \frac{x}{k-x}=1 \Rightarrow x = k-x \Rightarrow 2x = k \Rightarrow x = \frac{k}{2} \quad (\text{pois, } x \geq 0)$$

o Derivando novamente,

$$\begin{aligned} f''(x) &= \frac{1}{\left(\frac{x \cdot \ln 2}{k-x}\right)} \left(\frac{(k-x) - (-1)x}{(k-x)^2} \right) \\ &= \frac{k}{x(k-x)\ln 2} \end{aligned}$$

$$f''\left(\frac{k}{2}\right) = \frac{2}{k \ln 2} > 0 \quad \text{para } \forall x \in (0, k), \text{ então a função tem mínimo no ponto } \frac{k}{2}.$$

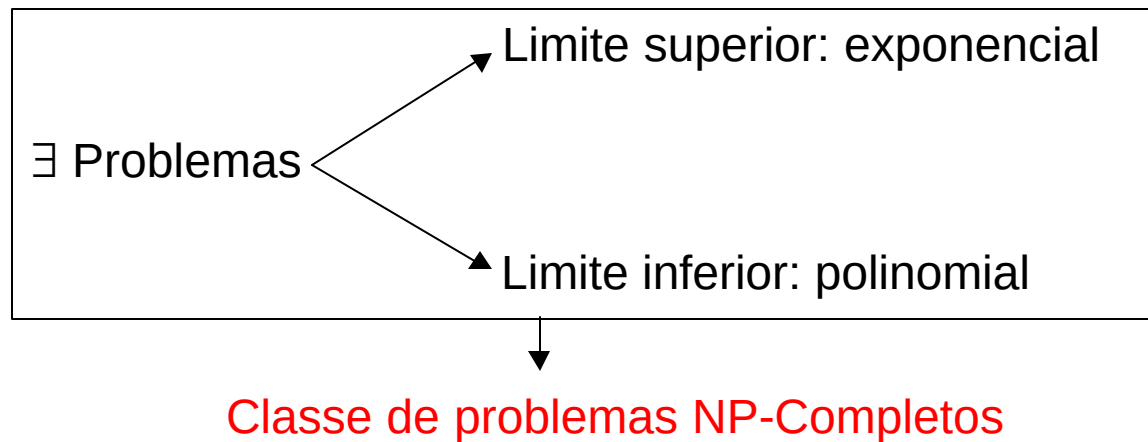
o Conclui-se que:

$$\begin{aligned}m(k) &\geq k + \frac{k}{2} \cdot \log \frac{k}{2} + \frac{k}{2} \cdot \log \frac{k}{2} \\&\geq k + k \log \frac{k}{2} \\&\geq k (\log k - \log 2 + 1) \\&\geq k \log k\end{aligned}$$

- Como a árvore tem $n!$ folhas, então $m(n!) \geq n! \cdot \log n!$, e a profundidade média mínima é $\frac{m(n!)}{n!} \geq \log n!$, que é, pela aproximação de Stirling, $O(n \log n)$.

6.2 INTRATABILIDADE

- Problema tratável: existe algoritmo eficiente, ou seja, **limite superior é polinomial**.
- Problema intratável: não tem algoritmo eficiente, ou seja, **limite inferior é exponencial**.
- Problemas que não satisfazem nenhuma das duas condições:



- Problemas NP-completos:
 - o Não se sabe se existe ou não algoritmo polinomial que o resolva;

- o Achar um algoritmo polinomial que resolva um problema NP-completo implica que todos os outros também possuem um algoritmo polinomial;
- o Acredita-se que não existe um algoritmo polinomial para os NP-completos.

6.2.1 Conceitos Básicos

⇒ **O que precisamos saber de um problema?**

- (i) Entradas (parâmetros)
- (ii) Quais propriedades a resposta ou solução deve satisfazer?
(Propriedades da solução)

⇒ **Tipos de problema:**

o Problema de decisão

- Se a pergunta tem **resposta SIM/NÃO**.
- Problema 6.2.1: *Problema da existência de Cobertura de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$ e um natural k .
 - o **Pergunta**: **Existe V'** (subconjunto de V) tal que $|V'| \leq k$ e toda aresta de G tenha pelo menos uma extremidade em V' ?

o Problema de localização

- Exige a exibição de uma solução, envolvendo a procura de **um elemento que satisfaça a condição de solução**.
- Problema 6.2.2: *Problema da determinação de Cobertura de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$ e um natural k .
 - o **Procura**: **Deseja-se V'** (subconjunto de V) tal que $|V'| \leq k$ e toda aresta de G tenha pelo menos uma extremidade em V' .

o Problema de otimização:

- Existe elemento que satisfaz a condição de solução e é um “**ótimo**”.
 - Problema 6.2.3: *Problema da Cobertura Mínima de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$.
 - o **Procura**: **Deseja-se V'** (subconjunto de V) tal que toda aresta de G tenha pelo menos uma extremidade em V' e **não exista V''** (subconjunto de V) com mesma propriedade com $|V''| < |V'|$.
- Qualquer problema de localização tem um correspondente de decisão.
 - Qualquer problema de otimização tem um correspondente problema de decisão.

Problema 6.2.4: *Problema do Caixeiro Viajante (PCV)*

- o **Instância:** Um grafo $G = (\{c_1, c_2, \dots, c_n\}, E)$ e uma função d tal que, se $(c_i, c_j) \in E$, então $d(c_i, c_j) \in \mathbb{Z}^+$ é a distância entre as cidades c_i e c_j .
- o **Ótimo:** Deseja-se c , um caminho fechado em G com custo mínimo.
- Problema de otimização, cuja versão de decisão, quando todas as arestas têm o mesmo custo, é o problema do Circuito Hamiltoniano.

Problema 6.2.5: *Problema da existência de Circuito Hamiltoniano*

- o **Instância:** Um grafo $G = (V, E)$.
- o **Pergunta:** G tem circuito hamiltoniano?

Problema 6.2.6: *Problema de decisão associado ao problema do Caixeiro Viajante*

- o **Instância:** Um grafo $G = (V, E)$, com arestas valoradas e um inteiro positivo k .
- o **Pergunta:** G tem circuito hamiltoniano de custo menor ou igual a k ?
- Um algoritmo que resolve um problema de localização ou de otimização pode ser modificado para resolver um problema de decisão associado.
- As classes de problemas P , NP , NP -Completa, são definidas para problemas de decisão (por serem mais simples).
- Os requisitos de tempo de um algoritmo são expressos em termos do “tamanho da entrada” porque se espera que a dificuldade do problema varie rigorosamente com essa medida.
- **Tamanho da entrada** de uma instância I de um problema Π é definido como o número de símbolos na descrição de I obtido em um esquema de codificação do problema Π .

- Um **algoritmo** é *polinomial* se é $O(p(n))$, p um polinômio.
- Se um **algoritmo não é polinomial**, diz-se que **é exponencial**.
- Um problema de decisão é dito:
 - o **P**: resolvido **deterministicamente** em tempo polinomial.
 - o **NP**: resolvido **não deterministicamente** em tempo polinomial.

6.2.2 Algoritmos não determinísticos

- Aos comandos usuais das linguagens de programação, acrescenta-se três comandos que caracterizam o não determinismo, que são:
 - o escolhe(S): função que escolhe arbitrariamente um elemento de S.
 - o falha: sinal de terminação falha.
 - o sucesso: sinal de terminação com sucesso.
- Um algoritmo **não determinístico**:
 - o faz uso da função escolhe(S) e das instruções falha e sucesso;
 - o **termina com insucesso**, sss, não existe escolha que leve a um terminação com sucesso;
 - o tem **vários caminhos de execução**, podendo inclusive mais de um terminar em sucesso;
 - o são interessantes na solução de problemas em que **existe várias alternativas** para serem testadas.

- Exemplos de algoritmos não determinísticos:

o *Algoritmo 6.2.1: Procura*

Algoritmo de procura de um elemento x em $A(1:n)$, $n \geq 1$.

Dado x , A , procura em A o elemento x ; devolve j tal que $A(j) = x$ ou $j = 0$ se x não está em A .

Procedimento Procura(A : vetor, x : elemento)

{*Entrada*: A : vetor, x : elemento *Saída*: j ou 0 : valor}

1. $j \leftarrow$ escolhe($1:n$) {escolhe posição}
2. se $A(j) = x$ {testa se é o elemento procurado}
3. então imprime(j); sucesso {fim com sucesso}
4. senão imprime("0"); falha {fim com falha}
5. fim-se {2: $A(j) = x$?}
6. fim-Procedimento {fim Procura}

→ A complexidade do algoritmo é $O(1)$.

o Algoritmo 6.2.2: Nclassifica

Classifica os elementos do vetor $A(i)$ $1 \leq i \leq n$. A contém inteiros positivos. $Nclassifica(A, n)$ coloca os elementos de A em ordem não crescente e imprime-os nessa ordem. É usado um vetor auxiliar B .

Procedimento $Nclassifica(A: \text{vetor}, n: \text{Nat})$

{*Entrada*: vetor: $[1..n]$ (a ordenar A); *Nat*: n
Saída: vetor: B (vetor A classificado)}

1. $B \leftarrow 0$	{inicializa B com zeros}
2. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u>	{preenche B : 2 a 6}
3. $j \leftarrow \text{escolhe}(1:n)$	{escolhe posição para atribuir $A(i)$ }
4. <u>se</u> $B(j) \neq 0$ <u>então</u> <u>falha</u> <u>fim-se</u>	{verifica se a posição está preenchida}
5. $B(j) \leftarrow A(i)$	{atribui $A(i)$ }
6. <u>fim-para</u>	{2: B preenchido}
7. <u>para</u> i <u>de</u> 1 <u>até</u> $n-1$ <u>faça</u>	{verifica se B está classificado}
8. <u>se</u> $B(i) > B(i+1)$ <u>então</u> <u>falha</u> <u>fim-se</u>	{falha se B não está classificado}
9. <u>fim-para</u>	{7: B está classificado}
10. <u>retorne-saída</u> (B)	{dá saída a B }
11. <u>sucesso</u>	{termina com sucesso}
12. <u>fim-Procedimento</u>	{fim $Nclassifica$ }

→ A complexidade do algoritmo é $O(n)$.

o Algoritmo 6.2.3: Cobertura de Vértices

Algoritmo para o Problema da Cobertura de Vértices (problema 6.4.2).

Dado $G = (V, E)$ e um inteiro positivo k , devolve ou $\emptyset$ ou V' , um subconjunto de V tal que $|V'| \leq k$ e toda aresta de G tem pelo menos uma extremidade em V' .

Procedimento Cobertura-de-Vértice (V, E : Conjuntos, k : valor)

{Entrada: Conjunto: V, E : k

Saída: Conjunto: V' ou $\emptyset$ }

1. $V' \leftarrow \emptyset$ {inicializa cobertura}
2. $j \leftarrow \text{escolhe}(1..k)$ {escolhe j , cardinalidade de cobertura}
3. para i de 1 até j faça {escolhe j vértices para a cobertura: de 3 a 7}
4. $v \leftarrow \text{escolhe}(V)$ {escolhe um vértice}
5. $V \leftarrow V - \{v\}$ {retira o vértice escolhido do conjunto de opções}
6. $V' \leftarrow V' \cup \{v\}$ {coloca o vértice escolhido na cobertura}
7. fim-para {3: cobertura completa}
8. para $(v, w) \in E$ faça {verifica se V' é realmente cobertura: de 8 a 10}
9. se $v \notin V'$ e $w \notin V'$ então $V' \leftarrow \emptyset$; falha fim-se { V' não cobertura, pára c/ falha}
10. fim-para {8: certificação da cobertura V' }
11. sucesso {fim com sucesso}
12. retorne saída(V') {dá saída à cobertura V' }
13. fim-Procedimento {fim Cobertura-de-Vértices}

6.2.3 Formalização do conceito de algoritmo

- Um **algoritmo** é um programa para uma Máquina de Turing Determinística (MTD) que sempre pára.

Máquina de Turing determinística (MTD)

- $MTD = (Q, \Gamma, \Sigma, \delta)$
 - o **Conjunto de estados finitos** $(Q) = \{q_o, q_s, q_n\}$
 - o **Alfabeto finito da fita** $(\Gamma) = \Sigma \subseteq \Gamma$, o alfabeto finito de entrada; $b \in \Gamma - \Sigma$, símbolo branco.
 - o Função de transição $\delta: (Q - \{q_s, q_n\}) \times \Gamma \rightarrow Q \times (\Gamma - \{b\}) \times \{1, -1\}$

MTD

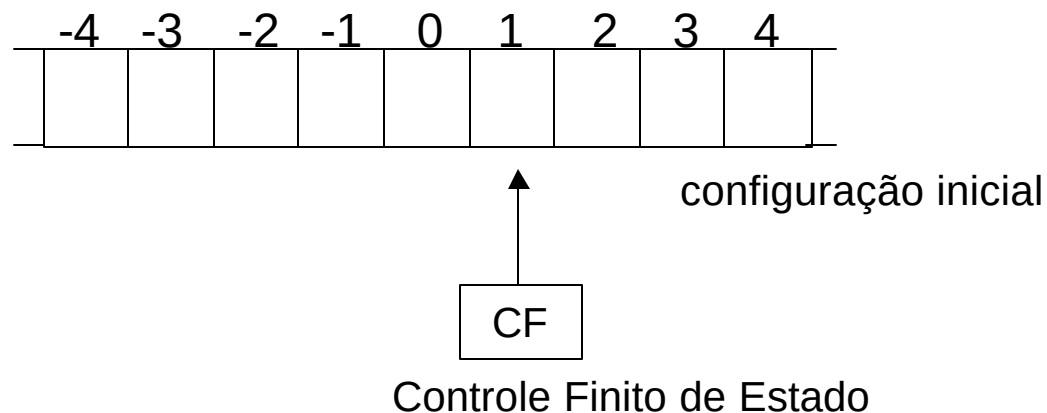


Figura 6.2.1: Máquina de Turing Determinística

- Uma MTD (ou um programa) M **aceita** $x \in \Sigma^*$ sse M pára no estado q_s , iniciando com x , e a **linguagem reconhecida** é $L_M = \{x \in \Sigma^* / M \text{ aceita } x\}$.
- O **tempo de computação** de M para x é o número de trocas de estados, até a máquina aceitar x .
- A complexidade de uma máquina M é definida pela função:

$$o \ T_M: \mathbb{N} \rightarrow \mathbb{N}$$

- o $T_M(n) = \text{Máx} \{m / \exists x \in \Sigma^*, \text{ com } |x|=n, \text{ tal que o tempo de computação de } M \text{ para } x \text{ é } m\}$
- o **M é de tempo polinomial** se existe um polinômio p tal que, para $\forall n \in \mathbb{N}$, $T_M(n)$ é $O(p(n))$.
- o **Classe P**
 $P = \{L / \text{existe um MTD } M \text{ de tempo polinomial t.q. } L = L_M\}$

Máquina de Turing não determinística (MTND)

- Um programa para uma MTND é definido exatamente como um programa para uma MTD, diferindo somente na execução.
- Uma MTND M tem dois cabeçotes, CF e MA.
- A execução da MTND:
 - o o cabeçote do MA pode escrever um símbolo de Γ e mover-se para a esquerda, ou

o ativar o cabeçote do CF, no estado inicial q_0 .

MTND

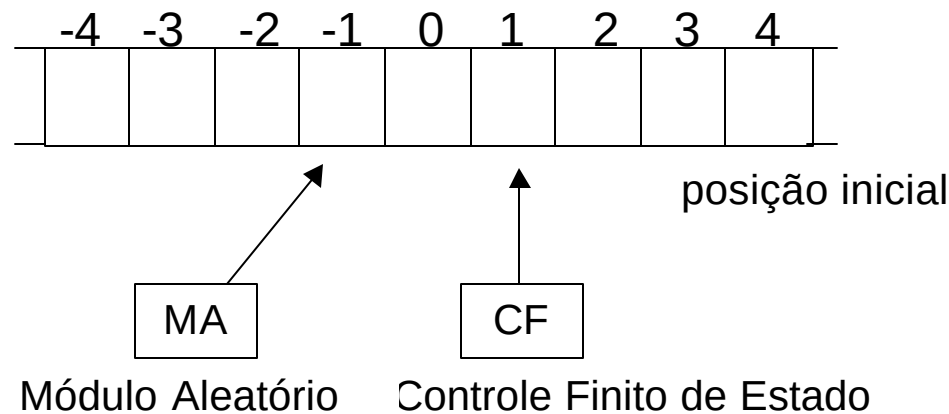


Figura 6.2.2: Máquina de Turing não determinística

- Um MTND M **aceita** x se pelo menos uma entre todas possíveis computações de M pára ao atingir um estado de aceitação, e a **linguagem reconhecida** é $L = \{ x \in \Sigma^* / M \text{ aceita } x \}$.
- O **tempo requerido pela computação** de M para a entrada x é o número mínimo de passos entre todas as computações que aceitam x .

- A complexidade de uma máquina M
 - o $T_M(n) = \text{Máx} \{ \{1\} \cup \{ m / \exists x \in L_M, \text{ com } |x| = n \text{ t.q. } m \text{ é o tempo de computação para } M \text{ aceitar } x \}$
 - o **Classe NP**

$$\text{NP} = \{ L / \exists M \text{ MTND de tempo polinomial t.q. } L_M = L \}.$$
- A MTND tem a **mesma expressividade** da MTD.
- A **complexidade** das máquinas, entretanto, **pode ser diferente**, pois as MTND costumam ser mais rápidas.
- Para um mesmo problema de decisão resolvido em:
 - o **MTND** $\rightarrow$ tempo polinomial.
 - o **MTD** $\rightarrow$ tempo exponencial.
- Está em aberto a questão da existência de MTD de tempo polinomial.

Problema 6.2.7: Problema da “Satisfiability” (SAT)

- o **Instância**: U conjunto de variáveis; C coleção de cláusulas sobre U.
- o **Pergunta**: Existe uma atribuição satisfaz C?
- A **certificação** de um problema de decisão é o teste de uma candidata a solução I é a verificação da candidata como solução do problema proposto.
- A Certificação é certamente um processo bem mais rápido que a solução do problema, que pode requerer um número não polinomial de certificações.
- **P** classe de problemas que podem ser **resolvidos eficientemente**.
- **NP** classe de problemas que podem ser certificados **eficientemente**.
- Os problemas de NP possuem algoritmos determinísticos de ordem exponencial, como é mostrado no teorema 6.2.1.

Teorema 6.2.1: Se $\Pi \in \text{NP}$, então existe um polinômio p tal que Π pode ser resolvido por algoritmo determinístico de complexidade $O(2^{p(n)})$.

6.2.4 Relação entre P e NP

- A relação entre P e NP é um **problema em aberto** na Ciência da Computação.
- Certamente $P \subseteq NP$, mas não se sabe se $P = NP$. Existe uma forte conjectura de que $P \neq NP$.
- Dado $\Pi \in NP$, se $NP - P \neq \emptyset$ há duas possibilidades para Π :
 - o $\Pi \in P$
 - o $\Pi \in NP - P$
- Mas $\Pi \in NP - P$ implica que $NP - P \neq \emptyset$. Considere a afirmação conversas:

Se $NP - P \neq \emptyset$ então existe Π , $\Pi \in NP - P$.

- Essa afirmação motivou a criação da classe dos problemas **mais difíceis de NP**, a classe dos problemas **NP-Completo**.

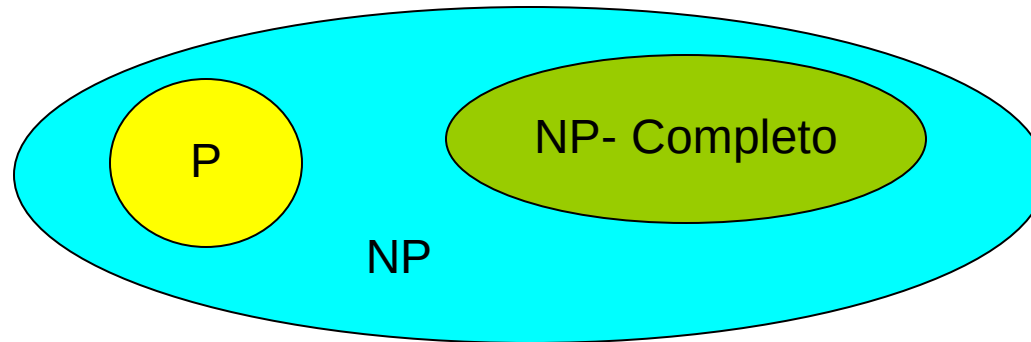


Figura 6.2.3: P NP

NP-Compleitude

- Se um problema NP-Completo tiver solução determinística polinomial, qualquer outro problema NP também terá.

- Redução polinomial (∞):

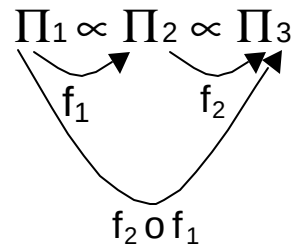
o de uma linguagem $L_1 \subseteq \Sigma_1^*$ para uma linguagem $L_2 \subseteq \Sigma_2^*$ é uma função $f : \Sigma_1^* \rightarrow \Sigma_2^*$ que satisfaz as duas condições seguintes:

(i) f é polinomial.

(ii) para todo $x \in \Sigma_1^*$, $x \in L_1$ sss $f(x) \in L_2$.

- Se existe uma redução polinomial de L_1 para L_2 , escreve-se $L_1 \infty L_2$, e diz-se que L_1 **é redutível a** L_2 .
- Se existe uma redução polinomial de um problema Π_1 para um problema Π_2 , escreve-se $\Pi_1 \infty \Pi_2$ e diz-se que Π_1 **é redutível a** Π_2 .

Lema 6.2.1: ∞ é transitiva.



Lema 6.2.2: Seja os problemas Π_1, Π_2 tal que se $\Pi_1 \infty \Pi_2$ então

- (i) Se $\Pi_2 \in P$ então $\Pi_1 \in P$.
- (ii) Se $\Pi_2 \in NP$ então $\Pi_1 \in NP$.
- Sejam Π_1 e Π_2 dois problemas. Π_1 e Π_2 são **polinomialmente equivalentes** sss $\Pi_1 \infty \Pi_2$ e $\Pi_2 \infty \Pi_1$.
- Um problema Π é dito NP-Completo se $\Pi \in NP$ e, para qualquer outro $\Pi' \in NP$, $\Pi' \infty \Pi$.
- Os problemas NP-Completos são os **mais difíceis** entre os problemas NP.

Lema 6.2.3: Se um o problema Π é NP-completo, então $\Pi \in P$ sss $P = NP$.

- O grande número de problemas NP-Completo e o fato de ninguém ter conseguido criar um algoritmo polinomial para um deles torna mais forte a conjectura de que $P \neq NP$.
- Outras questões referentes a essas classes de problemas também estão em aberto, por exemplo:

o A classe NP é fechada para o complemento? Isto é, se a linguagem L sobre o alfabeto Σ é NP, será que $L' = \Sigma^* - L$ é NP?

Classe Co-NP

- Classe de todas as linguagens L tais que $L' \in NP$.
- A questão é reescrita como: **NP = Co-NP?**
- Como P é fechada sob complemento, então $P \subseteq NP \cap \text{Co-NP}$. E a questão fica: $P = NP \cap \text{Co-NP}$? ou $(NP \cap \text{Co-NP}) - P \neq \emptyset$?

- Existem quatro relações possíveis entre essas classes:

o $P = NP$ e, portanto, $NP = Co-NP$

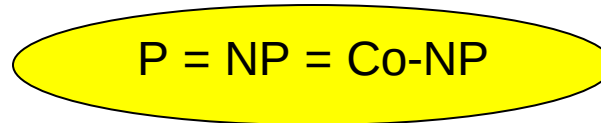


Figura 6.2.4: $P = NP$

o Se $P \neq NP$ então há três possibilidades:

- $NP = Co-NP$, mas $P \neq NP$



Figura 6.2.5: $P \neq NP = Co-NP$

- $NP \neq Co-NP$ e $P = NP \cap Co-NP$

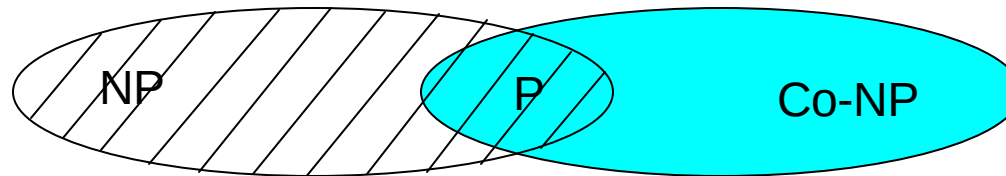


Figura 6.2.6: $NP \cap Co-NP = P \neq NP \neq Co-NP$

- $NP \neq Co-NP$ e $P \neq NP \cap Co-NP$

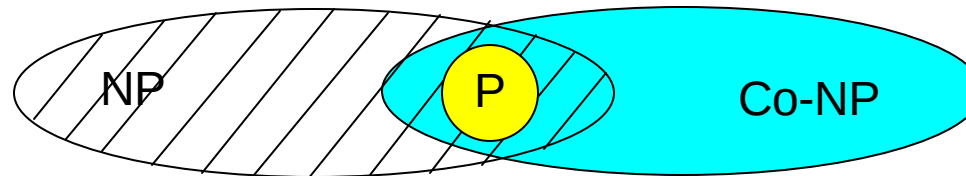


Figura 6.2.7: $P \neq NP \neq Co-NP \neq NP \cap Co-NP$

→ Problema: “x é primo?”.

→ Problema de Programação Linear

- Se $\Pi \in \text{NP-Completo}$, então não se conhece solução eficiente para o problema.

Teorema de Cook

- Os teoremas a seguir apresentam **uma maneira de identificar** um problema NP-Completo.

Teorema 6.2.2: Se Π_1 é NP-Completo, $\Pi_2 \in \text{NP}$ e $\Pi_1 \propto \Pi_2$, então Π_2 é NP-Completo.

- *Demonstração:* Sejam $\Pi_1, \Pi_2 \in \text{NP}$ tal que: Π_1 é NP-Completo e $\Pi_1 \propto \Pi_2$. Seja $\Pi \in \text{NP}$, como Π_1 é NP-Completo, $\Pi \propto \Pi_1$. Mas $\Pi_1 \propto \Pi_2$, e como $\propto$ é transitivo, $\Pi \propto \Pi_2$, logo Π_2 é NP-Completo.

Teorema 6.2.3 - Teorema de Cook: O problema da "Satisfiability" (SAT) é NP-Completo.

- *Demonstração:* É preciso mostrar que:

(i) SAT é NP (trivial)

(ii) $\forall \Pi \in \text{NP}, \Pi \leq \text{SAT}$

o Para mostrar (ii), considera-se a linguagem L_{SAT} e mostra-se que para $\forall L \in \text{NP}, L \leq L_{\text{SAT}}$.

o $L \in \text{NP}$ significa que **L é reconhecida por uma MTND** $M=(Q, \Gamma, \Sigma, \delta)$, tal que $L(M) = L$.

o Sejam:

- $Q = \{q_0, q_1, q_2, q_3, \dots, q_r\}$, com $r = |Q| - 1$; $q_1 = q_S$ e $q_2 = q_N$
- $\Gamma = \{s_0, s_1, s_1, \dots, s_v\}$, onde $v = |\Gamma| - 1$; $s_0 = b$

o função $f_L = \Sigma^* \rightarrow \Sigma_{\text{SAT}}^*$, com a seguinte propriedade:

- $\forall x \in \Sigma^*, x \in L \text{ sss } f_L(x) \text{ tem uma atribuição verdadeira que a satisfaz.}$
- o Se $x \in \Sigma^*$ é aceito por M , então existe uma computação de M que aceita x em, no máximo, $p(n)$ passos, onde $n = |x|$ e p é um polinômio.
- o A partir da computação de x , será construído $f_L(x)$: um conjunto de variáveis e de cláusulas, uma instância de SAT com resposta SIM.
- o $f_L(x)$ conterá um conjunto de cláusulas que simularão a computação de x em M .
- o A construção de f_L garantirá que o conjunto de cláusulas de $f_L(x)$ terá uma atribuição verdadeira sss M aceitar x .

o Variáveis do conjunto U:

- $Q[i, k]$: com $0 \leq i \leq p(n)$ e $0 \leq k \leq r$, significando que, ao completar o i° passo, M estará no estado q_k .
- $H[i, j]$: com $0 \leq i \leq p(n)$ e $-p(n) \leq j \leq p(n)+1$, significando que, ao completar o i° passo, o cabeçote CF estará apontando para a posição j da fita.
- $S[i, j, k]$: com $0 \leq i \leq p(n)$, $-p(n) \leq j \leq p(n)+1$ e $0 \leq k \leq v$, significando que ao completar o i° passo o conteúdo da posição j da fita será o símbolo s_k .
- $|U| = (p(n)+1)(r+1) + (p(n)+1)(2p(n)+2) + (p(n)+1)(2p(n)+2)(v+1) = O(p(n))^2$.

o Conjunto C de cláusulas de $f_L(x)$

Grupo 1:

→ $\{Q[i, 0], Q[i, 1], \dots, Q[i, r]\}$, $0 \leq i \leq p(n)$
garantem que **M estará em um estado**, ao completar o i^o passo.

→ $\{\neg Q[i, j], \neg Q[i, j']\}$, $0 \leq i \leq p(n)$ e $0 \leq j \leq j' \leq r$
garantem que **M não estará em mais de um estado** ao completar o i^o passo.

Grupo 2:

→ $\{H[i, -p(n)], H[i, -p(n)+1], \dots, H[i, p(n)+1]\}$, $0 \leq i \leq p(n)$

→ $\{\neg H[i, j], \neg H[i, j']\}$, $0 \leq i \leq p(n)$, $-p(n) \leq j \leq j' \leq v$

Ao completar o i^o passo, o **cabeçote CF estará apontando para exatamente uma posição**.

Grupo 3:

$\rightarrow \{S[i, j, 0], S[i, j, 1], \dots, S[i, j, v]\}, 0 \leq i \leq p(n) \text{ e } -p(n) \leq j \leq p(n)+1$

$\rightarrow \{\neg S[i, j, k], \neg S[i, j, k']\}, 0 \leq i \leq p(n), -p(n) \leq j \leq p(n)+1 \text{ e } 0 \leq k \leq k' \leq v$

Ao completar o i^o passo, **cada posição da fita conterá exatamente um símbolo** de Γ .

Grupo 4:

$\rightarrow \{Q[0, 0]\}, \{H[0, 1]\}, \{S[0, 0, 0]\}$

$\rightarrow \{S[0, 1, k]\}, \{S[0, 2, k]\}, \dots, \{S[0, n, k]\}$

$\rightarrow \{S[0, n+1, 0]\}, \{S[0, n+2, 0]\}, \dots, \{S[0, p(n), 0]\}$, onde $x = s_{k1} s_{k2} \dots s_{kn}$

- Antes do primeiro passo, a computação estará na configuração inicial: M estará no estado q_0 ; o cabeçote apontando para a posição 1; na posição zero deverá ter branco; nas posições de 1 a n estará a cadeia x e da posição n+1 para a direita estará branco.

Grupo 5:

$\rightarrow \{Q[p(n), 1] \}$

No passo $p(n)$, M entrará no estado q_s e aceitará x .

Grupo 6:

Para cada quádrupla (i, j, k, e) , $0 \leq i \leq p(n)$, $-p(n) \leq j \leq p(n)+1$, $0 \leq k \leq r$ e $0 \leq e \leq v$ esse grupo contém as seguintes 3 cláusulas:

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], H[i+1, j+\Delta]\}$$

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], Q[i+1, k']\}$$

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], S[i+1, j, e']\}$$

com Δ , k' , e' assim definidos:

- Se $q_k \in Q - \{q_s, q_N\}$ então $(q_{k'}, s_{e'}, \Delta) = \delta(q_k, s_e)$,

- Se $q_k \in \{q_s, q_N\}$ então $\Delta = 0$, $k' = k$, $e' = e$.

A cada passo i , $0 \leq i \leq p(n)$, a configuração de M para o passo $i+1$ seguirá da aplicação da função de transição δ à configuração do passo i . Assim, fica definido o conjunto C de cláusulas geradas pela entrada x , portanto está definida a função f_L .

o $|f_L(x)| = |U| + |C|$, é $O((p(n))^4)$

$|U|$ é $O((p(n))^2)$

$|C|$ é $O((p(n))^2)$

o Conclui-se que, para toda $L \in NP$, é possível construir uma redução polinomial f_L de L em L_{SAT} , tal que para $x \in L$, sss $f_L(x)$ é verdadeira.

o A partir do SAT, mostra-se, por exemplo, que o 3SAT é NP-Completo.

Problema 6.2.8: *Problema 3SAT*

o **Instância:** Um conjunto de variáveis C ; uma coleção de cláusulas sobre U , todas com exatamente três literais.

o **Pergunta:** Existe uma atribuição verdadeira para C ?

- A partir do 3SAT, mostra-se, por exemplo, que o 3DM é NP-Completo.

Problema 6.2.9: *Problema 3DM*

- o **Instância:** Conjuntos X, Y, Z , com cardinalidade q e $M \subseteq X \times Y \times Z$.
- o **Pergunta:** Existe $M' \subseteq M$ tal que cada elemento de X, Y, Z apareça exatamente uma vez em M' ?
- A partir do 3DM, mostra-se, por exemplo, que o problema da partição é NP-Completo.

Problema 6.2.10: *Problema Partição*

- o **Instância:** Um conjunto A , $s:A \rightarrow \mathbb{Z}^*$, uma função que associa a cada $a \in A$ um inteiro positivo $s(a)$.
- o **Pergunta:** Existe $A' \subseteq A$ tal que $\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a)$.
- Cadeia de redução polinomial: **SAT $\propto$ 3SAT $\propto$ 3DM $\propto$ Partição**

Problemas de Verificação da Comunicação Síncrona de Processos (VCSP)

- Considere um sistema concorrente com comunicação síncrona, por caixa postal, no qual as caixas postais são utilizadas somente para fins de comunicação, isto é, não contêm mensagens.
- **Instruções:** S (envio de sinais) e R (recebimento de sinais).

Problema 6.2.11: *Problemas de Verificação da Comunicação Síncrona de Processos (VCSP)*

- o **Instância:** St, um sistema de processos concorrentes com comunicação síncrona através de caixa postal; X, caixa postal.
- o **Pergunta:** É possível ocorrer, em St, sincronização pela caixa X?
- Mostrar-se que o problema **VCSP é um problema NP-Completo**, através do teorema 6.2.2.
 - (i) VCSP é NP
 - (ii) SAT $\propto$ VCSP

- *Demonstração:*

- o A prova de (i) é trivial.

- o Para provar (ii) considere U e C como sendo uma instância de SAT.

- $U = \{u_1, u_2, \dots, u_n\}$ um conjunto de variáveis.

- $C = \{c_1, c_2, \dots, c_m\}$, um conjunto de cláusulas.

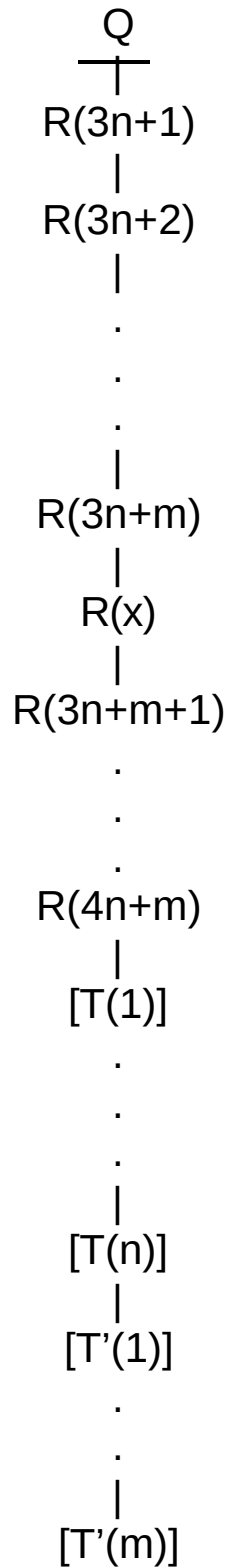
- o É preciso definir uma redução polinomial que, em tempo polinomial, descreva a partir de U e C uma instância St, X para VCSP tal que, se U tem uma interpretação que faz C verdadeira, então St tem uma possível sincronização pela caixa postal X e vice versa.

- o Os processos P e Q se comunicam pela caixa postal X.

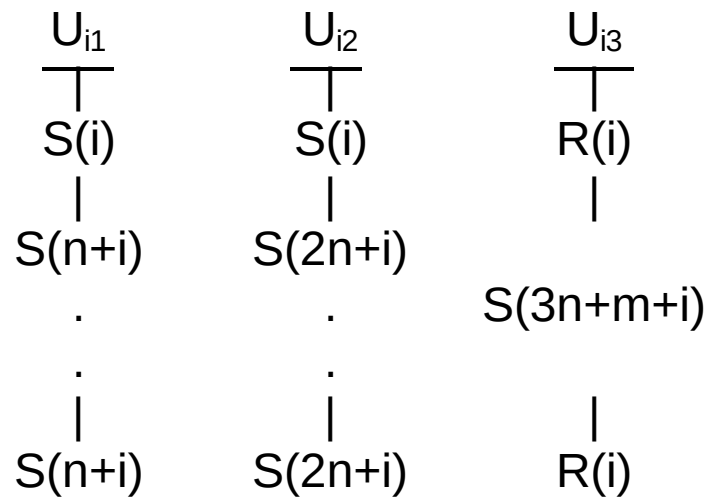
- o St será constituído por processos tipo U, tipo C, processo P e processo Q.

- o Para cada variável u_i de U , serão definidos 3 processos:
- U_{1i} , vai representar a atribuição “verdade” a u_i ;
 - U_{2i} , vai representar a atribuição “falso” para u_i ;
 - U_{3i} , vai representar a oportunidade de u_i ser verdadeira ou falsa.

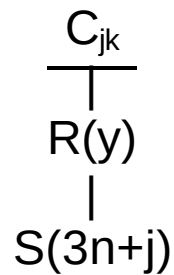
Processos Q



Processos tipo U



Processo tipo C



com $y = n+i$ se $a_{kj} = u_i$
 $y = 2n+i$ se $a_{kj} = \neg u_i$
 $j = 1, \dots, m$
 $k = 1, \dots, b_j$

Processo P

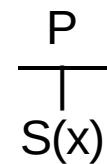


Figura 6.2.8: Problema VCSP

- o Conforme a sincronização na caixa postal i , será enviado sinal para a caixa postal $n+i$ ($u_i = \text{verdade}$) ou caixa $2n+i$ ($u_i = \text{falso}$).
- o O processo U_1 será constituído de $S(i)$ seguido de tantas ocorrências de $S(n+i)$ quantas forem as ocorrências de u_i nas cláusulas de C , ou uma ocorrência de $S(n+i)$ se u_i não ocorre nas cláusulas de C .
- o O processo U_{i2} será constituído de forma análoga: $S(i)$, seguido de tantas ocorrências de $S(2n+1)$ quantas forem as ocorrências de $\neg u_i$ nas cláusulas de C , ou uma ocorrência de $S(2n+i)$ se $\neg u_i$ não ocorre em C .
- o Para cada $c_j \in C$, $c_j = a_{1j}, a_{2j}, \dots, a_{bjj}$ são definidos b_j processos tipo C , cada um representando a possibilidade de o literal correspondente ser verdadeiro.
- o O processo Q inicia por m instruções R : $R(3n+1), R(3n+2), \dots, R(3n+m)$, onde $R(3n+j)$ representa a cláusula c_j verdadeira, depois vem $R(X)$. Assim, a execução do processo Q alcançará a instrução $R(X)$ se e somente se existir uma possibilidade de todas as cláusulas serem verdadeiras.

- o Após a instrução $R(X)$, as instruções R e S têm o objetivo de terminar a execução do sistema St . As instruções $R(3n+m+1)$, $\dots$, $R(4n+m)$ têm o objetivo de liberar a outra possibilidade de valor verdade de cada variável, cada instrução $R(3n+m+i)$ corresponde a uma instrução $S(3n+m+i)$ no processo U_{3i} .
- o $[T(i)]$ e $[T'(j)]$ são seqüências de instruções R e S , tais que:
 - $[T(i)]$ deve liberar todas as pendências com relação à variável u_i em processos tipo U .
 - haverá uma ocorrência de $R(n+i)$ se u_i não ocorrer em cláusulas de C ;
 - haverá uma ocorrência de $R(2n+i)$ se $\neg u_i$ não ocorrer em cláusulas de C .
 - $[T'(j)]$ deve liberar todas as pendências com relação a cláusulas c_j , em processos tipo C . É constituído de $b_j - 1$ ocorrências de $R(3n+j)$.

o Fazem um total de:

- Processos: $2 + 3n + \sum_{j=1}^m b_j \leq 2 + 3n + 2nm$

- Caixas postais: $4n + m$

- Instruções: $2 (5n + 2 + \sum_{j=1}^m b_j)$

o Existe uma atribuição verdade para as variáveis de U que tornam todas as cláusulas de C verdadeiras se e somente se o sistema correspondente St tiver possibilidade de sincronização na caixa X .

o Pelo número de processos, caixas postais e instruções de St , tem-se que a construção de St é possível em tempo polinomial.

Os exemplos 6.2.1 e 6.2.2 ilustram o procedimento.

Exemplo 6.2.1

$\frac{P}{S(x)}$

$\frac{Q}{R(10)}$
 $|$
 $R(11)$
 $|$
 $R(X)$
 $|$
 $R(12)$
 $|$
 $R(13)$
 $|$
 $R(14)$
 $|$
 $R(5)$
 $|$
 $R(9)$
 $|$
 $R(10)$
 $|$
 $R(10)$
 $|$
 $R(11)$

$\frac{U_{11}}{S(1)}$
 $|$
 $S(4)$

$\frac{U_{21}}{S(1)}$
 $|$
 $S(7)$

$\frac{U_{31}}{R(1)}$
 $|$
 $S(12)$
 $|$
 $R(1)$

$\frac{U_{12}}{S(2)}$
 $|$
 $S(5)$

$\frac{U_{22}}{S(2)}$
 $|$
 $S(8)$
 $|$
 $S(8)$

$\frac{U_{32}}{R(2)}$
 $|$
 $S(13)$
 $|$
 $R(2)$

$\frac{U_{13}}{S(3)}$
 $|$
 $S(6)$

$\frac{U_{23}}{S(3)}$
 $|$
 $S(9)$

$\frac{U_{33}}{R(3)}$
 $|$
 $S(14)$
 $|$
 $R(3)$

$\frac{C_{11}}{R(4)}$
 $|$
 $S(10)$

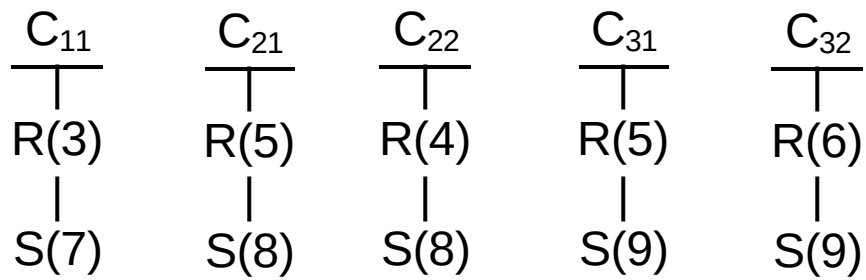
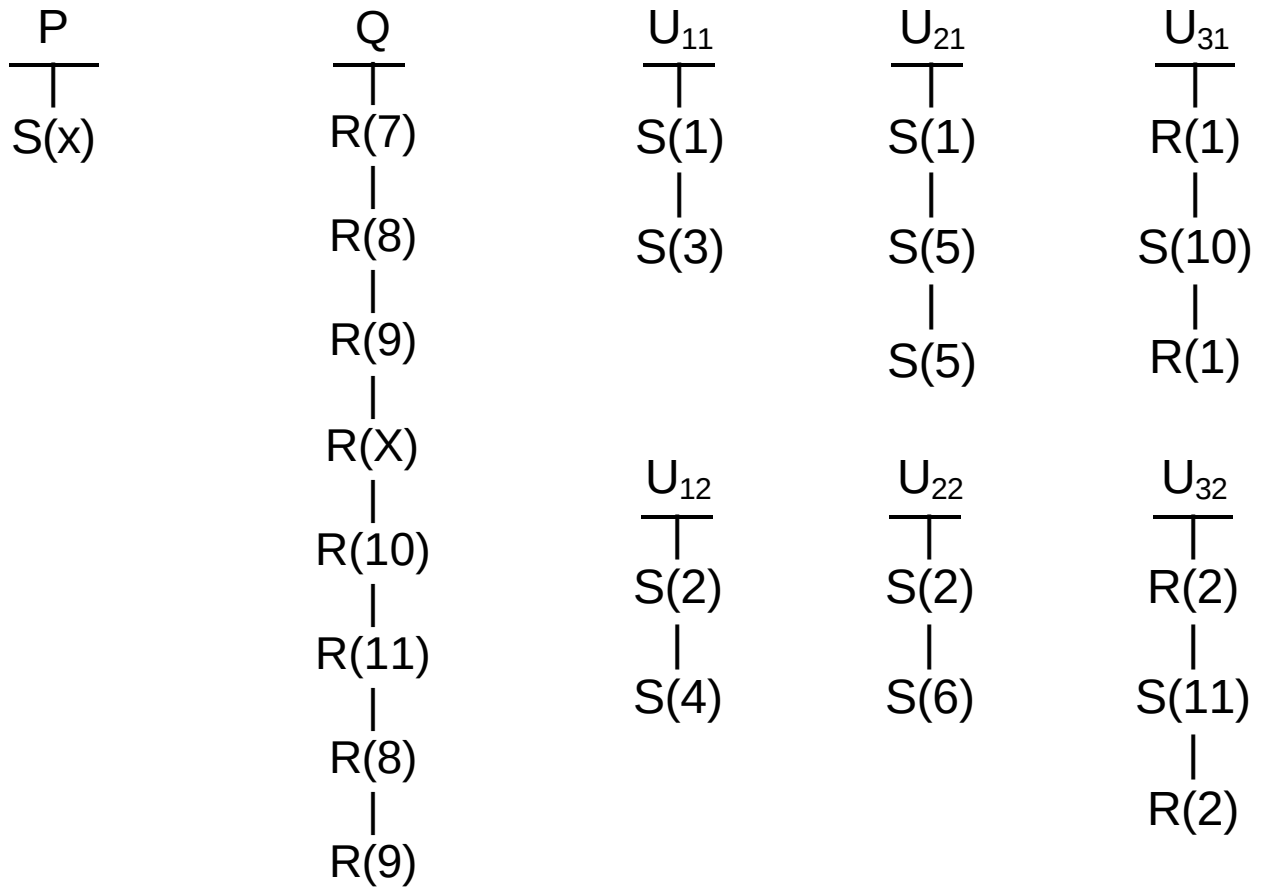
$\frac{C_{12}}{R(8)}$
 $|$
 $S(10)$

$\frac{C_{13}}{R(6)}$
 $|$
 $S(10)$

$\frac{C_{21}}{R(7)}$
 $|$
 $S(11)$

$\frac{C_{22}}{R(8)}$
 $|$
 $S(11)$

Exemplo 6.2.2



6.2.5 Algoritmos Pseudopolinomiais

- Um algoritmo A é dito de **complexidade pseudopolinomial** se sua complexidade é polinomial para o tamanho da entrada, quando essa é codificada em **unário**.
- Um algoritmo A é dito de **complexidade pseudopolinomial** quando a complexidade é polinomial no **valor da entrada** (e não no tamanho da entrada).
- **NP-Completo-fraco** - Problema NP-Completo que **admite algoritmo pseudopolinomial**.
- **NP-Completo-fortes** – Problema NP-Completo que não admitem algoritmo pseudopolinomial (exceto se $P \neq NP$).
 - o Caso exista um algoritmo pseudopolinomial, para eles, implica que $P=NP$. Ex.: *Problema do Circuito Hamiltoniano*.

6.2.6 Outras Classes de Problemas

- Existem problemas que estão numa classe “meio nebulosa”, como os de:
 - o decidir se duas expressões regulares são equivalentes;
 - o decidir se uma palavra é gerada por uma certa gramática sensível ao contexto.
- **Problemas NP-difíceis:**
 - o são problemas polinomialmente reduzíveis de problemas NP-Completos, entretanto não se conhece algoritmo não determinístico de tempo polinomial que os resolva.
 - o Em geral não se sabe se pertencem ou não a NP.
 - o se pertence a NP é NP-Completo.
 - o Um problema Π é **NP-Difícil** se, para qualquer $\Pi' \in \text{NP}$, $\Pi' \leq \Pi$.

- Os problemas NP-Difíceis têm a propriedade de **não serem resolvíveis deterministicamente em tempo polinomial exceto se $P=NP$** , mas não se sabe se são NP ou não.
- São potencialmente mais difíceis que os NP-Completos.

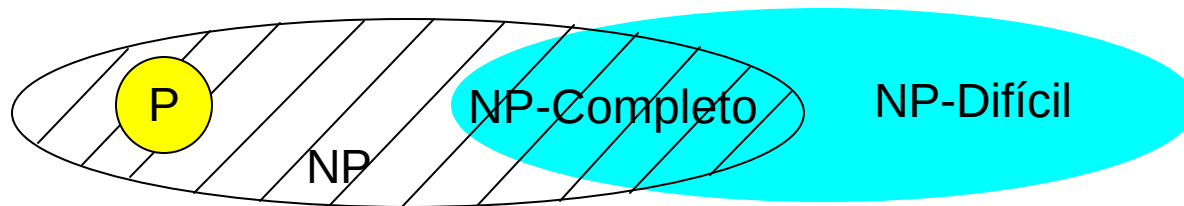


Figura 6.2.11: $NP\text{-}Completo = NP \cap NP\text{-}Difícil$

- Se $P \neq NP$, será que $NP\text{-}P = NP\text{-}Completo$?
- Existem problemas que são problemas NP, mas não foi possível até o momento reduzi-los polinomialmente de um problema NP-Completo, nem se conhecem algoritmos polinomiais que os resolvam. Exemplos:
 - o decidir se dois grafos são isomorfos;
 - o saber se um inteiro é primo.

- Esses problemas são fortes candidatos a estar em NP entre P e NP-Completo.
- **Classe NDI** - classe intermediária em NP, que é formada por problemas **mais difíceis que os de P**, mas **menos difíceis que os de NP-Completo**.

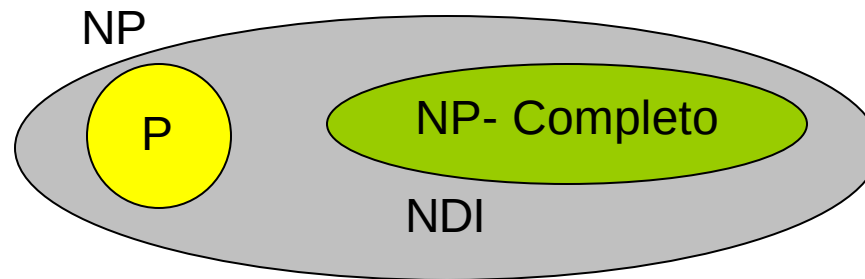


Figura 6.2.12: $NDI = NP - (P \cup NP\text{-Completo})$

- Se $P \neq NP$, então $NDI \neq \emptyset$ porém, não se conhece nenhum problema real em NDI.
- Há ainda problemas que não se sabe se são NP-Difíceis, como o de decidir se uma expressão lógica é ou não uma tautologia.

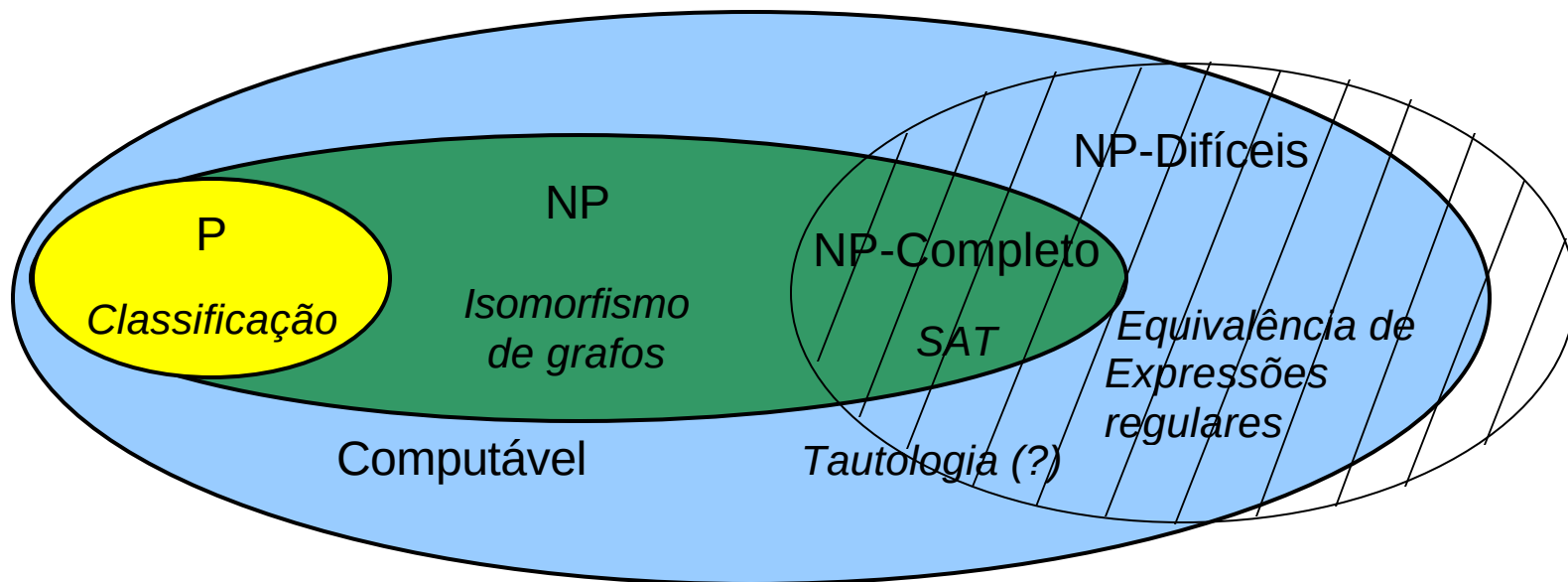


Figura 6.2.13: Exemplos em classes de problemas

Complexidade de Espaço (memória)

- **Classe Pspace**

o classe de problemas resolvidos num espaço polinomial, que requerem um número polinomial em n (tamanho da entrada) de unidades de memória.

o sua relação com NP e Co-NP é ilustrada na figura 6.2.14.

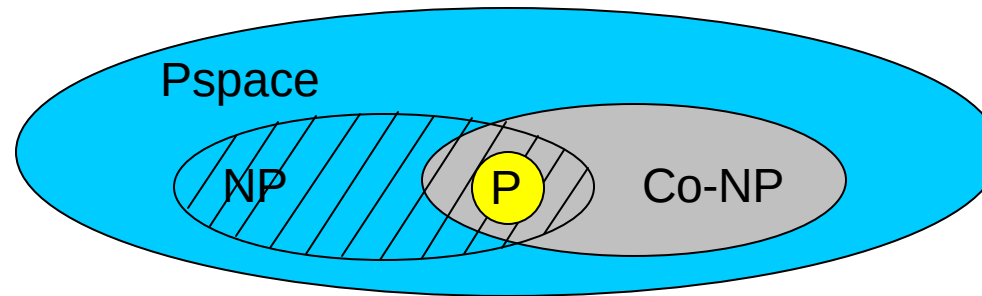


Figura 6.2.14: $\text{Pspace} \supset \text{NP} \cup \text{Co-NP}$

- Questão em aberto: “existe problema resolvido em espaço polinomial que não pode ser resolvido em tempo polinomial?”.
- $\text{Pspace} - (\text{NP} \cup \text{Co-NP}) \neq \emptyset$?
- Definir Co-NP

Relações de inclusão entre as principais classes de problemas

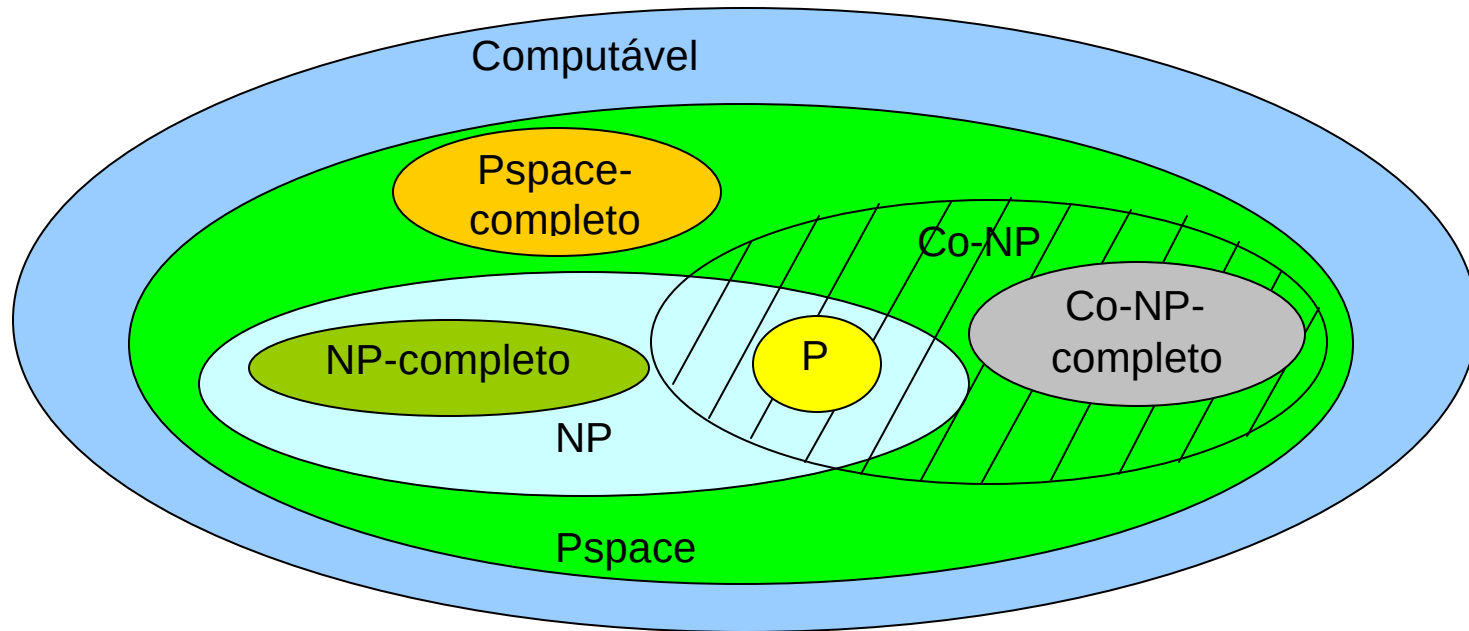


Figura 6.2.15: Hierarquia de Classes de Problemas

- A noção de completude aplica-se a qualquer uma dessas classes de problemas.
- A classe dos **problemas computáveis** é a classe dos problemas para os quais existe algoritmo que o resolva.

6.3 RESUMO DO CAPÍTULO

- O objetivo deste capítulo é estudar a **complexidade inerente ao problema**.
- **Limite inferior** - é um **resultado teórico** que determina não ser possível desenvolver um algoritmo para o problema com complexidade melhor que uma certa função.
- **Limite superior** - refere-se ao **melhor algoritmo conhecido** que resolve um determinado problema.
- Se o **limite inferior é igual ao limite superior**, diz-se que o **problema está fechado** (quanto à complexidade), e a essa função chama-se **complexidade mínima do problema**. Ex.: **Problema de Classificação**, tanto no pior caso como no caso médio, com complexidade mínima $\Omega(n \log n)$.
- **Problema tratável** - tem um **algoritmo polinomial** que o resolve.
- **Problema intratável** - só tem **algoritmos de complexidade exponencial** que o resolve.

- **Problemas de decisão** – se têm **resposta SIM/NÃO**.
- **Problemas de localização** - envolvem a **procura de um elemento que satisfaça a condição** de solução.
- **Problemas de otimização** - requerem uma **qualidade ótima da solução**.
- As mais **importantes classes de problemas**, P, NP e NP-Completa, são definidas para problemas de decisão.
- Algoritmos que resolvem um problema de localização ou de otimização podem ser modificados para resolver problemas de decisão associados. Permitindo que questões como **intratabilidade sejam estudadas no âmbito dos problemas de decisão**.
- Um problema de decisão é dito:
 - o **P** se é resolvível **deterministicamente em tempo polinomial** e
 - o **NP** se é resolvível **não deterministicamente em tempo polinomial**.

- Um algoritmo que faz uso da função escolhe(S) e das instruções falha e sucesso é **não determinístico**.
- A noção de algoritmo é formalizada através do modelo de computação da Máquina de Turing.
 - o Um algoritmo é um programa para um MTD que sempre pára.
 - o Um algoritmo não determinístico é um programa para um MTND que sempre pára.

Teorema 6.2.1: Se $\Pi \in \text{NP}$, então existe um polinômio p tal que Π pode ser resolvido por algoritmo determinístico de complexidade $O(2^{p(n)})$.

- Um problema Π é dito NP-Completo se $\Pi \in \text{NP}$ e, para qualquer outro $\Pi' \in \text{NP}$, $\Pi' \leq \Pi$.
- Os problemas NP-Completos são os **mais difíceis** entre os problemas de NP.

Lema 6.2.3: Se um o problema Π é NP-completo, então $\Pi \in P$ sss $P = NP$.

- A **classe Co-NP** é a classe de todas as linguagens L tais que $L' \in NP$ (L' é o complemento de L).
- Muitas questões envolvendo **relações entre classes** de problemas estão **em aberto**, como:
 - o $P = NP$?
 - o $NP = Co-NP$?
 - o $P = NP \cap Co-NP$?
- O **Teorema de Cook** estabeleceu: “O problema da **"Satisfiability" (SAT)** é **NP-Completo**”.

Teorema 6.2.2: Se Π_1 é NP-Completo, $\Pi_2 \in NP$ e $\Pi_1 \leq \Pi_2$, então Π_2 é NP-Completo.

- Um algoritmo A é dito de **complexidade pseudopolinomial** se sua complexidade é polinomial para o tamanho da entrada, quando esta é codificada em unário. Isto é, a complexidade é polinomial no valor da entrada (e não no tamanho da entrada).
- **NP-Completo-fraco** - problema NP-Completo que admite algoritmo pseudopolinomial.
- **NP-Completo-forte** - problema cuja existência de algoritmo pseudopolinomial implica que $P=NP$.
- Um problema Π é **NP-Difícil** se, para qualquer $\Pi' \in NP$, $\Pi' \leq \Pi$.
- Os problemas NP-Difíceis têm a propriedade de **não serem resolvíveis deterministicamente em tempo polinomial exceto se $P=NP$** , mas não se sabe se são NP ou não. Esses problemas são potencialmente mais difíceis que os NP-Completos.

1ª Lista de Exercícios

1 Descreva e comente com suas próprias palavras, de que forma o estudo da complexidade favorece o desenvolvimento e projeto de algoritmos ?

2 Comente com suas palavras os textos abaixo:

(a) "Com o crescente emprego de tecnologias de virtualização na construção de máquinas conceituais (e.g. Máquina Java), o estudo da complexidade vem recebendo destaque durante o projeto e desenvolvimento dessas tecnologias tanto para as camadas interpretadas como para as traduzidas."

(b) "O crescente avanço tecnológico, permitindo a criação de máquinas cada vez mais rápidas, pode ingenuamente parecer ofuscar a importância da complexidade de tempo de um algoritmo."

3. Um algoritmo é caracterizado como solucionador de um problema, se para qualquer tamanho de entrada, consiga produzir uma resposta correta. Como devem ser abordados os requisitos de tempo e espaço na qualificação dos algoritmos?

4 Faça um paralelo entre os algoritmos de Gauss e Cramer para solução de sistemas de equações lineares, pelo ponto de vista da complexidade (espaço e tempo requeridos), destacando as vantagens e desvantagens de cada um. Descreva e comente com suas próprias palavras.

5 Suponha dois computadores A e B, onde A é obsoleto e B de última geração (frequência do processador de B é 20 vezes maior do que o de A), e um algoritmo P linear executável nos dois computadores. Dado que P leva um tempo T para ser executado totalmente em A, a partir de uma entrada e, quais seriam os tamanhos máximos da entrada de P em B, para que o algoritmo fosse executado no mesmo tempo T em B? Repita o cálculo para complexidade P igual a e^3 , $\log_2 e$, 2^e e 3^e .

6 Supondo que um computador execute uma operação em $2\mu s$, calcule os tamanhos limites de problemas resolvíveis por algoritmos de complexidades e , e^3 , $\log_2 e$, 2^e e 3^e onde e é a entrada, para que sejam executados em 1s, 1min e 1 hora.

7 Uma empresa de software desenvolveu dois sistemas especialistas para serem executados em arquiteturas computacionais iguais para fins de análise de complexidade. Após um estudo concluiu-se que o núcleo do sistema A possuía complexidade proporcional a uma entrada n expressa por $14n^2$ e o núcleo do sistema B complexidade $2n^3$. Decidiu-se então desenvolver um sistema G que selecionasse dinamicamente uma das duas abordagens, para que fosse alcançada a maior eficiência em função de um tamanho de entrada e específico. Quais os valores de e (valor limite), para o qual o sistema G deve selecionar A ou B e alcançar maior eficiência? Explique seu raciocínio.

8 Suponha que uma empresa utiliza um algoritmo de complexidade n^2 que, em um tempo t, na máquina disponível, resolve um problema de tamanho x. Suponha que o tamanho do problema a ser resolvido aumentou 20%, mas o tempo de resposta deve ser mantido. Para isso, a empresa pretende trocar a máquina por uma mais rápida. Qual percentual de melhoria no tempo de execução das operações básicas é necessário para atingir sua meta ?

9 Mostre por indução que $\sum_{i=0}^n i^2 = \frac{1}{6}n(n+1)(2n+1)$

10 Mostre ou dê um contra-exemplo :

(a) Para $n > 0$: $n! \leq n^n$

(b) Para $n > 1$: $n! < n^n$

2ª Lista de Exercícios

1. Explique com suas palavras a afirmativa “a quantidade de trabalho requerido por um algoritmo não pode ser descrita simplesmente por um número”.
2. Defina com suas palavras as notações (ordens) O , Ω e Θ , usadas nas comparações de complexidade assintótica. Exemplifique e utilize gráficos para justificar sua resposta.
3. Explique com suas palavras o que caracteriza a “complexidade no pior caso” e a “complexidade média” no estudo da análise e projeto de algoritmos.
4. Descreva de maneira sucinta as principais medidas de complexidade, destacando de forma objetiva o alvo dessas abordagens.
5. Qual é o critério utilizado para medir a quantidade de trabalho de um algoritmo? De que forma ele é aplicado? Exemplifique.
6. “Analisar o desempenho de algoritmos para entradas específicas não costuma ser uma boa prática no estudo da análise e projeto de algoritmos”. Comente esta afirmativa com suas próprias palavras.
7. O que é uma Cota Assintótica Superior (CAS)? Explique algebricamente quando uma função $f(n)$ é CAS para uma outra função $g(n)$, sendo n as entradas dos algoritmos.
8. Para as funções linear n , quadrática n^2 e exponencial 2^n , verifique que n^2 é CAS para n e 2^n e CAS para n^2 , determinando uma constante n_0 e $\mathbb{N}$ (naturais), tal que para valores não menores que n_0 , a verificação seja suficiente.
9. Faça o esboço de um algoritmo de busca seqüencial de um valor x em uma matriz quadrada de ordem n (duas dimensões iguais), destacando a(s) operação(ões) fundamental(tais) para a análise de complexidade.
10. Dado o algoritmo Mínimo(mat, n) abaixo, onde mat refere-se à uma matriz cúbica de ordem n , pede-se justificando :
 - (a) A(s) operação(ões) fundamental(tais) do algoritmo.
 - (b) A ordem da complexidade assintótica do melhor e do pior caso.

1	Algoritmo Mínimo(mat : 1..n, 1..n, 1..n; n : inteiro):inteiro
2	x, y, z : inteiro;
3	Início
4	Mínimo := mat(1, 1, 1);
5	Para x de 1 a n
6	Início
7	Para y de 1 a n
8	Início
9	Para z de 1 a n
10	Início
11	Se mat(x, y, z) < mínimo então
12	Início
13	Mínimo := mat(x, y, z)
14	Fim
15	Fim
16	Fim
17	Fim
18	Fim

3ª Lista de Exercícios

1. Mostrar que $cP \leq [a0] = \Theta(cP \leq [a1] + cP \leq [a2])$, para $a0 = a1 ; a2$
2. A partir da sua definição, demonstre o princípio da absorção.
3. Verifique nas expressões abaixo, em relação a f e g, se os postulados referentes ao Máximo Assintótico (absorção e melhor cota superior) são satisfeitos:
 - a) $MxAO(n.logn, 2^n) = 2^n$
 - b) $MxAO(n.logn, n^2) = 2^n$
 - c) $MxAO(n, 2^n) = n^2$
 - d) $MxAO(n, n^2) = n.logn$
 - e) $MxAO(n.logn, n^2) = n$
4. Considere o princípio da complexidade pessimista da sequência com preservação (assintótica) de tamanho. Uma vez que ele envolve o conceito de preservação, formule-o precisamente. (e.g. utilize as funções tam, desemp e etc.).
5. Suponha que $desemp[S](d) \leq f(n)$, $desemp[T](S(d)) \leq g(n)$ e $tam(S(d)) \leq s(n)$ sempre que $tam(d) \leq n$. Nestas condições, pode-se garantir que $Cp[S;T](n) \leq f(n) + g(s(n))$, para entradas d com $tam(d) \leq n$?
6. Elabore e analise um algoritmo eficiente que procure pelo maior e menor valor em uma tabela (vetor). (Pesquise por MAX & MIN).
7. Aplique passo a passo, as equações para análise a partir das estruturas algorítmicas, para determinar a ordem de complexidade pessimista do algoritmo Classif_Max abaixo:

Procedimento <u>Classif_Max</u> (A : V)	<i>{classifica vetor em ordem ascendente}</i>
<i>{Entrada: vetor A : V.</i> <i>Saída: vetor A : V (o vetor de entrada classificado em ordem ascendente).}</i>	
<u>sorte</u> V := <u>vetor</u> [1..n] <u>de</u> Elmord ; <u>var</u> {auxiliar} Temp : Elmord {temporária (para troca)} ;	
1. <u>para</u> i <u>de</u> 1 <u>até</u> n - 1 <u>faça</u>	<i>{de 1 a 10}</i>
2. <u>para</u> j <u>de</u> 1 <u>até</u> n - i <u>faça</u>	<i>{de 2 a 9}</i>
3. <u>se</u> A[j] > A[j + 1]	<i>{testa inversão}</i>
4. <u>então</u>	<i>{troca A[j] com A[j + 1]}</i>
5. Temp ← A[j] ;	
6. A[j] ← A[j + 1] ;	
7. A[j + 1] ← Temp ;	
8. <u>fim-se</u> ;	<i>{3: A[j] ← A[j + 1]?}</i>
9. <u>fim-para</u> ;	<i>{2: j de 1 até n - i}</i>
10. <u>fim-para</u> ;	<i>{1: i de 1 até n}</i>
11. <u>retorne-saída</u> (A) ;	<i>{dá saída A}</i>
12. <u>fim-Procedimento</u> .	<i>{fim de Classif_Max: classificação ascendente}</i>